VIRTUAL CONTROL — VMWARE CLOUD FOUNDATION SOLUTIONS

# VCF Automation (VCFA) 9.0.1

## Deployment & Troubleshooting Compendium

Consolidated reference — guide, sizing, pre-deploy prep, deploy, prechecks, monitoring, RCA, cleanup · v1.0 · June 16, 2026 · 

# VCF Automation (VCFA) 9.0.1 — Deployment & Troubleshooting Compendium

**Prepared by:** Virtual Control LLC
**Date:** June 16, 2026
**Document Version:** 1.0
**Classification:** Internal

**This compendium consolidates nine VCFA 9.0.1 deployment and troubleshooting documents into one reference.** Each former document is preserved in full as a numbered chapter below; content is verbatim from the sources (no re-interpretation). Chapters proceed from planning and sizing through deploy, prechecks, monitoring, root-cause analysis, and cleanup. *(VCFA — the VCF-integrated Automation domain — is distinct from standalone Aria Automation 8.18, which is documented separately.)*

## Contents

1. VCF Automation 9.0.1 — Deployment Guide
2. Deploy — Airtight Plan
3. Nested-ESXi RAM Rebalance (making room)
4. Pre-Deploy Cleanup + Fleet Cert SSH-Replace
5. Simple Deploy Wizard — Precheck Gotchas
6. Deployment — Verified Fix Runbook
7. Deploy Monitoring & Health Checks
8. Root-Cause Analysis & Troubleshooting Report
9. Failed-Deployment Cleanup Runbook

---

## 1. VCF Automation 9.0.1 — Deployment Guide

*Source: VCF_Automation_Deployment_Guide.md*

**Prepared by:** Virtual Control LLC **Date:** May 9, 2026 **Document Version:** 2.0 (reconstructed from PDF)
**Classification:** Internal — Lab Environment

> **About this reconstruction.** This document was reconstructed from a PDF that had no Markdown source. The body below is the verbatim extracted text — page headers, page numbers, and file URL artifacts have been stripped, but **fixed-width formatting may not be perfect** and **section headings are not yet promoted to Markdown H2/H3 syntax**. Treat this as an editable starting point — clean up specific sections as needed. The PDF and HTML companion files are regenerated from this Markdown source going forward.

---

```
                          VCF Automation

                    VMware Cloud Foundation 9.0.1 Lab Environment
                    Deployed via VCF Operations Fleet Management

VCF Version:          9.0.1 (SDDC Manager 9.0.1.0)
vCenter:              vcenter.lab.local / 192.168.1.69
SDDC Manager:         sddc-manager.lab.local / 192.168.1.241
NSX Manager VIP:      nsx-vip.lab.local / 192.168.1.70
VCF Operations:       vcf-ops.lab.local / 192.168.1.77
VCF Ops for Logs:     logs.lab.local / 192.168.1.242
Fleet Manager:        fleet.lab.local / 192.168.1.78
VCF Automation FQDN:  automation-node1.lab.local
VCF Automation IP:    192.168.1.91
Generated:            March 30, 2026

Source: Broadcom Official Documentation (techdocs.broadcom.com)

Words You Need to Know

Read this list before you start. Come back any time you see a word you do not remember.

Word                   What It Means
VCF Automation
                       The new name for Aria Automation in VCF 9. It lets people deploy VMs by
Fleet Manager          filling out a form on a web page instead of manually creating them in
                       vCenter.
VCF Operations
Binary                 The appliance that manages the lifecycle (install, upgrade, patch) of VCF
Depot                  management components. Its old name was Aria Suite Lifecycle or
FQDN                   vRealize Lifecycle Manager (vRLCM).
VIP
Cluster Node IP Pool   The monitoring and management platform for your VCF environment. Fleet
Internal Cluster CIDR  Manager is accessed through VCF Operations.

Precheck               A software installation file. You download the VCF Automation binary
Day-N Operation        (installer) before you can deploy it.

                       A storage location where software binaries are kept. Can be online
                       (Broadcom servers) or offline (your own HTTPS server).

                       Fully Qualified Domain Name. The full address of a server, like
                       automation-node1.lab.local.

                       Virtual IP address. The main IP address you use to access VCF
                       Automation. It must have a DNS record pointing to the FQDN.

                       A set of IP addresses reserved for the VCF Automation appliance nodes to
                       communicate internally and for rolling upgrades.

                       An IP address range used for internal communication between VCF
                       Automation components. Must not overlap with any existing networks in
```

| | |
|---|---|
| | your environment. |
| | A validation step that checks all your settings before the actual deployment starts. Every precheck must pass before you can click Submit. |
| | Something you do after the initial VCF deployment. Deploying VCF Automation after VCF is already running is a Day-N operation. |

Table of Contents


Words You Need to Know
1. How VCF Automation Deployment Works
2. Environment Summary
3. Prerequisites

      3.1 Create DNS Records
      3.2 Verify DNS Resolution
      3.3 Fix NSX Admin Password
      3.4 Verify NTP Synchronization
      3.5 Appliance Sizing Requirements
4. Download VCF Automation Binary
      4.1 Log Into VCF Operations
      4.2 Configure the Depot
      4.3 Download the Install Binary
5. Deploy VCF Automation
      5.1 Navigate to the VCF Automation Tile
      5.2 Installation Configuration
      5.3 Certificate Configuration
      5.4 Infrastructure Properties
      5.5 Network Configuration
      5.6 Component Information
      5.7 Run Precheck
      5.8 Submit and Monitor
6. Post-Deployment Verification
      6.1 Access the VCF Automation UI
      6.2 Verify Services
      6.3 Set Up Provider Infrastructure
7. Known Issues
8. Removed Features in VCF 9
9. Verification Checklist
10. Official Documentation Sources


## 1. How VCF Automation Deployment Works

In VCF 9, the product formerly known as Aria Automation is now called VCF Automation. The way you deploy it has also changed. Here is what you need to know:

You deploy VCF Automation through VCF Operations Fleet Management. You do NOT use SDDC Manager. The SDDC Manager UI is deprecated in VCF 9.

Here is the path you will follow:
VCF Operations  Fleet Management  Lifecycle  Overview  VCF Automation tile  Add

VCF Automation runs on containers inside a Kubernetes cluster that is deployed on dedicated virtual appliances. When you click Submit, Fleet Management automatically builds the Kubernetes cluster and deploys all the services inside it.

There are two ways to deploy VCF Automation:

| Method | When to Use |
|---|---|
| VCF Installer (Day-0) | When deploying VCF for the first time. VCF Automation is included in the initial deployment wizard. |
| Fleet Management (Day-N) | When VCF is already running and you want to add VCF Automation after the fact. This is what we are doing. |

Only one VCF Automation instance is supported per VCF fleet.

## 2. Environment Summary

These components are already deployed and running in the VCF 9.0.1 lab environment:

| Component | FQDN | IP Address |
|---|---|---|
| SDDC Manager | sddc-manager.lab.local | 192.168.1.241 |
| vCenter Server | vcenter.lab.local | 192.168.1.69 |
| NSX Manager VIP | nsx-vip.lab.local | 192.168.1.70 |
| VCF Operations | vcf-ops.lab.local | 192.168.1.77 |
| VCF Ops for Logs | logs.lab.local | 192.168.1.242 |
| Fleet Manager | fleet.lab.local | 192.168.1.78 |
| DNS Server | -- | 192.168.1.230 |
| NTP Server | -- | 192.168.1.230 |

ESXi Hosts (4 total):

| Host | IP Address |
|---|---|
| ESXi Host 1 | 192.168.1.74 |
| ESXi Host 2 | 192.168.1.75 |
| ESXi Host 3 | 192.168.1.76 |
| ESXi Host 4 | 192.168.1.82 |

VCF Automation Target Configuration:

| | |
|---|---|
| FQDN: | automation-node1.lab.local |
| Primary VIP: | 192.168.1.91 |
| Deployment Model: | Simple (single node) |
| Licensing: | Evaluation mode |



## 3. Prerequisites

Complete every step in this section before you start the deployment. If you skip a step, the deployment will fail.

### 3.1 Create DNS Records

The Primary VIP for VCF Automation must resolve to the FQDN via DNS. Create these records on your DNS server (192.168.1.230):

| Record Type | Name | Value |
|---|---|---|
| A Record | automation-node1.lab.local | 192.168.1.91 |
| PTR Record | 91.1.168.192.in-addr.arpa | automation-node1.lab.local |

The Primary VIP MUST resolve to the FQDN. Cluster Node IPs do NOT require DNS records.

### 3.2 Verify DNS Resolution

Forward Lookup:

```
nslookup automation-node1.lab.local
```

Expected result: 192.168.1.91

Reverse Lookup:

```
nslookup 192.168.1.91
```

Expected result: automation-node1.lab.local

Do not proceed until both lookups return correct results.

### 3.3 Fix NSX Admin Password

The NSX admin password is expired. VCF Automation needs NSX connectivity. Fix this before deploying.

Steps:

1. SSH to the NSX Manager node:

```
ssh admin@192.168.1.71
```

2. The system will tell you the password is expired and prompt you to set a new one
3. Enter your current password, then the new password twice
4. Update the credential in VCF Operations (not SDDC Manager)

### 3.4 Verify NTP Synchronization

All VCF components must be time-synchronized. The NTP server is 192.168.1.230. Verify synchronization on SDDC Manager, vCenter, NSX Manager, and all ESXi hosts.

### 3.5 Appliance Sizing Requirements

VCF Automation has a fixed appliance size per node. There is no t-shirt sizing option like VCF Operations. You cannot shrink the appliance after deployment.

| Resource | Per Node | Notes |
|---|---|---|
| vCPU | 24 | Fixed -- cannot be changed |
| Memory | 96 GB RAM | Fixed -- cannot be changed |
| Cluster Node IPs | Minimum 2 | Simple model: 2 IPs minimum for node pool |

| Deployment Models: | Nodes | Use Case |
|---|---|---|
| Model | 1 node | Lab, POC, evaluation, development |
| Simple (Small) | 3 nodes | Production, mission-critical |
| HA (Medium) | 3 nodes | Production, large scale |
| HA (Large) | | |

For this lab we will use the Simple (Small) model with 1 node. This requires 24 vCPU and 96 GB RAM. You can scale
out to HA later if needed.



## 4. Download VCF Automation Binary

Before you can deploy VCF Automation, you need to download the install binary (the installer file) to Fleet
Manager. This is done through Binary Management.

### 4.1 Log Into VCF Operations

Open a browser and go to:

```
https://vcf-ops.lab.local
```

Log in with your admin credentials.

### 4.2 Configure the Depot

Navigation Path:

```
Fleet Management  Lifecycle  VCF Management  Depot Configuration
```

You need a depot configured so Fleet Manager knows where to download the binary from. There are two options:

| Depot Type | How It Works |
|---|---|
| Online Depot | Downloads directly from Broadcom servers. Requires a download token from the Broadcom Support Portal. |
| Offline Depot | Downloads from a local HTTPS server that you host. Used for air-gapped (no internet) environments. |

For Online Depot: You need a download token from the Broadcom Support Portal. Log into the portal, navigate to VMware Cloud Foundation, go to Quick Links, and click Generate Download Token.

### 4.3 Download the Install Binary

Navigation Path:

```
     Fleet Management  Lifecycle  Binary Management

Steps:
       1. Select binary type: Install (not Upgrade)
       2. Choose the VCF version (9.0.x)
       3. Select the VCF Automation component
       4. Click Download
       5. Wait for the download to complete

     You MUST download Install binaries, not Upgrade binaries. If the version dropdown is empty when you
create the
     deployment plan later, it means the wrong binary type was downloaded.

5. Deploy VCF Automation

Now that the binary is downloaded, you will deploy VCF Automation through the Fleet Management Lifecycle
interface.

5.1 Navigate to the VCF Automation Tile

Navigation Path:
     VCF Operations  Fleet Management  Lifecycle  Overview tab

Find the VCF Automation tile on the Overview page and click Add.

5.2 Installation Configuration

Installation Type:              New Install

Version:                        9.0.0.0 (or latest available)

Deployment Type:                Small (single node, lab appropriate)

5.3 Certificate Configuration

You need a certificate for VCF Automation. You can create a new one or select an existing one.

To create a new certificate:
       1. Click the "+" (Add Certificate) button
       2. Enter the FQDN: automation-node1.lab.local
       3. The Common Name must include the FQDN
       4. The Subject Alternative Name (SAN) must include the FQDN

5.4 Infrastructure Properties

Tell Fleet Manager where to deploy the VCF Automation VM:

Setting              What to Select
vCenter              vcenter.lab.local (Management Domain vCenter)
Cluster              Your management cluster
Folder               (Optional) Select a VM folder
Resource Pool        (Optional) Select a resource pool
Network              Management network
Datastore            Your vSAN datastore

Virtual Control LLC                         Page 9                   VCF Automation Deployment Guide

5.5 Network Configuration

Domain Name:               lab.local

Domain Search Path:        lab.local

DNS Servers:               192.168.1.230

NTP Servers:               192.168.1.230

IPv4 Gateway:              192.168.1.1
```

```
IPv4 Netmask:              255.255.255.0

For DNS and NTP, click "Edit Server Selection" to select or add the server addresses.

5.6 Component Information

This is the most important page. Fill in these values carefully:

Component Properties:

FQDN:                                    automation-node1.lab.local

Certificate:                             (Prepopulated from Step 5.3)

vmware-system-user                       (Choose a strong password, minimum 15 characters)
Password:

Provider Admin Password:                 (Choose a strong password, minimum 15 characters)

Load Balancer:

Choose Internal (the default). This means the native load balancer is configured automatically. The FQDN
will
resolve to the Primary VIP.

Cluster Configuration:        Value                    Explanation
            Setting
            VM Node Prefix    (Choose a unique prefix) Used to name the VCF Automation node VMs.
                                                       Must be unique in the cluster.
            Primary VIP
                              192.168.1.91             The main IP address for accessing VCF
            Internal Cluster                           Automation. Must resolve to the FQDN via DNS.
            CIDR
                              (Enter a non-conflicting An IP range for internal communication. Must
            Cluster Node IP   range)                   NOT overlap with any existing networks. Example:
            Pool                                       10.244.0.0/16

                              (Enter minimum 2 IPs)    IPs for node communication and rolling upgrades.
                                                       Must be on the same subnet.

Passwords must be minimum 15 characters. The Internal Cluster CIDR must not conflict with any network in
your
environment.

5.7 Run Precheck

Click "Run Precheck" to validate all your settings. The system will check that:
       - DNS resolves correctly
       - The IP addresses are reachable and not in use
       - The certificate is valid
       - The infrastructure has enough resources (CPU, RAM, storage)
       - The binary is available

Virtual Control LLC                                Page 11                 VCF Automation Deployment
Guide

     Every precheck must pass before you can submit. If any check fails, fix the issue and run the
precheck again.

5.8 Submit and Monitor

Once all prechecks pass, click Submit to start the deployment.

The deployment takes approximately 1 hour. During this time, Fleet Manager will:
       - Deploy the VCF Automation virtual appliance
       - Build a Kubernetes cluster on the appliance
       - Deploy all VCF Automation services as containers
       - Configure networking and load balancing
       - Register with VCF Operations inventory
```

Do not close the browser or interrupt the process. You can monitor progress in the Fleet Management request tracking.

6. Post-Deployment Verification

6.1 Access the VCF Automation UI

Open a browser and go to:
https://automation-node1.lab.local

Log in with the provider admin credentials you configured during deployment.

6.2 Verify Services

After logging in, verify that all services are healthy. All service tiles on the dashboard should show green status.

6.3 Set Up Provider Infrastructure

After deployment, set up the provider infrastructure. According to Broadcom KB 426100, the recommended journey is:

| Phase | What You Do |
|---|---|
| 1. Initialize provider infrastructure | Configure the infrastructure that VCF Automation will manage (vCenter, NSX integration) |
| 2. Create tenant organizations | Set up organizations for users who will request VMs |
| 3. Configure IaaS self-service | Design cloud templates (blueprints) and publish catalog items |
| 4. Test deployment | Deploy a test VM from a catalog item to verify everything works |

Licensing: VCF Automation is licensed automatically when connected to a vCenter that has a VCF license assigned from VCF Operations. You do not need to enter a separate license key.



7. Known Issues

| Issue | Impact | Resolution |
|---|---|---|
| NSX admin password expired | VCF Automation cannot connect to NSX | SSH to NSX Manager (192.168.1.71), reset password, update in VCF Operations |
| vSAN at ~81% utilization | VCF Automation VM needs significant resources | ~446 GB free is sufficient; monitor during deployment |
| Evaluation licensing | 60–90 day evaluation period | Sufficient for lab and learning |
| Appliance requires 24 vCPU, 96 GB RAM | Large resource requirement for a lab | Ensure hosts have enough available capacity |
| Fleet Manager service not responding | Backend Java process (port 8080) may hang | Check with: curl http://127.0.0.1:8080/lcm/bootstrap/api/status |
| Offline depot HTTPS server not running | Fleet Manager cannot retrieve binaries or patches | Start the HTTPS depot server on the host machine |

8. Removed Features in VCF 9

The following features from Aria Automation 8.x have been removed in VCF Automation 9.0. Do not expect

them to be available:

| Removed Feature |
| --- |
| VCF Automation Pipelines (Code Stream) |
| VMware Update Manager (VUM) plug-in |
| NSX-V and NSX-T cloud account integration |
| Avi Load Balancer functionality |
| Kubernetes integrations (TKG, PKS, OpenShift, TMC) |
| Cloud Consumption Interface (CCI) |
| Virtual Private Zone and provider image management |
| VMware Cloud Director integration |
| Storage DRS functionality |

Source: techdocs.broadcom.com -- VCF Automation Product Support Notes

9. Verification Checklist

Use this checklist to confirm a successful deployment:

| ## | Verification Step | Expected Result |
| --- | --- | --- |
| 1 | nslookup automation-node1.lab.local | Returns 192.168.1.91 |
| 2 | nslookup 192.168.1.91 | Returns automation-node1.lab.local |
| 3 | Browse to https://automation-node1.lab.local | VCF Automation login page loads |
| 4 | Log in with provider admin credentials | Dashboard displays with green service tiles |
| 5 | VCF Automation appears in Fleet | Shows as deployed component with version |
| | Management Lifecycle | |
| 6 | Initialize provider infrastructure | vCenter and NSX connected |
| 7 | Create a basic cloud template | Template saves without errors |
| 8 | Deploy a test VM from catalog | VM deploys and powers on successfully |



10. Official Documentation Sources

Every step in this guide is based on the following official Broadcom documentation:

| Document | Location |
| --- | --- |
| Deploying VCF Automation as a Day-N Operation | techdocs.broadcom.com VCF 9.0 Fleet Management Install VCF Automation |
| VCF Automation Deployment Models | techdocs.broadcom.com VCF 9.0 Design VCF Automation Models |
| Binary Management for VCF Components | techdocs.broadcom.com VCF 9.0 Lifecycle Management Binary Management |
| Connect to an Online Depot | techdocs.broadcom.com VCF 9.0 Lifecycle Management Configure Online Depot |
| KB 426100: Recommended Deployment Journey | knowledge.broadcom.com/external/article/426100 |
| VCF Automation Product Support Notes | techdocs.broadcom.com VCF 9.0 Release Notes Product Support Notes |

```
Scale VCF Automation            techdocs.broadcom.com  VCF 9.0  Fleet Management
                                 Scaling

VMware Cloud Foundation 9.0 Documentation Home:
techdocs.broadcom.com/us/en/vmware-cis/vcf/vcf-9-0-and-later/9-0.html

Virtual Control LLC             Page 17                                          VCF Automation
Deployment Guide
```

---



---

## 2. Deploy — Airtight Plan

*Source: VCFA-Deploy-Airtight-Plan-2026-05-12.md*

**Prepared by:** Virtual Control LLC **Date:** May 12, 2026 **Document Version:** 1.0 (review-only; no execution) **Classification:** Internal — Lab/Operations **Companion Doc:** `VCFA-Deployment-Fix-Runbook-2026-05-08` (operational runbook this plan supersedes for pre-flight)

> **Source-tag conventions used throughout this plan:**
>
> - BROADCOM-DOC — a Broadcom KB or TechDocs URL directly authorizes the step. Considered authoritative.
> - LAB-VERIFIED — we have made the step work, Broadcom has not published it. Use with explicit acknowledgment that we lack a Broadcom escalation path if the step fails.
> - HARD BLOCKER — must resolve before any deploy submit. Broadcom-mandated.

---

### 1. Pre-Flight — Hard Blockers (Broadcom-Mandated)

#### *HB-1. vCenter must move OFF the standard vSwitch (TempMgmt) back onto a Distributed Switch port group BROADCOM-DOC HARD BLOCKER*

- KB 420920 — Import vCenter to VCF Instance fails when vCenter is on a standard switch
- KB 430625 — Upgrading to VCF 9.0 requires distributed switch

**Why this is non-negotiable:** Both KBs gate VCF lifecycle operations on vCenter being on a DVS. Standard vSwitch placement (current "TempMgmt" port group via `vmnic2` on esxi02) is unsupported. Any VCFA deploy attempt will either be blocked at precheck or fail with non-recoverable state.

#### *HB-2. Management port group must be ephemeral, not static-binding BROADCOM-DOC HARD BLOCKER*

- KB 324492 — Static (non-ephemeral) or ephemeral port binding on a vSphere Distributed Switch
- KB 426300 — Moving vCenter Server to an ephemeral port group on a vSphere Distributed Switch
- Broadcom design decision 81611 (legacy ID) recommends ephemeral binding for the management/vCenter port group.

**Why:** Static binding cannot be reconfigured without a running vCenter — this is the exact chicken-and-egg that took down our cluster on 2026-05-11 when vCenter VM needed a DVPort and the running vCenter (the only thing

authorized to allocate it) was down. Our existing `pg-vm-mgmt` is static-binding and is the anti-pattern Broadcom warns against.

#### *HB-3. Fleet certificate SAN must include the target VCFA FQDN BROADCOM-DOC HARD BLOCKER*

- KB 411354 — VCF Automation 9 installation through Fleet fails (LCMVMSP10002)

**Broadcom's documented root cause for `LCMVMSP10002` / `VmspPkgPushTask "No failed pods found"` is Fleet certificate SAN mismatch.** Resolution per the KB: regenerate cert with correct SAN, verify Fleet Management shows "Connected" in vcf-ops admin UI, verify Certificates+Passwords populated, retry.

Note: We had previously identified a separate `/etc/hosts` + systemd-resolved workaround for LCMVMSP10002 in our `reference_vcfa_systemd_resolved_break.md` memory. That remains LAB-VERIFIED only; Broadcom's published root cause is cert SAN. **Verify cert SAN first.**

#### *HB-4. DRS must be Fully Automated BROADCOM-DOC HARD BLOCKER*

- TechDocs — VCF Automation HA Deployment Model

**Why:** Broadcom's VCF Automation deployment design mandates DRS Fully Automated for any LCM task. This conflicts with our earlier lab guidance ("avoid vMotion during deploy"); Broadcom doc wins. Do NOT disable DRS.

---

## 2. Phase A — Network Repair

Cluster prerequisite: previous Phase 1 rebalance must be complete ( `data-move = 0` , `GB-left = 0` , all 162 objects `healthy` ). Do not start Phase A while rebalance is in flight.

| Step | Action | Source | Notes |
|---|---|---|---|
| A1 | Verify cluster rebalance complete ( `esxcli vsan debug object health summary get` , `esxcli vsan debug resync summary get` from esxi02 — both must show 0/0) | BROADCOM-DOC KB 389599 | Hard gate before any network change |
| A2 | Create new ephemeral port group on `vcenter-cl01-vds01-new` (e.g. `pg-mgmt-ephemeral` ). VLAN, MTU, teaming must match `pg-vm-mgmt` | BROADCOM-DOC KB 324492 | One new port group instead of converting pg-vm-mgmt is the lower-risk path |
| A3 | Power off vcenter VM | Standard | NIC binding change cannot be live-applied to running vCenter |
| A4 | Edit `vcenter.vmx` on the vSAN datastore: replace `ethernet0.networkName = "TempMgmt"` with the DVS-style lines `ethernet0.dvs.switchId = "<DVS UUID>"` + `ethernet0.dvs.portgroupId = "<new ephemeral DVPG ID>"` | BROADCOM-DOC KB 426300 | Get the DVS UUID and the new DVPG ID from `net-dvs -l` on esxi02 |
| A5 | Power on vcenter | Standard | Verify ping `.69` and full UI |
| A6 | Confirm vCenter is on the new ephemeral DVPG, not standard vSwitch | BROADCOM-DOC KB 420920 | `esxcli network vm port list` on esxi02 should show vcenter on `pg-mgmt-ephemeral` |
| A7 | (Optional) Convert existing `pg-vm-mgmt` from static to ephemeral | BROADCOM-DOC KB 324492 | Not strictly required if A2 created a new one. |

| Step | Action | Source | Notes |
|---|---|---|---|
| | | | Documented as non-destructive. |
| A8 | Remove `vSwitchTemp` standard switch + `TempMgmt` portgroup on esxi02 | BROADCOM-DOC standard esxcli | Only after A6 confirmed |

**Decision needed (#1):** Convert existing `pg-vm-mgmt` to ephemeral, OR create a new `pg-mgmt-ephemeral` and leave `pg-vm-mgmt` alone? Both supported by Broadcom; new-PG is lower risk.

---

## 3. Phase B — Cleanup

| Step | Action | Source | Notes |
|---|---|---|---|
| B1 | **Done** — Delete `vcfa-mgmt-wsv57` VM (executed 2026-05-12) | BROADCOM-DOC TechDocs VM lifecycle | `vim-cmd vmsvc/destroy 13` on esxi04 |
| B2 | Verify no other `vcfa-*` / `vcfa-mgmt-*` / `wsv*` VMs anywhere in inventory | Operational | Already verified clean today |
| B3 | Power on `sddc-manager` (currently powered off on esxi04) | Standard | Wait 5–10 min for full boot; check UI |
| B4 | Verify SDDC Manager DB clean — no stuck tasks, no stale locks, NSX in ACTIVE state | LAB-VERIFIED (Undocumented Issues #1–7) | SQL: `SELECT * FROM task_metadata WHERE resolved=false; SELECT * FROM lock; SELECT id, state FROM nsxt;` |
| B5 | Verify NSX Manager: connected to vCenter, all services running, Skyline checks green | BROADCOM-DOC KB 405048 | NSX was "(disconnected)" in vCenter UI earlier; verify recovered |
| B6 | Verify VCF Operations: UI loads, all 4 nodes healthy, Collector reporting | BROADCOM-DOC TechDocs | |
| B7 | Clean LCM DB orphans on Fleet appliance — 6-table delete in dependency order | LAB-VERIFIED | Tables: `vm_lcops_node_properties`, `vm_lcops_product_nodes`, `vm_lcops_product_properties`, `vm_lcops_product` (where id NOT IN ('vrops','vrli')), `vm_lcops_infrastructure_properties`, `vm_lcops_environments`. **Broadcom KB 388305 covers Networks/Logs only, NOT Automation cleanup.** |
| B8 | Restart `vmware-vcops-web` on VCF Ops appliance (clears the frontend parallel-deploy cache) | LAB-VERIFIED | `systemctl restart vmware-vcops-web` |

| Step | Action | Source | Notes |
|---|---|---|---|
| B9 | Validate locker credentials — cross-check `vcUsername`, NSX root, SDDC admin against current locker UUIDs | BROADCOM-DOC TechDocs lookup_passwords | Reconcile-after-redeploy gotcha ( `feedback_validate_credentials_before_deploy` ) |
| B10 | Verify Fleet certificate SAN includes target VCFA FQDN (e.g. `vcfa.lab.local` if that's the chosen FQDN) | BROADCOM-DOC KB 411354 | `openssl s_client -connect fleet.lab.local:443 -showcerts \| openssl x509 -noout -text \| grep -A2 "Subject Alternative Name"` |
| B11 | Verify NTP healthy on all components | BROADCOM-DOC KB 421274 | NTP drift causes deploy failures. Check `vmware-toolbox-cmd stat hosttime` inside each appliance |

---

## 4. Phase C — Sizing Decision (Critical)

> **Broadcom only documents two VCF Automation deployment models:**
>
> 1. **Simple deployment** — 1 node
> 2. **HA deployment** — 3 nodes
>
> **"Medium" is not a Broadcom-documented term.** Our `reference_vcfa_sizing_fixed.md` memory note (24 vCPU / 96 GB per node, memory hard-required) is lab/community-verified only.

VCF Automation HA Deployment Model — TechDocs VCF Automation Vertical Scale-Up — TechDocs

| Configuration | Resource ask | Fits this homelab? | Risk |
|---|---|---|---|
| **Simple** (1 node, 24 vCPU / 96 GB) | 96 GB total | Yes — fits on a single 90 GB host with no other VMs, or comfortably across cluster | Low |
| **HA** (3 node, 24 vCPU / 96 GB each = 288 GB) | 288 GB total | Marginal — cluster total 360 GB; existing VMs (vCenter, fleet, vcf-ops, collector, sddc-manager, nsx-manager) consume ~120 GB; leaves ~240 GB for VCFA | High — tight memory could trigger kubelet OOM during VCFA cluster init |

**Decision needed (#2):** Simple or HA?

**Decision needed (#3):** If HA, which VMs to power off during deploy to free memory? ( `collector` is a candidate — non-critical observability appliance.)

---

## 5. Phase D — Pre-Flight Gates

All gates must be GREEN before deploy submit. If any gate fails, stop and remediate before proceeding.

| # | Gate | Condition | Source |
|---|---|---|---|
| D 1 | All 4 hosts pingable + SSH-responsive | All 4 UP | Operational |
| D 2 | vSAN: `inaccessible=0`, no reduced-availability, all 162 objects `healthy`, `data-move=0`, `GB-left=0` | Clean | BROADCOM-DOC KB 389599 |
| D 3 | vSAN datastore: ≥30% free capacity per host | Headroom | BROADCOM-DOC TechDocs vSAN sizing |
| D 4 | Cluster Store / etcd port 2379 healthy on all 4 hosts | `clusterAdmin status` returns OK | LAB-VERIFIED (no Broadcom KB) |
| D 5 | vCenter on ephemeral DVPG, NOT TempMgmt | Confirmed via `esxcli network vm port list` | BROADCOM-DOC KB 420920 |
| D 6 | NSX Manager: Connected, Skyline Green, all services running | `get services` shows all running; UI loads | BROADCOM-DOC KB 405048 |
| D 7 | SDDC Manager: API responds 200; DB clean (no locks, no IN_PROGRESS tasks); NSX state ACTIVE | Per Undocumented Issues #1–7 | LAB-VERIFIED + BROADCOM-DOC KB 422426 (DB cleanup mentions) |
| D 8 | Fleet Manager: cert SAN OK; all services running; lookup_passwords healthy | | BROADCOM-DOC KB 411354 |
| D 9 | DRS: Fully Automated | vCenter UI → Cluster → Configure → vSphere DRS | BROADCOM-DOC (HA model doc) |
| D 10 | No in-flight tasks anywhere | `vim-cmd vimsvc/task_list` clean on every host | Operational |

---

## 6. Phase E — Submit Deploy

Two paths. Both have caveats.

### *E-Path-1: UI deploy BROADCOM-DOC (Recommended by Broadcom)*

- Recommended deployment journey for VCF Automation in VCF 9.0 — KB 426100

**Pros:** Broadcom-documented path; if this fails Broadcom support has escalation guidance.

**Cons:** We've hit the "Parallel Deployments of automation and identity broker are not allowed" lock via UI multiple times. Phase B7 + B8 cleanup may or may not fully clear that. Broadcom KB 388305 does NOT cover this scenario — it's the VCF Operations Networks/Logs equivalent.

### *E-Path-2: API deploy LAB-VERIFIED*

- Endpoint: `POST /lcm/lcops/api/v2/environments` on Fleet
- Critical structure: `ipPool`, `primaryVip`, `internalClusterCidr` must go in `products[0].nodes[0].properties` NOT `products[0].properties`

**Pros:** Bypasses the UI's frontend parallel-deploy gate (which lives in `vmware-vcops-web`).

**Cons:** Not Broadcom-documented. The closest Broadcom reference is the SDDC Manager API `VcfAutomationSpec` (different endpoint — `developer.broadcom.com/xapis/sddc-manager-api/latest/data-structures/VcfAutomationSpe`

`c/` ). If the deploy fails via this path we may not have authoritative recovery guidance.

**Decision needed (#4):** UI or API path?

---

## 7. Phase F — Known-Issue Reaction Plan

| Symptom | Broadcom response (first) | Lab fallback (if Broadcom path fails) |
|---|---|---|
| `LCMVMSP10002` / `VmspPkgPushTask "No failed pods found"` | BROADCOM-DOC KB 411354 — regenerate Fleet cert with correct SAN, verify Fleet "Connected" in vcf-ops admin UI, retry | LAB-VERIFIED add `/etc/hosts` entries on VCFA appliance for unresolvable FQDNs; then click Retry |
| Deploy stalls at "Failed to init fcd-catalog" / stale `catalog` namespace | BROADCOM-DOC KB 330164 (FCD catalog) + KB 320790 (mkdir procedure) + KB 428195 (objtool for inaccessible objects) | LAB-VERIFIED `objtool delete -u <UUID>` then `mkdir` from owner host. **Note: KB 382405 referenced in prior memory is not findable today — re-verify KB number before quoting.** |
| etcd / kube-apiserver PostStartHook timeout | None directly | Verify vSAN write latency <10 ms (should be fine post Phase 1 all-flash fixes). If recurring, check ESXi host CPU/memory contention. |
| "Parallel Deployments of automation and identity broker are not allowed" | BROADCOM-DOC KB 388305 covers Networks/Logs only — does NOT cover Automation/Identity Broker | LAB-VERIFIED LCM DB cleanup (Phase B7) + `vmware-vcops-web` restart (Phase B8) |
| Cluster Store / `clusterAdmin runReplica` exit:2 on a host | None directly | LAB-VERIFIED manual MM bypass via `esxcli system maintenanceMode set --enable true`. Should NOT be needed during VCFA deploy — only during host MM operations. |
| vSAN reduced-availability after deploy | BROADCOM-DOC KB 389599 for inaccessible objects; vSAN UI "Repair Objects Immediately" for the documented user trigger | **Per 2026-05-11 incident: do NOT set `ClomRepairDelay` to 0 or run `Repair Objects Immediately` aggressively. Let the 60-min default delay timer expire and let vSAN auto-repair at standard pace.** |

---

## 8. Honest Assessment — Where Broadcom Coverage Has Gaps

Six of our routine lab procedures do NOT have Broadcom KBs:

1. **LCM DB 6-table cleanup** (Phase B7) — KB 388305 covers Networks/Logs only; Automation cleanup is undocumented
2. `vmware-vcops-web` **restart for parallel-deploy lock** (Phase B8) — not in any Broadcom KB
3. `ipPool` **placement in** `nodes[0].properties` for the Fleet `/lcm/lcops/api/v2/environments` endpoint (Phase E-Path-2) — the SDDC Manager API doc shows a similar structure but Broadcom does not document the Fleet endpoint with this exact placement
4. `/etc/hosts` **+ systemd-resolved workaround for LCMVMSP10002** (Phase F) — Broadcom says cert SAN; we say (additionally) DNS resolution on appliance
5. **KB 382405 (FCD catalog)** — exact KB number not findable in current Broadcom search; closest are KB 330164 / KB 320790 / KB 428195
6. **Trust Manager Bundle CA injection** for VCFA-trusting-Fleet self-signed certs — this is a Kubernetes `cert-manager.io` pattern, not Broadcom-documented for VCFA specifically

**Operating principle:** When the deploy hits any of these, we are using lab knowledge that has worked before. Broadcom support will not necessarily recognize the workaround.

## 9. Highest Failure Risks (Honest Triage)

In order of likelihood:

1. **HA sizing on this 4-host nested cluster** — 288 GB ask vs ~240 GB realistic free across remaining VMs. Could trigger kubelet OOM during VCFA cluster init. **Mitigation:** consider Simple deployment instead, OR power off `collector` for the duration of the deploy.
2. **Fleet certificate SAN** — if still pointed at the wrong/missing FQDN, LCMVMSP10002 will return. **Mitigation:** verify SAN before deploy submit (Phase B10).
3. **Cluster Store flakiness** — we've seen `clusterAdmin runReplica` exit:2 block MM operations multiple times in this cluster. **Mitigation:** verify Cluster Store health on all 4 hosts in Phase D4 before submit.
4. **NSX Manager state** — was "(disconnected)" in vCenter UI earlier today; never re-verified as fully healthy. **Mitigation:** Phase B5 + Phase D6.
5. **vSAN reduced-availability cascades** — per the 2026-05-11 incident, aggressive vSAN repair triggers can take down hosts. **Mitigation:** no `ClomRepairDelay=0`, no "Repair Objects Immediately" buttons during deploy.

---

## 10. Decisions Required Before Execution

| # | Decision | Options | Default if not specified |
|---|---|---|---|
| 1 | Convert pg-vm-mgmt to ephemeral, OR create new pg-mgmt-ephemeral? | A (convert existing) / B (new PG) | B (new PG, lower risk) |
| 2 | Sizing: Simple (1 node) or HA (3 node)? | Simple / HA | Simple (only fits cleanly) |
| 3 | If HA, which VMs to power off during deploy to free memory? | collector / vcf-ops / both | None — if HA, decide before |
| 4 | UI or API deploy path? | UI (Broadcom-doc) / API (lab-verified) | UI |
| 5 | Accept lab-verified-only steps (B7, B8, E-Path-2, F-fallbacks) where Broadcom is silent? | Yes / No | Yes — otherwise we have no path for documented failures |

---

## 11. Pre-Execution Summary

Before any execution begins, the following must be true:

1. Phase 1 cluster rebalance complete ( `data-move=0` , `GB-left=0` ) — gate
2. All 5 decisions above made and recorded
3. Each Phase A–D step has a Broadcom KB or explicit lab-verified acknowledgment
4. Reaction plans for Phase F symptoms are agreed to in advance
5. Rollback path defined for each Phase A network change (e.g., if A4 power-on of vcenter fails, revert .vmx to TempMgmt)

Once these are signed off, execution can begin one step at a time with verification gates between each step.

---

## 12. Document Information

| Field | Value |
|---|---|
| Document Title | VCF Automation Deploy — Airtight Plan |
| Document Version | 1.0 (review-only; no execution) |
| Author | Virtual Control LLC |
| Date Prepared | May 12, 2026 |
| Classification | Internal — Lab/Operations |
| Source Audit | Broadcom KBs and TechDocs verified against current state (2026-05-11) by independent research agent. Six gaps identified and flagged. |
| Companion Docs | `VCFA-Deployment-Fix-Runbook-2026-05-08.md` (operational runbook), `vSAN-OSA-Disk-Group-Rebuild-Handbook-2026-05-08.md` (Phase 1 complete), `VCF-Undocumented-Issues-Reference.md` v6.0 (40 verified-undocumented issues) |
| Status | Awaiting decisions on items #1–5 in Section 10 |

---

## 3. Nested-ESXi RAM Rebalance (making room)

*Source: Nested-ESXi-RAM-Rebalance-for-VCFA-Deploy-2026-05-18.md*

**Prepared by:** Virtual Control LLC **Date:** May 18, 2026 **Document Version:** 1.0 **Classification:** Internal — Lab Environment **Workstation:** Dell Precision 7920 Tower (384 GB physical RAM, 382.65 GB OS-visible after BIOS reserve) **Cluster:** vcenter-cl01 (4-node nested ESXi vSAN) **Trigger:** VCFA Simple deploy failure `LCMVSPHERECONFIG1000095` — K8s bootstrap timeout from host RAM overcommit

> **Scope.** Procedure for redistributing the 352 GB allocated across four nested ESXi VMs so that `esxi04.lab.local` can host a 96 GB VCFA Simple appliance without overcommit-induced K8s bootstrap latency. The math: VCFA Simple = 96 GB hard requirement; each host today = 88 GB physical; overcommit is what's tripping the deploy. Fix: take 24 GB from `esxi02` (least-loaded), give 40 GB to `esxi04`, net workstation reserve unchanged.
>
> **Companion docs.** - `Fleet-vCenter-Trust-Cascade-Repair-2026-05-18.md` — the trust state that had to be resolved before the deploy could even reach this failure mode - `VCFA-Simple-Deploy-Wizard-Precheck-Gotchas-2026-05-18.md` — wizard-level gotchas (cert SAN, IP pool count, dual drafts) - `VCF-Depot-Move-USB-to-NVMe-2026-05-18.md` — the depot-side bottleneck that delayed OVA download in the same deploy attempt

---

**Table of Contents**

1. Why This Is Needed
2. Current State
3. Target State

4. The Constraint Explained
5. Procedure
   5.1 Step A — Shrink esxi02 (88 → 64 GB)
   5.2 Step B — Grow esxi04 (88 → 128 GB)
6. Verification Matrix
7. Rollback
8. Risk and Failure Modes
9. Lessons Learned
10. Document Information

---

## 1. Why This Is Needed

VCFA Simple/small in VCF 9.0.1 requires a **96 GB hard-allocated VM** (per [[reference-vcfa-sizing-fixed]] memory). The 2026-05-18 deploy attempt placed `vcfa-mgmt-rq998` on `esxi04` which has **88 GB physical RAM**. ESXi accepted the overcommit (8 GB short of the VM's allocation), but during internal Kubernetes bootstrap the API server's startup phases were sensitive to memory pressure on the host:

```
 error: timed out waiting for the condition on clusters/vcf-mgmt-2d334d21da
"the request has been made before all known HTTP paths have been installed,
 please try again - error from a previous attempt:
 read tcp 127.0.0.1:41200->127.0.0.1:37153: read: connection reset by peer"
ERR:INFRA0002 - Waiting for VCF services platform cluster nodes to become ready
```

Memory metrics during the failure showed the VM was only *consuming* 3.5 GB of its 96 GB allocation, but the **host's reservation tracking treated the full 96 GB as committed**, leaving little room for ESXi's own kernel + services and the brief allocation spikes during kubelet/etcd/kube-apiserver startup.

> **The fix is not "more RAM in absolute terms"** — it's **eliminating the overcommit** on the host that runs VCFA. Going from 88 GB physical (96 GB VM = 109% commit) to 128 GB physical (96 GB VM = 75% commit) restores headroom for ESXi (~6 GB), kubelet bootstrap spikes (~10 GB), and unused VM reservation slack without triggering ballooning or swap.

---

## 2. Current State

**Physical workstation (Dell Precision 7920):** 384 GB installed (12 × 32 GB DIMMs). OS-visible 382.65 GB after BIOS reserve. Verified via `Get-CimInstance Win32_PhysicalMemory | Measure-Object Capacity -Sum`.

| Host | Total RAM | VM memory sum | Headroom | Active workload |
|---|---|---|---|---|
| esxi01 | 88 GB | **48.2 GB** | 40 GB free | nsx-manager (48) + vCLS (0.16) |
| esxi02 | 88 GB | **30 GB** | 58 GB free | vcenter (18) + fleet (12) |
| esxi03 | 88 GB | **52.2 GB** | 36 GB free | vcf-ops (20) + collector (16) + sddc-manager (16) + vCLS |
| esxi04 | 88 GB | **(none after VCFA cleanup)** | 88 GB free | (idle) — was vcfa-mgmt-rq998 (96 GB), now deleted |

**Allocation summary:** - Total physical installed: 384 GB - OS-visible: 382.65 GB - Allocated to nested ESXi: 4 × 88 = 352 GB - Reserved for Windows host + apps: ~32 GB (382.65 − 352 ≈ 30.65; rounds to 32 GB working)

**Per-host CPU/CPU-ratio for context** (relevant if you also tune vCPU later): all four hosts have 24 vCPU. Cluster total: 55,056 MHz available. esxi04 usage during the failure was ~9% — CPU was not a constraint.

---

## 3. Target State

| Host | Current | Target | Δ | Justification |
|---|---|---|---|---|
| esxi01 | 88 GB | 88 GB | 0 | nsx-manager dedicated host; keep at 88 with 40 GB headroom |
| **esxi02** | **88 GB** | **64 GB** | **-24** | Donor — vcenter+fleet only need 30 GB; 34 GB headroom remains |
| esxi03 | 88 GB | 88 GB | 0 | Most loaded with VCF mgmt VMs; keep room for growth |
| **esxi04** | **88 GB** | **112 GB** | **+24** | Recipient — VCFA's 96 GB + 16 GB ESXi/bootstrap headroom |
| Windows | 32 GB | 32 GB | 0 | Unchanged — strict conservation |
| **Total** | 384 | 384 | 0 | Conservation |

Net change: **352 GB allocated to nested ESXi unchanged**. esxi02 down 24, esxi04 up 24, perfect swap. Windows reserve stays at 32 GB — important for Workstation responsiveness during the deploy retry.

### *Why 112 GB (not 128 GB) for esxi04*

128 GB would give more headroom (32 GB above VCFA's 96 GB) but requires taking 16 more GB from somewhere. The only safe sources:

| Source | Impact | Verdict |
|---|---|---|
| Windows reserve 32 → 16 GB | Tight: Win11 + Workstation + Defender + browser + terminals = 12-20 GB working. File system cache evicted → slow disk I/O for everything | Too tight — pushes back |
| Shrink esxi01 88 → 72 GB (nsx-manager has 40 GB headroom today) | NSX is sensitive to memory pressure; corfu compactor issues per KB 390592 | Risky |
| Shrink esxi03 88 → 72 GB (loaded with vcf-ops + collector + sddc-mgr = 52 GB) | Only 20 GB headroom remaining; growth blocked | Tight |

**Verdict:** 112 GB on esxi04 gives 16 GB host headroom — enough for ESXi (~6-8 GB), kubelet/etcd bootstrap spikes (~4-6 GB), and VCFA active growth (~2-4 GB). Today's failure was at 88 GB host = 8 GB **deficit** to VCFA's 96 GB. Going to 112 = 16 GB **surplus**. That's a 24 GB net swing — substantially better than today's overcommit.

**Fallback if 112 still fails:** later, shrink esxi03 by 16 GB (88 → 72) to fund esxi04 = 128 GB. Don't pay that cost preemptively.

---

## 4. The Constraint Explained

Why we **must shrink one host before growing another**, not just grow `esxi04` in isolation:

```
 Physical RAM installed          :  384 GB (12 × 32 GB DIMMs)
OS-visible after BIOS reserve    :  382.65 GB
Currently committed:
  Windows + Defender + apps      :  ~32 GB
  4 nested ESXi VMs @ 88 GB each:  352 GB
                                  -----
                                   384 GB ← fully committed, no slack
```

If we power on a hypothetical `esxi04` at 112 GB (24 GB more than current 88) **without first shrinking another host**, VMware Workstation would try to reserve 112 GB from the OS but only `88 GB (esxi04's old reservation) + ~7 GB (free)` = ~95 GB is actually available. Workstation would either refuse to start the VM, or use Windows' page file to back the extra ~17 GB → significant performance penalty (page file on disk vs RAM = 1000x slower).

**Therefore the order is critical:** shrink first, grow second.

---

## 5. Procedure

> **Per [[feedback-homelab-extreme-caution]]:** This lab has interlocked dependencies. **Do hosts ONE AT A TIME.** Verify vSAN health between steps. Never put two hosts in Maintenance Mode simultaneously on this 4-node cluster — FTT=1 means losing two hosts at once would make data inaccessible.
>
> **Per [[feedback-vsan-mm-full-data-migration]]:** Use **Ensure Accessibility**, NOT Full Data Migration. The host is returning with the same identity and disks; FDM would unnecessarily migrate every object off the host (hours) and back.

### *5.1 Step A — Shrink esxi02 (88 → 64 GB)*

esxi02 hosts `vcenter` (18 GB) + `fleet` (12 GB). We evacuate those VMs temporarily, resize, bring them back.

#### A.1 Pick the temporary host for vcenter + fleet

vcenter (18 GB) and fleet (12 GB) total 30 GB. Targets:

| Target host | Free RAM today | Fits? |
|---|---|---|
| esxi01 | 40 GB | Yes (after vcenter+fleet land, 10 GB free) |
| esxi03 | 36 GB | Yes (after vcenter+fleet land, 6 GB free — tight) |
| esxi04 | 88 GB | Best (no VMs after VCFA cleanup) |

**Recommended:** vMotion both to **esxi04** (most headroom; will be expanded soon anyway).

#### A.2 vMotion vcenter and fleet to esxi04

In vSphere Client:

1. Right-click `vcenter` (VM) → **Migrate** → **Change compute resource only** → select `esxi04.lab.local` → Next → Next → Finish
2. Wait for migration to complete (~2 min for 18 GB nested vMotion)
3. Right-click `fleet` (VM) → **Migrate** → **Change compute resource only** → `esxi04.lab.local` → Finish

Verify both are running on esxi04 with healthy guest heartbeats before proceeding.

> **Note:** since DRS is Manual (per the cluster config), vMotion is operator-initiated. There is no automatic migration during the resize window.

#### A.3 Verify esxi02 has no powered-on workload VMs

```
Get-VMHost esxi02.lab.local | Get-VM |
 Where-Object {$_.PowerState -eq 'PoweredOn'} |
 Format-Table Name, MemoryGB -AutoSize
```

Expected: empty result (vCLS may still be present — that's harmless and will move automatically when esxi02 goes into MM).

#### A.4 Put esxi02 in Maintenance Mode (Ensure Accessibility)

In vSphere Client → right-click `esxi02.lab.local` → **Maintenance Mode** → **Enter Maintenance Mode** → vSAN data migration: **Ensure Accessibility** → OK.

Wait for the task to reach 100%. Typically <1 min for a host with only vCLS to evacuate.

#### A.5 Power off esxi02 nested VM in VMware Workstation

1. Open VMware Workstation on the 7920
2. Find the `esxi02.lab.local` VM
3. **VM** → **Power** → **Shut Down Guest** (graceful), or if unresponsive, **Power Off**

Wait for VM to show "Powered Off" state.

#### A.6 Resize esxi02 RAM from 88 GB to 64 GB

1. VM → Settings → **Memory** → change **Memory** value from `90112 MB` (88 × 1024) to `65536 MB` (64 × 1024)
2. Click OK
3. Verify in VM Settings the new value is 64 GB

#### A.7 Power on esxi02 nested VM

VM → Power → Start Up Guest (or Power On to Guest)

Boot takes 1-2 min in a healthy nested environment.

#### A.8 Wait for esxi02 to rejoin cluster

In vSphere Client, watch `esxi02.lab.local` connection state transition from `Disconnected` → `Connected`. Verify in **Hosts** view that the host shows up with the new memory total (64 GB) in **Summary** → **Memory Capacity**.

#### A.9 Exit Maintenance Mode

Right-click `esxi02.lab.local` → **Maintenance Mode** → **Exit Maintenance Mode**.

#### A.10 Verify vSAN health

SSH to any nested ESXi host (or use Posh-SSH from Windows):

```
 ssh root@192.168.1.74 # or any esxi host
esxcli vsan debug object health summary get
```

Expected output:

```
 Health Status               Number Of Objects
healthy ###
inaccessible                                0
data-move                                   0
nondata-move                                0
```

If `inaccessible` is non-zero, wait 5 more minutes and re-check. Typical vSAN resync after a brief absence (host coming back same identity with same disks) takes 3-10 min on a healthy small cluster.

#### A.11 vMotion vcenter and fleet back to esxi02

Once vSAN reports 0 inaccessible, migrate the VMs back to their normal home:

1. Right-click `vcenter` → **Migrate** → `esxi02.lab.local` → Finish
2. Right-click `fleet` → **Migrate** → `esxi02.lab.local` → Finish

Verify both are powered on, heartbeat healthy, IPs unchanged.

#### A.12 End-of-step verification

```
Get-VMHost esxi02.lab.local | Select-Object Name,
 @{n='TotalGB';e={[math]::Round($_.MemoryTotalGB,0)}},
 @{n='UsedGB';e={[math]::Round($_.MemoryUsageGB,1)}},
 ConnectionState
```

Expected: `TotalGB=64`, `ConnectionState=Connected`.

---

### *5.2 Step B — Grow esxi04 (88 → 112 GB)*

esxi04 has no workload VMs (after the VCFA failed-deploy cleanup). This step is simpler than Step A.

#### B.1 Confirm esxi04 has no powered-on VMs

```
Get-VMHost esxi04.lab.local | Get-VM |
 Where-Object {$_.PowerState -eq 'PoweredOn'}
```

Expected: empty result. If vMotion'd VMs from Step A are still on esxi04, **vMotion them back to their home host first** before proceeding.

#### B.2 Put esxi04 in Maintenance Mode (Ensure Accessibility)

Right-click `esxi04.lab.local` → **Maintenance Mode** → **Enter Maintenance Mode** → **Ensure Accessibility** → OK.

vSAN will mark objects on esxi04 as needing the "Ensure Accessibility" protection. This is fast (no data movement; data stays on the host's disks).

#### B.3 Power off esxi04 nested VM in Workstation

VM → Power → Shut Down Guest.

#### B.4 Resize esxi04 RAM from 88 GB to 112 GB

VM → Settings → Memory → change from `90112 MB` to **`114688 MB`** (112 × 1024) → OK.

> **Sanity check:** VMware Workstation must show no "memory not available" warning at this point. If it does, you don't have enough free physical RAM (Step A should have freed 24 GB; if it didn't, re-verify Step A.12 before proceeding).

#### B.5 Power on esxi04 nested VM

VM → Power → Start Up Guest.

#### B.6 Wait for esxi04 to rejoin cluster

In vSphere Client, watch `esxi04.lab.local` state transition. Verify Summary → Memory Capacity shows **112 GB total**.

#### B.7 Exit Maintenance Mode

Right-click → **Maintenance Mode** → **Exit Maintenance Mode**.

#### B.8 Verify vSAN health

```
 ssh root@192.168.1.74
esxcli vsan debug object health summary get
```

Wait for `inaccessible: 0` and `data-move: 0`.

#### B.9 End-of-step verification

```
Get-VMHost esxi04.lab.local | Select-Object Name,
 @{n='TotalGB';e={[math]::Round($_.MemoryTotalGB,0)}},
 ConnectionState
```

Expected: `TotalGB=112`, `ConnectionState=Connected`.

---

## 6. Verification Matrix

After both steps complete, before resubmitting VCFA deploy:

| Check | Expected | Command |
|---|---|---|
| Total RAM committed to nested ESXi | **352 GB** (unchanged — perfect swap) | `(Get-VMHost \| Measure-Object MemoryTotalGB -Sum).Sum` |
| esxi02 memory total | 64 GB | `(Get-VMHost esxi02.lab.local).MemoryTotalGB` |

| Check | Expected | Command |
|---|---|---|
| esxi04 memory total | **112 GB** | `(Get-VMHost esxi04.lab.local).MemoryTotalGB` |
| All hosts connected | 4/4 | `(Get-VMHost \| Where-Object {$_.ConnectionState -eq 'Connected'}).Count` |
| vCenter on esxi02 | Yes | `(Get-VM vcenter).VMHost.Name` |
| Fleet on esxi02 | Yes | `(Get-VM fleet).VMHost.Name` |
| vSAN inaccessible objects | 0 | `esxcli vsan debug object health summary get` |
| Windows free RAM | ~7 GB free of ~32 GB working set (unchanged) | `(Get-CimInstance Win32_OperatingSystem).FreePhysicalMemory / 1MB` |
| No swap on hosts | 0 KB | `Get-Stat -Entity (Get-VMHost) -Stat mem.swapused.average` |

---

## 7. Rollback

If anything goes wrong mid-procedure, revert to the original 88/88/88/88 configuration:

### *Rollback Step A (if esxi02 won't come back at 64 GB)*

1. In Workstation, power off the resized esxi02 VM
2. VM → Settings → Memory → change back to `90112 MB`
3. Power on
4. Verify rejoin in vSphere Client
5. Exit MM
6. vMotion vcenter+fleet back

### *Rollback Step B (if esxi04 won't come back at 112 GB or causes Workstation memory exhaustion)*

1. Power off resized esxi04 VM
2. Change Memory back to `90112 MB`
3. Power on
4. Exit MM

After rollback, the original 4×88 GB layout is restored and VCFA cannot be reliably deployed at Simple/96 GB on this host. Consider: - Skipping VCFA deployment (lab can run on just VCF Operations) - Adding physical RAM to the workstation (requires hardware purchase) - Re-investigating whether VCFA can be made to work at 88 GB with **VM-level memory reservation** of less than 96 GB (would require a custom OVA edit — not supported by VMware)

---

## 8. Risk and Failure Modes

| Risk | Probability | Mitigation |
|---|---|---|
| vSAN resync takes longer than 10 min | Medium | Wait. Don't proceed to next host until 0 inaccessible. |
| vCenter (running on esxi02) becomes inaccessible during Step A | Low (we vMotion it off first) | A.2 vMotions vcenter to esxi04 before MM. Verify vMotion success before A.4. |

| Risk | Probability | Mitigation |
|---|---|---|
| Nested ESXi VM won't power back on after resize | Low | Rollback per §7; check Workstation event log for memory pre-allocation errors. |
| Workstation runs out of RAM when starting esxi04 at 112 GB | Low if Step A completed | Step A.10 verifies. If `Free RAM < 28 GB` after Step A (we need 24 GB more than current esxi04 + minimal slack), do not proceed to Step B. |
| vSAN cluster temporarily at FTT=0 during MM | Expected | Single-host MM in 4-node FTT=1 cluster stays at FTT=1 (with reduced redundancy). Do NOT MM two hosts simultaneously. |
| `nsx-manager` co-tenancy violation if you vMotion something to esxi01 temporarily | Low (we use esxi04 as temp home) | A.1 deliberately picks esxi04 as the temp host, not esxi01, per [[feedback-nsx-manager-dedicated-host]]. |
| HA tries to restart a VM during the operation | None | Cluster HA is currently disabled (per the cluster config verified 2026-05-18). |

---

## 9. Lessons Learned

**1. 88 GB physical RAM cannot reliably host VCFA Simple's 96 GB VM.** Even though ESXi accepts the overcommit at power-on, the K8s bootstrap inside the VM is sensitive to host memory pressure. The deploy timeout (~30 min) is too aggressive for the latency overcommit introduces. **Workaround: bump the VCFA host to 112 GB minimum so the 96 GB VM runs with 16 GB host headroom (enough for ESXi ~6-8 GB + kubelet/etcd bootstrap spikes ~4-6 GB).** 128 GB is "ideal" but requires shrinking Windows reserve to 16 GB which causes its own problems.

**2. Shrink-before-grow is mandatory.** The workstation's physical RAM is the absolute ceiling. You cannot grow one nested ESXi VM without first shrinking another (or accepting that the workstation OS will swap, which is catastrophic for nested perf).

**3. "Ensure Accessibility" is correct for memory resize.** The host returns with the same identity and the same disks. Full Data Migration would unnecessarily move every object off the host (hours) and back. EA is the documented choice per Broadcom KB pattern.

**4. vMotion vcenter+fleet to esxi04 as the temp home, not esxi01.** Per [[feedback-nsx-manager-dedicated-host]], NSX Manager (on esxi01) should not share with other VCF VMs even temporarily. esxi04 is the natural choice — empty after VCFA cleanup, lots of headroom.

**5. Do hosts ONE AT A TIME.** Single-host MM on a 4-node FTT=1 vSAN keeps redundancy. Two hosts in MM simultaneously = data inaccessibility. The procedure is explicitly serialized for this reason.

**6. Memory active vs allocated is misleading during failure diagnosis.** During the 2026-05-18 failure, the VCFA VM reported only 3.5 GB *active* memory but had 96 GB *allocated*. Naive reading of metrics suggests "no memory pressure." But ESXi's host-side reservation tracking treated the full 96 GB allocation as committed, and that's what tripped the bootstrap timeout. **Always use `mem.consumed.average` and host-level `mem.swapused.average` plus the host's `MemoryUsageGB / MemoryTotalGB` ratio — not the VM's `mem.active`.**

---

## 10. Document Information

| Field | Value |
|---|---|
| Title | Nested ESXi RAM Rebalance — Make Room for VCFA Simple on a 7920 Lab |
| Document ID | VC-RUNBOOK-NESTED-RAM-REBALANCE-20260518 |
| Version | 1.0 |
| Issued | May 18, 2026 |
| Author | Virtual Control LLC |
| Classification | Internal — Lab Environment |
| Related Documents | `Fleet-vCenter-Trust-Cascade-Repair-2026-05-18.md`, `VCFA-Simple-Deploy-Wizard-Precheck-Gotchas-2026-05-18.md`, `VCF-Depot-Move-USB-to-NVMe-2026-05-18.md` |
| Distribution | Lab notebook; values (host names, IPs) need redaction before sharing externally |

---

## 4. Pre-Deploy Cleanup + Fleet Cert SSH-Replace

*Source: VCFA-PreDeploy-Cleanup-CertReplace-2026-05-16.md*

**Author:** Virtual Control LLC **Date:** May 16, 2026 **Document Version:** 1.0 **Classification:** Internal — Operations Record **Scope:** Phase 1 (LCM DB six-table cleanup + `vmware-vcops-web` restart) and Phase 2 (Fleet cert regen with Strict Simple SAN scope, SSH direct replace after UI Import path failed) executed on 2026-05-16 against `lcm.lab.local` and `vcf-ops.lab.local`. Anchors against the VCFA Deploy Airtight Plan dated 2026-05-12.

---

**Table of Contents**


1. Executive Summary
2. Context — Where We Were Before This Work
3. Verified Facts at Execution Time
4. Phase 1.1 — Confirm No Orphan VCFA VMs
5. Phase 1.2 — LCM DB Six-Table Cleanup
6. Phase 1.3 — Restart vmware-vcops-web
7. Phase 2.1 — Cert Generation
8. Phase 2.2 — Path A (UI Import Certificate) Failure
9. Phase 2.3 — Path B (SSH Direct File Replace) — Successful
10. Verification — Workstation-Side TLS Probe
11. Outcomes — Pre-Deploy Gate Status
12. Tooling Notes — Why Posh-SSH Beat plink
13. KB References & Companion Docs

Appendix A — Full Commands That Worked


---

### 1. Executive Summary

**Outcome:** Both Phase 1 (cleanup) and Phase 2 (cert) of the VCFA pre-deploy airtight plan completed successfully on 2026-05-16. HB-3 (cleanup + cert) is now closed in addition to HB-1, HB-2, HB-4 satisfied earlier. Environment is ready for VCFA deploy submit.

| Phase | Action | Outcome |
|---|---|---|
| 1.1 | Confirm zero `vcfa-*` / `automation-*` VMs in vCenter | **0 found** — workload-VM cleanup already done in earlier sessions |
| 1.2 | LCM DB six-table cleanup (deepest-first) | **66 rows deleted** across `vm_lcops_*` tables; **124 stuck request rows** marked FAILED; vrlcm-server restarted |
| 1.3 | Restart `vmware-vcops-web` on VCF Ops appliance | Service active in 5s; VCF Ops UI HTTP 200 on first poll |
| 2.1 | Generate Fleet cert with Strict Simple SAN (6 DNS + 3 IPs) | Cert + key written to `/root/vcfa/` on Fleet; copied to workstation |
| 2.2 | **Path A** — VCF Operations UI Import Certificate | **FAILED.** UI form has no Private Key field; expects CSR-based workflow that did not match our externally-generated keypair |
| 2.3 | **Path B** — SSH direct file replace on Fleet | **SUCCEEDED.** Replaced nginx ( `/opt/vmware/vlcm/cert/server.{crt,key}` ) and lighttpd ( `/opt/vmware/etc/lighttpd/server.pem` ). Reload + restart clean |
| Verify | Workstation TLS probe against `lcm.lab.local:443` | New thumbprint `49:B3:F9:95:AA:26:8F:F1:E2:EE:CC:B4:3D:2B:C3:10:0D:D2:D5:E0` confirmed; SAN list complete |

---

## 2. Context — Where We Were Before This Work

Coming out of the VCF Management Plane vDS Migration (2026-05-16, earlier same day), the environment had:

- HB-1 satisfied (vCenter on `pg-mgmt-ephemeral` DVPG on live `vcenter-cl01-vds01` )
- HB-2 satisfied (Ephemeral binding)
- HB-4 satisfied (DRS FullyAutomated)
- NSX cluster STABLE, TNP corrected, vCenter "vDS proxy switches" alarm cleared
- vSAN rebalance done (0 GB left)
- All 4 management VMs on live vDS

What remained per the VCFA Deploy Airtight Plan 2026-05-12:

- **HB-3 cleanup** — 7+ failed VCFA deploy attempts from May 5–8 left rows in Fleet's LCM database and a stale parallel-deploy lock in VCF Operations' web tier cache
- **HB-3 cert** — Fleet's TLS cert had SANs `lcm.lab.local` + `192.168.1.95` only (thumbprint `8B:D1:E6:52...` ). Per `[BROADCOM-DOC: KB 411354]` this is the documented #1 cause of `LCMVMSP10002` VCFA deploy failures

This document records the execution of those two remaining gates.

---

## 3. Verified Facts at Execution Time

All values below are observed on 2026-05-16 via PowerShell + Posh-SSH; nothing is inferred.

### *3.1 Target appliances*

| Appliance | Hostname | IP | SSH user used | Role |
|---|---|---|---|---|
| Fleet (vRSLCM) | `lcm.lab.local` | 192.168.1.95 | `root` (admin failed with keyboard-interactive denial) | LCM DB host, Fleet cert host |
| VCF Operations | `vcf-ops.lab.local` | 192.168.1.77 | `root` | Hosts `vmware-vcops-web` |
| vCenter | `vcenter.lab.local` | 192.168.1.69 | (PowerCLI) | Source of truth for `vcfa-*` VM presence check |

### *3.2 Cert state — before vs after*

| Attribute | Before | After |
|---|---|---|
| Thumbprint (SHA-1) | `8B:D1:E6:52:77:4E:EC:E0:84:9F:81:0A:5D:89:1E:50:EF:39:A1:AA` | `49:B3:F9:95:AA:26:8F:F1:E2:EE:CC:B4:3D:2B:C3:10:0D:D2:D5:E0` |
| Subject CN | `lcm.lab.local` | `automation-node1.lab.local` |
| SAN DNS list | `lcm.lab.local` only | `automation-node1.lab.local`, `automation-node1`, `vcfa-mgmt-0.lab.local`, `vcfa-mgmt-0`, `lcm.lab.local`, `fleet.lab.local` |
| SAN IP list | `192.168.1.95` only | `192.168.1.91`, `192.168.1.92`, `192.168.1.95` |
| Validity | 2026-05-04 → 2031-05-03 | 2026-05-16 → 2028-05-15 (730 days) |
| Issuer | self-signed (Broadcom default) | self-signed (matches lab pattern) |

### *3.3 LCM DB state — before vs after*

| Item | Before | After |
|---|---|---|
| `vm_lcops_environments` rows | 2 (1 FAILED + 1 COMPLETED) | 1 (COMPLETED only) |
| `vm_lcops_product_nodes` rows under FAILED env | 2 | 0 |
| `vm_lcops_node_properties` rows under FAILED env | 34 | 0 |
| `vm_lcops_product_properties` rows under FAILED env | 24 | 0 |
| `vm_lcops_product` rows under FAILED env | 2 | 0 |
| `vm_lcops_infrastructure_properties` rows under FAILED env | 36 | 0 |
| `vm_rs_request` stuck rows | 0 (already in COMPLETED/FAILED) | 0 |
| `vm_engine_execution_request` stuck rows ( `INITIATED` + `CREATED` ) | 124 | 0 (all transitioned to FAILED) |

---

## 4. Phase 1.1 — Confirm No Orphan VCFA VMs

### *4.1 Action*

```
Connect-VIServer vcenter.lab.local -User administrator@vsphere.local -Password '<lab-pw>' -Force
```

```
Get-VM | Where-Object { $_.Name -match '^(vcfa|automation)' }
```

### *4.2 Result*

**Zero rows returned.** The vCenter inventory had no `vcfa-*` or `automation-*` VMs at execution time. The May 5–8 failed deploys had either auto-cleaned their VMs during prior recovery operations or never reached the VM-creation stage. The "Phase 1.2 — Fleet UI Delete" step from the airtight plan was therefore not required for the vCenter side; only the LCM DB cleanup (§5) and web-tier cache reset (§6) were.

---

## 5. Phase 1.2 — LCM DB Six-Table Cleanup

### *5.1 Why this is needed*

Per the VCFA Deploy Airtight Plan §1.3, LCM DB cleanup is required when Fleet UI delete leaves orphans OR when the prior delete attempts timed out / errored. The May 5–8 deploy attempts left rows in the `vm_lcops_*` hierarchy that match the failed `envId`s and would conflict with new deploys at the same target.

Direct delete of an environment row violates foreign-key constraints because of the parent → child relationships:

```
vm_lcops_environments  (parent — must be deleted LAST)
 └─ vm_lcops_product (FK environment_vmid)
      ├─ vm_lcops_product_nodes (FK product_vmid)
      │    └─ vm_lcops_node_properties (FK node_vmid)
      └─ vm_lcops_product_properties (FK product_vmid)
 └─ vm_lcops_infrastructure_properties (FK environment_vmid)
```

The six-table cleanup deletes deepest first.

### *5.2 Pre-cleanup query*

```
SELECT vmid, environmentname, status FROM vm_lcops_environments ORDER BY vmid;
```

Result:

```
 601ea82d-1f72-4d8e-848a-125265ad1aa7 | 601ea82d-1f72-4d8e-848a-125265ad1aa7 | FAILED
888501ce-eb95-4ce8-8043-ab216906bc67 | 888501ce-eb95-4ce8-8043-ab216906bc67 | COMPLETED
```

Confirmed: 1 FAILED env (the one tracked in the airtight plan), 1 COMPLETED (management env — must NOT be deleted).

### *5.3 Stop vrlcm-server*

```
systemctl stop vrlcm-server
```

Result: state `inactive` (it was already stopped at execution time; exit code 3 = "not running" — that's safe to proceed).

### *5.4 Six-table DELETE sequence (deepest first)*

All DELETEs are scoped `WHERE ... environment_vmid IN (SELECT vmid FROM vm_lcops_environments WHERE status != 'COMPLETED')` so the COMPLETED management env is **never touched**.

```
PSQL="sudo -u postgres /opt/vmware/vpostgres/14/bin/psql -d vrlcm"

## 1/6 — Node properties (deepest child)
$PSQL -c "DELETE FROM vm_lcops_node_properties WHERE node_vmid IN (
  SELECT vmid FROM vm_lcops_product_nodes WHERE product_vmid IN (
    SELECT vmid FROM vm_lcops_product WHERE environment_vmid IN (
      SELECT vmid FROM vm_lcops_environments WHERE status != 'COMPLETED')));"
## Result: DELETE 34

## 2/6 — Product nodes
$PSQL -c "DELETE FROM vm_lcops_product_nodes WHERE product_vmid IN (
  SELECT vmid FROM vm_lcops_product WHERE environment_vmid IN (
    SELECT vmid FROM vm_lcops_environments WHERE status != 'COMPLETED'));"
## Result: DELETE 2

## 3/6 — Product properties
$PSQL -c "DELETE FROM vm_lcops_product_properties WHERE product_vmid IN (
  SELECT vmid FROM vm_lcops_product WHERE environment_vmid IN (
    SELECT vmid FROM vm_lcops_environments WHERE status != 'COMPLETED'));"
## Result: DELETE 24

## 4/6 — Products
$PSQL -c "DELETE FROM vm_lcops_product WHERE environment_vmid IN (
  SELECT vmid FROM vm_lcops_environments WHERE status != 'COMPLETED');"
## Result: DELETE 2

## 5/6 — Infrastructure properties
$PSQL -c "DELETE FROM vm_lcops_infrastructure_properties WHERE environment_vmid IN (
  SELECT vmid FROM vm_lcops_environments WHERE status != 'COMPLETED');"
## Result: DELETE 36

## 6/6 — Environments (parent — LAST)
$PSQL -c "DELETE FROM vm_lcops_environments WHERE status != 'COMPLETED';"
## Result: DELETE 1
```

### *5.5 Stuck request table cleanup*

```
## Transition stuck in-flight request states to FAILED
$PSQL -c "UPDATE vm_rs_request SET state='FAILED'
          WHERE state NOT IN ('COMPLETED','FAILED');"
## Result: UPDATE 0 (vm_rs_request already clean)

$PSQL -c "UPDATE vm_engine_execution_request SET enginestatus='FAILED'
          WHERE enginestatus NOT IN ('SUCCESS','FAILED','COMPLETED');"
## Result: UPDATE 124 (cleared 30 INITIATED + 94 CREATED zombies)
```

### *5.6 Start vrlcm-server*

```
systemctl start vrlcm-server
sleep 5
systemctl is-active vrlcm-server
## Result: active
```

### *5.7 Post-cleanup verification*

```
SELECT vmid, environmentname, status FROM vm_lcops_environments ORDER BY vmid;
```

Result:

```
888501ce-eb95-4ce8-8043-ab216906bc67 | 888501ce-eb95-4ce8-8043-ab216906bc67 | COMPLETED
```

Exactly the COMPLETED management env, no FAILED rows.

---

## 6. Phase 1.3 — Restart vmware-vcops-web

### *6.1 Why this is needed*

The VCF Operations web tier holds an in-memory cache that includes a parallel-deploy lock. Per `[LAB-VERIFIED]` (Issue #35 in `VCF-Undocumented-Issues-Reference.md`), this lock survives LCM DB cleanup and persists across the appliance until the web service is restarted. Restart drops the cache.

### *6.2 Prerequisite — SSH must be enabled on VCF Ops*

SSH is disabled by default on the VCF Operations appliance — `[LAB-VERIFIED]` Issue #19. Enable via:

**VCF Operations Admin UI → SSH → Enable** (or whichever UI path your build exposes; varies by minor version).

### *6.3 Action (after SSH enabled)*

```
 ## From workstation via Posh-SSH as root
systemctl restart vmware-vcops-web
## Poll until active (returns within 5s typically)
for i in 1..12; do
  STATE=$(systemctl is-active vmware-vcops-web)
  if [ "$STATE" = "active" ]; then break; fi
  sleep 5
done
```

### *6.4 Result*

```
[1 x 5s] state=active
```

Service active on first 5-second poll. UI subsequently confirmed responding HTTP 200 on first workstation probe.

---

## 7. Phase 2.1 — Cert Generation

### *7.1 OpenSSL config (Strict Simple SAN scope)*

Per the airtight plan's locked decision (2026-05-14): Simple sizing (1 VCFA node), so the SAN list contains only `automation-node1` + `vcfa-mgmt-0` + Fleet's own names — no HA expansion names.

Written to `/root/vcfa/vcfa-openssl.cnf` on Fleet:

```
 [req]
default_bits = 2048
prompt = no
default_md = sha256
req_extensions = v3_req
distinguished_name = dn

[dn]
C = US
```

```
ST = State
L = City
O = Lab
OU = VCF
CN = automation-node1.lab.local

[v3_req]
basicConstraints = CA:FALSE
keyUsage = digitalSignature, keyEncipherment
extendedKeyUsage = serverAuth, clientAuth
subjectAltName = @alt_names

[alt_names]
DNS.1 = automation-node1.lab.local
DNS.2 = automation-node1
DNS.3 = vcfa-mgmt-0.lab.local
DNS.4 = vcfa-mgmt-0
DNS.5 = lcm.lab.local
DNS.6 = fleet.lab.local
IP.1 = 192.168.1.91
IP.2 = 192.168.1.92
IP.3 = 192.168.1.95
```

### *7.2 Generate key + self-signed cert (730-day validity)*

```
 cd /root/vcfa
openssl genrsa -out vcfa.key 2048
openssl req -new -x509 -key vcfa.key -out vcfa.crt -days 730 \
  -config vcfa-openssl.cnf -extensions v3_req
```

### *7.3 Verify SAN list*

```
 openssl x509 -in vcfa.crt -text -noout | grep -A 12 "Subject Alternative Name"
openssl x509 -in vcfa.crt -noout -fingerprint -sha1
openssl x509 -in vcfa.crt -noout -dates
```

Output:

```
 X509v3 Subject Alternative Name:
  DNS:automation-node1.lab.local, DNS:automation-node1,
  DNS:vcfa-mgmt-0.lab.local, DNS:vcfa-mgmt-0,
  DNS:lcm.lab.local, DNS:fleet.lab.local,
  IP Address:192.168.1.91, IP Address:192.168.1.92, IP Address:192.168.1.95
sha1 Fingerprint=49:B3:F9:95:AA:26:8F:F1:E2:EE:CC:B4:3D:2B:C3:10:0D:D2:D5:E0
notBefore=May 16 16:22:01 2026 GMT
notAfter=May 15 16:22:01 2028 GMT
```

### *7.4 Copy cert + key to workstation*

```
 ## Via Posh-SSH from workstation
$crt = (Invoke-SSHCommand -SessionId $sess.SessionId -Command 'cat /root/vcfa/vcfa.crt').Output -join "`n"
$key = (Invoke-SSHCommand -SessionId $sess.SessionId -Command 'cat /root/vcfa/vcfa.key').Output -join "`n"
$crt | Out-File 'I:\VCF Health check\vcfa-cert-2026-05-16\vcfa.crt' -Encoding ASCII -NoNewline
$key | Out-File 'I:\VCF Health check\vcfa-cert-2026-05-16\vcfa.key' -Encoding ASCII -NoNewline
```

---

## 8. Phase 2.2 — Path A (UI Import Certificate) Failure

**Path A failed.** This section documents the failure for the catalog so the next operator does not waste time on it.

### *8.1 What was attempted*

VCF Operations Admin UI → Certificates → VCF Management → select the Fleet appliance row → Replace Certificate → fill the Import Certificate dialog with:

- **Name**: any descriptive name
- **Source**: Paste Text
- **Server Certificate** (PEM): contents of `vcfa.crt`
- **Certificate Authority** (PEM): contents of `vcfa.crt` (self-signed, so the cert is its own CA)
- VALIDATE → SAVE

### *8.2 Symptom*

A red banner appeared: **"Certificate replacement for appliance lcm.lab.local has failed. Failed to perform specified operation. Certificate replace operation for VCF Operations Fleet Management failed."**

### *8.3 Root cause*

The VCF Operations 9.0.1 **Import Certificate** dialog has **no Private Key field**. It only accepts:

- Name
- Source (Paste Text)
- Server Certificate (PEM)
- Certificate Authority (PEM)

This UI workflow assumes Fleet's private key already exists internally — generated by a Fleet-side CSR flow that the user is expected to invoke separately. Pasting a certificate without a matching internally-generated CSR results in cert-vs-key mismatch and the generic failure banner. Broadcom does not document this prerequisite in the Import Certificate dialog itself; this is a workflow gap.

**Added to `VCF-Undocumented-Issues-Reference.md` as Issue #46.**

---

## 9. Phase 2.3 — Path B (SSH Direct File Replace) — Successful

### *9.1 File layout on Fleet (verified before replace)*

```
Main HTTPS (port 443) – nginx
  /opt/vmware/vlcm/cert/server.crt      ← public cert
  /opt/vmware/vlcm/cert/server.key      ← private key
  (configured in /etc/nginx/ssl.conf)

Appliance management – lighttpd
  /opt/vmware/etc/lighttpd/server.pem   ← combined: private key + public cert
```

Both are referenced by configs ( `/etc/nginx/ssl.conf` , `/opt/vmware/etc/lighttpd/cap-applmgmt-lighttpd.conf` ).

### *9.2 Sanity check — cert and key paired correctly*

Before replacement, verify the new cert and key are a matching pair (cert's public key derives from the same key as `vcfa.key`):

```
 CRT_MD5=$(openssl x509 -in /root/vcfa/vcfa.crt -pubkey -noout | openssl md5 | awk '{print $NF}')
KEY_MD5=$(openssl pkey -in /root/vcfa/vcfa.key -pubout | openssl md5 | awk '{print $NF}')
[ "$CRT_MD5" = "$KEY_MD5" ] && echo "MATCH" || echo "MISMATCH — abort!"
```

Result: MATCH.

### *9.3 Backup + replace + reload*

```
 set -e
TS=$(date +%Y%m%d-%H%M%S)

## Backup old files in place
cp /opt/vmware/vlcm/cert/server.crt /opt/vmware/vlcm/cert/server.crt.bak-$TS
cp /opt/vmware/vlcm/cert/server.key /opt/vmware/vlcm/cert/server.key.bak-$TS
cp /opt/vmware/etc/lighttpd/server.pem /opt/vmware/etc/lighttpd/server.pem.bak-$TS

## Install new files atomically
install -m 600 -o root -g root /root/vcfa/vcfa.crt /opt/vmware/vlcm/cert/server.crt
install -m 600 -o root -g root /root/vcfa/vcfa.key /opt/vmware/vlcm/cert/server.key

## Build the lighttpd combined PEM (key first, cert second — matches original ordering)
cat /root/vcfa/vcfa.key /root/vcfa/vcfa.crt > /opt/vmware/etc/lighttpd/server.pem.tmp
chmod 600 /opt/vmware/etc/lighttpd/server.pem.tmp
mv /opt/vmware/etc/lighttpd/server.pem.tmp /opt/vmware/etc/lighttpd/server.pem

## nginx config syntax check
nginx -t
## Output: nginx: configuration file /etc/nginx/nginx.conf test is successful

## Reload nginx (graceful — no connection drops)
systemctl reload nginx

## Restart lighttpd (combined PEM read at startup)
systemctl restart lighttpd
```

There is also a prior session-wide backup at `/root/fleet-cert-backup-<TS>/cert/` taken before this operation.

---

## 10. Verification — Workstation-Side TLS Probe

### *10.1 Local probe on Fleet*

```
 echo Q | openssl s_client -connect 127.0.0.1:443 -servername lcm.lab.local 2>/dev/null | \
  openssl x509 -text -noout | grep -A 3 "Subject Alternative Name"
echo Q | openssl s_client -connect 127.0.0.1:443 -servername lcm.lab.local 2>/dev/null | \
  openssl x509 -noout -fingerprint -sha1
```

Output confirms new SAN list and new thumbprint `49:B3:F9:95:AA:26:8F:F1:E2:EE:CC:B4:3D:2B:C3:10:0D:D2:D5:E0`.

### *10.2 Workstation-side independent probe*

```
 $tcp = New-Object System.Net.Sockets.TcpClient
$tcp.Connect('lcm.lab.local', 443)
$ssl = New-Object System.Net.Security.SslStream($tcp.GetStream(), $false,
       ({$true} -as [System.Net.Security.RemoteCertificateValidationCallback]))
$ssl.AuthenticateAsClient('automation-node1.lab.local')
```

```
$cert = [System.Security.Cryptography.X509Certificates.X509Certificate2]$ssl.RemoteCertificate
"Subject     : $($cert.Subject)"
"Thumbprint  : $($cert.Thumbprint)"
"Valid       : $($cert.NotBefore) → $($cert.NotAfter)"
($cert.Extensions | Where-Object { $_.Oid.FriendlyName -eq 'Subject Alternative Name' }).Format($true)
$ssl.Close(); $tcp.Close()
```

Output:

```
 Subject     : CN=automation-node1.lab.local, OU=VCF, O=Lab, L=City, S=State, C=US
Thumbprint  : 49B3F995AA268FF1E2EECCB43D2BC3100DD2D5E0
Valid       : 5/16/2026 12:22:01 → 5/15/2028 12:22:01
DNS Name=automation-node1.lab.local
DNS Name=automation-node1
DNS Name=vcfa-mgmt-0.lab.local
DNS Name=vcfa-mgmt-0
DNS Name=lcm.lab.local
DNS Name=fleet.lab.local
IP Address=192.168.1.91
IP Address=192.168.1.92
IP Address=192.168.1.95
```

Independent verification confirmed the cert install. The probe used `AuthenticateAsClient('automation-node1.lab.local')` so the SAN's first DNS name is matched explicitly — proving that VCFA-side TLS verification (which will use the same hostname) will succeed.

---

## 11. Outcomes — Pre-Deploy Gate Status

| Gate | Source | Status at end of execution |
|---|---|---|
| HB-1 — vCenter on DVS portgroup `[BROADCOM-DOC: KB 420920, KB 430625]` | Airtight Plan §HB-1 | ✅ Done (vDS migration earlier today) |
| HB-2 — Management port group ephemeral binding `[BROADCOM-DOC: KB 324492, KB 426300, KB 428279]` | Airtight Plan §HB-2 | ✅ Done (vDS migration earlier today) |
| HB-3 cleanup of failed VCFA deploys `[LAB-VERIFIED]` | Airtight Plan §1.2–§1.4 | ✅ **Done (this work — §5, §6)** |
| HB-3 Fleet cert with VCFA SANs `[BROADCOM-DOC: KB 411354]` | Airtight Plan §2.1–§2.4 | ✅ **Done (this work — §7, §9)** |
| HB-4 — DRS FullyAutomated | Airtight Plan §HB-4 | ✅ Done (vDS migration earlier today) |

Phase 0 health gate from the airtight plan (vSAN, NSX, vCenter, all components green) is also satisfied as of the morning of 2026-05-16 — captured in the VCF Mgmt Plane vDS Migration plan and the F-grade triage methodology docs.

**Environment is ready for the VCFA deploy submit.**

---

## 12. Tooling Notes — Why Posh-SSH Beat plink

PuTTY `plink` was tried first and hung consistently — the issue is that the lab admin password `<LAB_PW>` contains `!` characters which interact poorly with PowerShell's pipe / quote rules and `cmd.exe`'s delayed expansion. Multiple `cmd /c "echo y | plink ..."` variations all hung.

The fix was to install the **Posh-SSH** PowerShell module ( `Install-Module Posh-SSH -Scope CurrentUser` ) which handles passwords via `PSCredential` (a `SecureString` -wrapped object) rather than command-line argument parsing. Both `New-SSHSession` and `Invoke-SSHCommand` execute cleanly with bang-containing passwords.

This pattern should be the default for SSH automation from this workstation going forward. Captured as a tooling preference in memory.

---

## 13. KB References & Companion Docs

### *13.1 Broadcom KBs cited*

| KB | Title | Applies to |
|---|---|---|
| `[BROADCOM-DOC: KB 411354]` | VCF Automation 9 install through Fleet fails LCMVMSP10002 — cert SAN requirements + import procedure | §7, §9 — cert regen scope |
| `[BROADCOM-DOC: KB 410338]` | Deployment of VCFA from Fleet Mgmt fails with deadline exceeded — retry/clean path | §4, §5 — cleanup rationale |
| `[BROADCOM-DOC: KB 388305]` | Parallel Deployments lock (Networks/Logs equivalent) | §6 — cache restart context |
| `[BROADCOM-DOC: KB 420920, KB 430625]` | HB-1 cert/portgroup requirements | §11 |
| `[BROADCOM-DOC: KB 324492, KB 426300, KB 428279]` | HB-2 ephemeral binding | §11 |

### *13.2 Companion documents (this library)*

- `VCFA-Deploy-Airtight-Plan-2026-05-12.md` — the plan of record this execution closes against
- `VCF-Mgmt-Plane-vDS-Migration-Plan-2026-05-16.md` — earlier same-day plan that satisfied HB-1 + HB-2
- `VCFA-Deployment-Fix-Runbook-2026-05-08.md` — earlier runbook the airtight plan supersedes
- `VCF-Undocumented-Issues-Reference.md` — Issues #19 (SSH enable), #35 (parallel-deploy cache), #46 (UI Import Certificate dialog gap)
- `../../VCF-Deployment-Chronicle-2026.md` — narrative wrapper

---

## Appendix A — Full Commands That Worked

### *A.1 Phase 1 — Pre-check + LCM cleanup (one session)*

```
 Import-Module Posh-SSH
$pw   = ConvertTo-SecureString '<LAB_PW>' -AsPlainText -Force
$cred = New-Object System.Management.Automation.PSCredential('root', $pw)
$sess = New-SSHSession -ComputerName 'lcm.lab.local' -Credential $cred -AcceptKey -Force

## Stop vrlcm-server
Invoke-SSHCommand -SessionId $sess.SessionId -Command 'systemctl stop vrlcm-server' -TimeOut 30 | Out-Null

## Six-table cleanup (deepest first)
$PSQL = 'cd /tmp && sudo -n -u postgres /opt/vmware/vpostgres/14/bin/psql -d vrlcm'
foreach ($sql in @(
  "DELETE FROM vm_lcops_node_properties WHERE node_vmid IN (SELECT vmid FROM vm_lcops_product_nodes WHERE
product_vmid IN (SELECT vmid FROM vm_lcops_product WHERE environment_vmid IN (SELECT vmid FROM
vm_lcops_environments WHERE status != 'COMPLETED')));",
  "DELETE FROM vm_lcops_product_nodes WHERE product_vmid IN (SELECT vmid FROM vm_lcops_product WHERE
```

```
environment_vmid IN (SELECT vmid FROM vm_lcops_environments WHERE status != 'COMPLETED'));",
  "DELETE FROM vm_lcops_product_properties WHERE product_vmid IN (SELECT vmid FROM vm_lcops_product WHERE
environment_vmid IN (SELECT vmid FROM vm_lcops_environments WHERE status != 'COMPLETED'));",
  "DELETE FROM vm_lcops_product WHERE environment_vmid IN (SELECT vmid FROM vm_lcops_environments WHERE
status != 'COMPLETED');",
  "DELETE FROM vm_lcops_infrastructure_properties WHERE environment_vmid IN (SELECT vmid FROM
vm_lcops_environments WHERE status != 'COMPLETED');",
  "DELETE FROM vm_lcops_environments WHERE status != 'COMPLETED';",
  "UPDATE vm_rs_request SET state='FAILED' WHERE state NOT IN ('COMPLETED','FAILED');",
  "UPDATE vm_engine_execution_request SET enginestatus='FAILED' WHERE enginestatus NOT IN
('SUCCESS','FAILED','COMPLETED');"
)) {
  Invoke-SSHCommand -SessionId $sess.SessionId -Command "$PSQL -c `"$sql`" 2>&1" -TimeOut 30 | Select-
Object -Expand Output
}

## Start vrlcm-server
Invoke-SSHCommand -SessionId $sess.SessionId -Command 'systemctl start vrlcm-server && sleep 5 &&
systemctl is-active vrlcm-server' -TimeOut 60 | Out-Null

Remove-SSHSession -SessionId $sess.SessionId | Out-Null
```

### *A.2 Phase 1.3 — vmware-vcops-web restart*

```
 $cred = New-Object System.Management.Automation.PSCredential('root', $pw)
$op   = New-SSHSession -ComputerName 'vcf-ops.lab.local' -Credential $cred -AcceptKey -Force
Invoke-SSHCommand -SessionId $op.SessionId -Command 'systemctl restart vmware-vcops-web' -TimeOut 60
Remove-SSHSession -SessionId $op.SessionId | Out-Null
```

### *A.3 Phase 2 — Cert generation + SSH replace*

See §7 and §9 above for the full multi-step transcript. Key file paths to remember:

| Workstation | Fleet appliance |
|---|---|
| `I:\VCF Health check\vcfa-openssl.cnf` | `/root/vcfa/vcfa-openssl.cnf` |
| `I:\VCF Health check\vcfa-cert-2026-05-16\vcfa.crt` | `/root/vcfa/vcfa.crt` → `/opt/vmware/vlcm/cert/server.crt` |
| `I:\VCF Health check\vcfa-cert-2026-05-16\vcfa.key` | `/root/vcfa/vcfa.key` → `/opt/vmware/vlcm/cert/server.key` |
| — | `/opt/vmware/etc/lighttpd/server.pem` (key + cert concatenated) |

Backups remain on Fleet at: - `/opt/vmware/vlcm/cert/server.crt.bak-<TS>` - `/opt/vmware/vlcm/cert/server.key.bak-<TS>` - `/opt/vmware/etc/lighttpd/server.pem.bak-<TS>` - `/root/fleet-cert-backup-<TS>/cert/` (session-wide backup taken before any replace)

---

## 5. Simple Deploy Wizard — Precheck Gotchas

*Source: VCFA-Simple-Deploy-Wizard-Precheck-Gotchas-2026-05-18.md*

**Prepared by:** Virtual Control LLC **Date:** May 18, 2026 **Document Version:** 1.0 **Classification:** Internal — Lab Environment **VCF Version:** 9.0.1.0 **Component:** VCF Automation ( `vra` / `vcfa` ) **Deploy Profile:** Simple / small (1-

node)

**Scope.** This document catalogs **four wizard-side gotchas** that cause the VCF Automation Simple/small deploy wizard's Precheck step to fail despite all underlying trust/credential issues being resolved. Each gotcha has a non-obvious failure mode, a misleading or incomplete error message, and a deterministic fix. The gotchas surface in this order during a typical first-time deploy of VCFA Simple/small in VCF 9.0.1.

**Companion docs.** - `Fleet-vCenter-Trust-Cascade-Repair-2026-05-18.md` — for the trust/credential cascade that must be resolved before reaching these wizard gotchas. - `VCF-Automation-Deployment-Troubleshooting-Report.md` — for deploy-time failures (LCMVMSP*) that occur after Precheck passes.

---

**Table of Contents**


1. Summary Table
2. Gotcha 1 — Dual-Draft Cert Binding
3. Gotcha 2 — Cert Generator Multi-Line Input Rejected
4. Gotcha 3 — SAN Must Cover Every IP and FQDN the Deploy Uses
5. Gotcha 4 — Cluster Node IP Pool Requires 2 IPs for `small`
6. Gotcha 5 — Nested vSAN ESXi Host Tunings Required (Williamlam)
7. Resource Pool Warning (Non-Blocking)
8. Verified Working Values
9. Recovery Flow When Precheck Fails
10. Lessons Learned
11. Document Information


---

## 1. Summary Table

| # | Gotcha | Wizard error | Fix |
|---|---|---|---|
| 1 | Two drafts in Components page with different cert references | Same Precheck error code twice in a row despite "fixing" cert | Use the **Non Integrated** draft, NOT the **Integrated** one (the latter still references the first broken cert) |
| 2 | Generate Certificate wizard's multi-line text areas reject newline-separated entries with `Invalid Server Domain / FQDN` | "Invalid Server Domain / FQDN" red error | Generate the cert externally with openssl + a config file; Import via API or UI; one SAN per `DNS.N` / `IP.N` line in the openssl config |
| 3 | Cert SANs missing one or more deploy-used hostnames or IPs | `Certificate validation for cluster virtual IP vra:vcfa &mdash; The hosts in the certificate doesn't match with the provided/product hosts` | Cert must cover: cluster VIP FQDN + IP, node 0 FQDN + IP, plus any short-name forms. For Simple/small that's 5 DNS + 2 IPs minimum (see §4) |
| 4 | `small` deploy profile requires **2 IPs** in Cluster Node IP Pool (not 1, despite being a "single-node" deploy) | `IP Pool count check &mdash; Check if required number of` | Add a second IP to the Cluster Node IP Pool. Total: Primary VIP (1) + Cluster Node Pool (2) + |

| # | Gotcha | Wizard error | Fix |
|---|---|---|---|
| | | `IPs 2 are passed for deploy option small` | Additional VIPs (0) = 3 IPs in play, but the precheck only counts the Cluster Node Pool size |
| 5 | Nested vSAN ESXi hosts ship with `1,1,1,0,0` defaults for 5 LSOM/VSAN advanced settings; under VCFA bootstrap burst writes, LSOM unmounts disk groups mid-deploy — etcd inside the bootstrap VM crashes with `LCMVSPHERECONFIG1000095` | Deploy reaches bootstrap stage then fails 4h+ later with "Failed to create services platform cluster" — `[-]etcd failed: reason withheld` in bootstrap log | Set the 5 settings to `0,0,1,1,1` via `esxcli system settings advanced set` on every nested ESXi host BEFORE submit; no reboot required (§6) |

---

## 2. Gotcha 1 — Dual-Draft Cert Binding

When a VCFA deploy wizard run fails partway through and you exit (or the session times out), Fleet creates a **second draft env** for the next attempt instead of overwriting the first. The Components page then shows TWO `automation` rows:

| Section | Env vmid | Cert reference | Status |
|---|---|---|---|
| **Integrated Components** | `<env-vmid-A>` | First cert you generated — possibly broken | Draft |
| **Non Integrated Components** | `<env-vmid-B>` | Second cert (the corrected one) | Draft |

The labels are misleading: "Integrated" doesn't mean "more correct" — it just means an earlier attempt got further into the workflow before failing. If you continued from the **Integrated** draft, you'd hit the same Precheck cert error as the first attempt, because that draft still references the first cert.

### *Identify which draft has which cert (via API)*

```
 ## On Fleet, list drafts
curl -sk -u admin@local:'<password>' \
  https://localhost/lcm/lcops/api/v2/environments \
  | python3 -c "
import json, sys
for e in json.load(sys.stdin):
    print(f'{e[\"environmentId\"]}  envStatus={e[\"environmentStatus\"]}  status={e.get(\"status\")}')"

## For each draft env, fetch its certificate reference
for ENV in <env1> <env2>; do
  su - postgres -c "/opt/vmware/vpostgres/current/bin/psql -U postgres vrlcm -At -c \
    \"SELECT key, value FROM vm_lcops_infrastructure_properties \
       WHERE environment_vmid='$ENV' AND key='certificate';\""
done
```

The `value` is a `locker:certificate:<vmid>:<alias>` string. Pick the draft whose alias is your latest/corrected cert.

### *Fix*

Click **CONTINUE** on the draft whose cert reference is correct, regardless of which section (Integrated/Non Integrated) it appears in. Then delete the orphan draft after the deploy succeeds:

```
 ## Delete orphan VCFA draft via API
ENV=<orphan-env-vmid>
curl -sk -u admin@local:'<password>' \
  -X DELETE \
```

```
  -H "Content-Type: application/json" \
  -d
'{"controllerType":"string","deleteFromInventory":true,"deleteFromVcenter":false,"deleteLbFromSddc":true,"
deleteWindowsVMs":true}' \
  "https://localhost/lcm/lcops/api/environments/$ENV/delete"
```

The `deleteFromVcenter: false` flag is mandatory — otherwise Fleet attempts to delete the (nonexistent) VCFA VMs from vCenter, which would error.

---

## 3. Gotcha 2 — Cert Generator Multi-Line Input Rejected

The Generate Certificate dialog inside the wizard ( `+ → Generate Certificate` ) has multi-line text areas labeled **Server Domain / FQDN** and **IP Address**, with the helper text "Either Server Domain/FQDN or IP Address must be provided."

The dialog **looks** like it accepts multiple entries (the boxes are textareas, not single-line inputs). It does not. Three formats fail:

| Format tried | Result |
|---|---|
| Newline-separated ( `automation-node1.lab.local` `vcfa-mgmt-0.lab.local` ) | `Invalid Server Domain / FQDN` (red error border) |
| Space-separated on one line ( `automation-node1.lab.local vcfa-mgmt-0.lab.local` ) | Cert generates — but treats the whole string as ONE SAN entry (which is then literally `automation-node1.lab.local vcfa-mgmt-0.lab.local` , invalid as a DNS name) |
| Single FQDN, repeated cert generation in wizard | One SAN per cert, can't bundle |

### *Workaround — generate externally with openssl*

```
 ## On Fleet (has openssl), build a config with the full SAN list
cat > /tmp/vcfa-openssl.conf <<'EOF'
[req]
distinguished_name = req_distinguished_name
req_extensions = v3_req
prompt = no

[req_distinguished_name]
CN = automation-node1.lab.local
O  = Virtual Control LLC
OU = LAB
C  = US
L  = Harmony
ST = FL

[v3_req]
keyUsage = critical, digitalSignature, keyEncipherment
extendedKeyUsage = serverAuth, clientAuth
subjectAltName = @alt_names

[alt_names]
DNS.1 = automation-node1.lab.local
DNS.2 = automation-node1
DNS.3 = vcfa-mgmt-0.lab.local
DNS.4 = vcfa-mgmt-0
DNS.5 = vcfa.lab.local
IP.1  = 192.168.1.91
```

```
IP.2  = 192.168.1.92
EOF

openssl req -x509 -newkey rsa:2048 -nodes -days 730 \
  -keyout /tmp/vcfa.key -out /tmp/vcfa.crt \
  -config /tmp/vcfa-openssl.conf -extensions v3_req

openssl x509 -in /tmp/vcfa.crt -noout -text | grep -A1 "Subject Alternative Name"
## Expect: DNS:automation-node1.lab.local, DNS:automation-node1, DNS:vcfa-mgmt-0.lab.local, ...
```

***Import into Fleet locker via API***

```
 ## Read cert + key as JSON-escaped strings
CRT=$(cat /tmp/vcfa.crt | python3 -c "import sys,json; print(json.dumps(sys.stdin.read()))")
KEY=$(cat /tmp/vcfa.key | python3 -c "import sys,json; print(json.dumps(sys.stdin.read()))")

## POST to locker (use 'certificateChain' not 'certificateChainData' — the latter returns
LCM_CERTIFICATE_API_ERROR0001)
cat > /tmp/import.json <<EOF
{
  "alias": "vcfa-import-2",
  "certificateChain": $CRT,
  "privateKey": $KEY,
  "passphrase": ""
}
EOF

curl -sk -u admin@local:'<password>' \
  -X POST -H 'Content-Type: application/json' -d @/tmp/import.json \
  -w "\nHTTP %{http_code}\n" \
  "https://localhost/lcm/locker/api/v2/certificates/import"
## Expect: HTTP 200, JSON body with new vmid + parsed SAN list confirming all entries
```

Then in the wizard's Certificate step, click the dropdown and select the new alias.

---

## 4. Gotcha 3 — SAN Must Cover Every IP and FQDN the Deploy Uses

The Precheck step performs a strict SAN-coverage match against every hostname and IP the deploy will use. For Simple/small, that's:

| Type | Value | Source |
|---|---|---|
| DNS | `automation-node1.lab.local` | Component Properties → FQDN (cluster VIP FQDN) |
| DNS | `vcfa-mgmt-0.lab.local` | Auto-derived from Node Prefix `vcfa-mgmt` + `-0` suffix + domain |
| DNS | `automation-node1` (short) | Implicit (some validators check short-name too) |
| DNS | `vcfa-mgmt-0` (short) | Implicit |
| DNS | `vcfa.lab.local` | Cluster Virtual IP → vcfa → FQDN (separate sub-product internal name) |
| IP | `192.168.1.91` | Primary VIP |
| IP | `192.168.1.92` | Cluster Node IP Pool, entry 1 |

**If you add a second IP to the Cluster Node IP Pool (per Gotcha 4 below), the cert must also cover that IP.** Default is 2 IPs in the pool; if you use IP `192.168.1.93` as the second pool entry, add `IP.3 = 192.168.1.93` to the openssl config above.

### *Wizard error wording*

```
Failed: Certificate validation for cluster virtual IP vra:vcfa
       Certificate Validation
       The hosts in the certificate doesn't match with the provided/product hosts
```

The error names only the cluster VIP ( `vra:vcfa` ) but actually checks all hostnames/IPs in the spec. The wording is misleading.

### *Wildcard certs are documented but rejected by this Precheck*

The wizard's own recommendation reads: "*Use a valid SAN certificate with all hostnames present for product nodes or use a wildcard certificate.*" In practice, wildcard CN ( `*.lab.local` ) is **also rejected** by this same Precheck check — it requires explicit SAN coverage. Stick with the SAN approach.

---

## 5. Gotcha 4 — Cluster Node IP Pool Requires 2 IPs for `small`

Despite being labeled "Simple" / "small" (which implies 1 node), the deploy profile validation requires **2 IPs in the Cluster Node IP Pool**. The exact wizard error:

```
Failed: IP Pool count check.
       Deploy option profile check - (small)
       Check if required number of IPs 2 are passed for deploy option small.
```

This is **separate from**: - **Primary VIP** (always 1 IP — the cluster endpoint) - **Additional VIPs** (optional, typically 0 for Simple)

### *Working pool layout for Simple/small*

| Field | Count | Wizard input format | Example |
|---|---|---|---|
| Primary VIP | 1 IP | Single IP | `192.168.1.91` |
| **Cluster Node IP Pool** | **2 IPs** | **Hyphen-range** — the wizard rejects comma-separated input | **`192.168.1.92-192.168.1.93`** |
| Additional VIPs | 0 (leave empty) | — | — |
| Internal Cluster CIDR | 1 CIDR | `x.x.x.x/yy` | `198.18.0.0/15` |

> **Format gotcha — verified:** typing `192.168.1.92,192.168.1.93` (comma-separated) in the wizard's Cluster Node IP Pool field is rejected as invalid input. The wizard parses **only the hyphen-range form** `start-end` . Fleet stores the parsed pool internally as comma-separated (visible via `vm_lcops_node_properties.ipPool` ), but that's the storage representation, not the input format.

### *Why 2*

Both IPs in the pool are consumed by the VCFA appliance's internal Kubernetes cluster: one for the kubelet/control-plane interface and one for the kube-vip floater (which manages the Primary VIP within the appliance). On a single-

node Simple deploy the second IP is *not* used for a second VCFA VM — it's used by the K8s services running inside the single VM. This is poorly documented; the requirement is enforced only at Precheck time.

#### *Verified env state after fix*

Wizard input (hyphen-range):

```
Cluster Node IP Pool: 192.168.1.92-192.168.1.93
```

Fleet DB storage (comma-separated, post-parse):

```
 ipPool | 192.168.1.92,192.168.1.93
primaryVip | 192.168.1.91
deployOption| small
```

Confirmed by direct DB query against `vm_lcops_node_properties` for the active draft env.

---

## 6. Gotcha 5 — Nested vSAN ESXi Host Tunings Required (Williamlam)

If your VCF Operations stack runs on **nested ESXi** (e.g., ESXi VMs inside VMware Workstation), this Precheck gotcha is **the most expensive failure mode** — Precheck passes cleanly, the deploy reaches the bootstrap stage, the bootstrap VM is created and powered on, and then the workflow fails ~4+ hours later with `LCMVSPHERECONFIG1000095` "Failed to create services platform cluster." The bootstrap log shows the smoking gun:

```
 "failed to wait for apiserver being healthy: [-]etcd failed: reason withheld"
"failed to renew lease vmsp-platform/<...>: timed out waiting for the condition"
"leader election lost"
"dial tcp 198.19.0.1:443: connect: connection refused"
"dial tcp 192.168.1.91:6443: connect: no route to host"
```

#### *Root cause*

Five ESXi advanced settings ship with defaults tuned for **physical hardware** ( `1, 1, 1, 0, 0` ). On nested vSAN, those defaults cause the inner LSOM (Log Structured Object Manager) to misinterpret nested-induced latency as failing devices and unmount disk groups mid-bootstrap. The bootstrap VM's internal etcd loses its backing store; the entire K8s control plane inside the VM collapses.

| # | Setting | Default | Required for Nested |
|---|---|---|---|
| 1 | `/LSOM/VSANDeviceMonitoring` | **1** | **0** |
| 2 | `/LSOM/lsomSlowDeviceUnmount` | **1** | **0** — most dangerous default; LSOM can yank disk groups mid-deploy |
| 3 | `/VSAN/SwapThickProvisionDisabled` | 1 | 1 (already correct) |
| 4 | `/VSAN/GuestUnmap` | **0** | **1** |
| 5 | `/VSAN/FakeSCSIReservations` | **0** | **1** |

#### *Fix — runtime, no reboot*

Apply on EVERY nested ESXi host **BEFORE** clicking Precheck (or any deploy submission):

```
 esxcli system settings advanced set -o /LSOM/VSANDeviceMonitoring -i 0
esxcli system settings advanced set -o /LSOM/lsomSlowDeviceUnmount -i 0
esxcli system settings advanced set -o /VSAN/SwapThickProvisionDisabled -i 1
esxcli system settings advanced set -o /VSAN/GuestUnmap -i 1
esxcli system settings advanced set -o /VSAN/FakeSCSIReservations -i 1
```

### *Verify before submit*

```
 for s in /LSOM/VSANDeviceMonitoring /LSOM/lsomSlowDeviceUnmount \
         /VSAN/SwapThickProvisionDisabled /VSAN/GuestUnmap \
         /VSAN/FakeSCSIReservations; do
  val=$(esxcli system settings advanced list -o $s | awk '/Int Value:/ {print $3}')
  echo "$s = $val"
done
```

**Expected output on every host:**

```
 /LSOM/VSANDeviceMonitoring = 0
/LSOM/lsomSlowDeviceUnmount = 0
/VSAN/SwapThickProvisionDisabled = 1
/VSAN/GuestUnmap = 1
/VSAN/FakeSCSIReservations = 1
```

### *Documented but not by Broadcom*

| Source | Status |
|---|---|
| **William Lam** — "VCF 9.1 Comprehensive ESX Configuration Workarounds for Lab Deployments" | **Names all 5 settings explicitly as required for VCF 9 nested vSAN deployments** |
| Broadcom KB 418755 ( `LCMVSPHERECONFIG1000095` ) | Attributes the same error code to vCenter port 443 connectivity — does NOT mention these settings |
| Broadcom KB 397561 ( `LCMVSPHERECONFIG100009` family) | Attributes to a kind-cluster CIDR conflict — does NOT mention these settings |

The Precheck wizard does **NOT** verify these host-level settings — you can pass every Precheck check and still hit this failure 4 hours into the deploy. **Add this as a pre-flight verification step before clicking Precheck.**

### *Do NOT apply on physical-hardware production vSAN*

These tunings disable safeguards (SMART monitoring, slow-device protection, real SCSI reservations) that exist for good reasons on real hardware. Lab/nested only.

### *Cross-reference*

Full apply procedure (incl. PowerCLI batch loop across all hosts) and the cascading-failure mechanism explained step-by-step are in `Nested-vSAN-WilliamLam-Tunings-Runbook-2026-05-19.md`.

---

## 7. Resource Pool Warning (Non-Blocking)

A common Precheck **Warning** that confuses operators:

```
Warning: Existence check for resource pool in the given vCenter=vcenter.lab.local
        and cluster=vcenter-cl01
        resgroup-4014(vcenter-cl01-mgmt-001)
        Check if resource pool exists in the given vCenter and cluster.
```

This is **not** a "resource pool doesn't exist" warning. The pool clearly exists (the moRef `resgroup-4014` is right there in the message). The Warning fires when LCM is unable to fully validate the pool's reachability via the credentials in the deployment target.

#### *Action*

None — safe to submit. The actual deploy will succeed if the credentials work (verified via the trust cascade repair in `Fleet-vCenter-Trust-Cascade-Repair-2026-05-18.md` ). If the credentials are still broken, the deploy fails further in with a more specific error; Precheck warning is not the right signal to debug auth.

---

## 8. Verified Working Values

This table records the exact values that produce a passing Precheck. Use as a reference for Simple/small VCFA on VCF 9.0.1 with a `/24` flat management network.

| Step | Field | Value |
|---|---|---|
| Deployment | Topology | Simple |
| Deployment | Disk | Thin |
| Deployment | Size | small |
| Certificate | Selected alias | `vcfa-import-2` (externally-generated 7-SAN cert) |
| Infrastructure | vCenter | `vcenter.lab.local` |
| Infrastructure | Cluster | `vcenter-dc01#vcenter-cl01` |
| Infrastructure | Folder | `Discovered virtual machine` (default) |
| Infrastructure | Resource Pool | `vcenter-cl01-mgmt-001` |
| Infrastructure | Network | `pg-vm-mgmt` (bare name, no DVS prefix) |
| Infrastructure | Datastore | `vcenter-cl01-ds-vsan01` |
| Network | Gateway | `192.168.1.1` |
| Network | Netmask | `255.255.255.0` |
| Network | DNS | `192.168.1.230` |
| Network | NTP | `Windows.lab.local` |
| Network | Search Path | `lab.local` |
| Network | Domain | `lab.local` |
| Components | Component FQDN | `automation-node1.lab.local` |
| Components | Component Password | `VCFA root` (locker alias) |
| Components | Cluster VIP → vcfa FQDN | `automation-node1.lab.local` |

| Step | Field | Value |
|---|---|---|
| Components | Controller Type | `Internal Load Balancer` |
| Components | Node Prefix | `vcfa-mgmt` |
| Components | Primary VIP | `192.168.1.91` |
| Components | Internal Cluster CIDR | `198.18.0.0/15` |
| Components | Additional VIPs | (empty) |
| Components | Cluster Node IP Pool (wizard INPUT format) | **`192.168.1.92-192.168.1.93`** (hyphen-range, NOT comma) |
| Components | (Fleet DB stores it as) | `192.168.1.92,192.168.1.93` |

### *Cert SAN list verified passing*

```
 DNS: automation-node1.lab.local
DNS: automation-node1
DNS: vcfa-mgmt-0.lab.local
DNS: vcfa-mgmt-0
DNS: vcfa.lab.local
IP : 192.168.1.91
IP : 192.168.1.92
```

> **For the verified-passing config above, IP `192.168.1.93` is in the Cluster Node IP Pool but NOT in the cert SAN list, and Precheck still passed.** The SAN check apparently only validates the Primary VIP IP and the first Cluster Node IP — not additional pool entries. This is undocumented behavior; safer practice is to include all pool IPs in SAN. If you regenerate the cert, add `IP.3 = 192.168.1.93` to the openssl config.

---

## 9. Recovery Flow When Precheck Fails

When Precheck fails with one of the gotchas above:

| Failure | Don't | Do |
|---|---|---|
| Gotcha 1 (dual draft) | Click Continue on the same draft repeatedly | Identify the correct draft via API (§2), click Continue on that one |
| Gotcha 2 (newline in cert form) | Repeatedly re-try the Generate Certificate dialog | Generate externally with openssl, import via API (§3) |
| Gotcha 3 (SAN mismatch) | Generate another single-SAN cert via wizard | Regenerate the openssl SAN config to add the missing entry, re-import (§3 + §4) |
| Gotcha 4 (IP count) | Try alternate deploy sizes hoping the count changes | Add a second IP to Cluster Node IP Pool (§5) |
| Gotcha 5 (nested vSAN host tunings) | Re-submit the deploy without changing anything — same etcd failure recurs after another 4 hours | Apply the 5 williamlam settings on every nested ESXi host (§6) BEFORE the next deploy submission. Then clean up the failed env and bootstrap VM per `Failed-VCFA-Deployment-Cleanup-Runbook-2026-05-18.md`. |

### *If you get further than Precheck and a deploy stage fails*

See companion document `VCF-Automation-Deployment-Troubleshooting-Report.md`. The error codes there are `LCMVMSP*` and `LCMVCFA*`, distinct from the `LCMCOMMON*` codes covered in trust-cascade docs.

---

## 10. Lessons Learned

**1. Two drafts after a wizard failure is common.** Always verify which draft references the corrected cert before clicking Continue. The labels "Integrated" vs "Non Integrated" do not indicate which is correct.

**2. The Generate Certificate dialog cannot produce multi-SAN certs.** Use openssl externally with an `alt_names` config block; import via the Fleet locker `POST /lcm/locker/api/v2/certificates/import` endpoint using `certificateChain` (NOT `certificateChainData`).

**3. The cert generator's wildcard option is dishonest.** The wizard recommends "use a wildcard certificate" but the same Precheck check rejects wildcards. Always use explicit SAN entries.

**4. Simple/small VCFA needs 2 Cluster Node IPs, not 1.** The second IP is for the internal K8s control-plane interface inside the appliance, not for a second VM.

**5. Cert SAN coverage matters for Primary VIP IP + first Cluster Node IP only.** The second Cluster Node Pool IP can be outside the cert SAN list and Precheck still passes. Including it anyway is best practice.

**6. The Resource Pool "warning" is signal-less noise.** Treat as informational; do not debug it.

**7. The error wording for Gotcha 3 names only `vra:vcfa` cluster VIP.** Misleading — the actual check covers all hostnames AND IPs in the spec.

**8. Precheck output is the source of truth, not the wizard's "Save & Exit / Continue" state.** Even if the wizard claims values are saved, re-verify by re-entering Precheck after navigation.

---

## 11. Document Information

| Field | Value |
|---|---|
| Title | VCFA Simple Deploy Wizard — Precheck Gotchas Field Reference |
| Document ID | VC-REF-VCFA-PRECHECK-20260518 |
| Version | 1.1 |
| Issued | May 18, 2026 (rev 1.1: May 19, 2026) |
| Author | Virtual Control LLC |
| Classification | Internal — Lab Environment |
| Related Documents | `Fleet-vCenter-Trust-Cascade-Repair-2026-05-18.md`, `Fleet-Trust-Recovery-After-Cert-Rotation-KB402063-2026-05-17.md`, `VCF-Automation-Deployment-Troubleshooting-Report.md`, `Nested-vSAN-WilliamLam-Tunings-Runbook-2026-05-19.md`, `Failed-VCFA-Deployment-Cleanup-Runbook-2026-05-18.md` |

| Field | Value |
|---|---|
| Distribution | Lab notebook; safe to share verified-working values table externally with vCenter/credentials redacted |

---

## 6. Deployment — Verified Fix Runbook

*Source: VCFA-Deployment-Fix-Runbook-2026-05-08.md*

**Prepared by:** Virtual Control LLC **Date:** May 8, 2026 **Document Version:** 1.0 **Classification:** Internal — Lab Environment **VCF Version:** 9.0.1.0

---

**Table of Contents**


1. Executive Summary
2. Confidence and Uncertainty
3. Environment Reference
4. Pre-flight Checks
5. Phase 1 — Fix vSAN OSA Disk Misclassification
   - 5.1 Target disks by host
   - 5.2 The procedure (per host)
   - 5.3 Suggested host order and blast radius
   - 5.4 Rollback if something goes wrong
6. Phase 2 — Clean the failed VCFA environment
7. Phase 3 — Fresh VCFA deploy
8. Optional — C: drive swap to NVMe
9. Recovery Verification
10. Broadcom References
11. Document Information


---

### 1. Executive Summary

After six failed attempts to deploy VCF Automation 9.0.1 in this homelab, root cause was verified on 2026-05-08:

**Root cause chain (verified):**

1. **Layer 1 (root):** vSAN OSA cluster has capacity disks reporting `SSD=False` on physical NVMe/SSD media. Verified via vCenter PowerCLI VsanSystem enumeration — 4 capacity disks across 3 hosts are misclassified as HDD.
2. **Layer 2:** vSAN runs those disk groups in hybrid mode for the misclassified disks → all writes destage through cache to "HDD" capacity tier → ~70 ms write latency, 0% Local Client Cache hit rate.
3. **Layer 3:** The VCFA workload K8s appliance VM lives on this vSAN datastore. etcd's WAL fsync inherits the slow path → `apply request took too long` warnings of 1-3 seconds (expected 100 ms).
4. **Layer 4:** kube-apiserver depends on etcd. Its PostStartHook `rbac/bootstrap-roles` calls itself at `https://[::1]:6443/...` to reconcile cluster roles. The listener isn't ready yet because etcd handshake is

slow → connection refused → after timeout, fatal exit (exit code 255).
5. **Layer 5 (visible symptom):** kubelet retries apiserver every ~3 minutes. 19+ hours of crashlooping (event count: 202 since 2026-05-08 00:51 UTC). containerd accumulated 50+ stuck `Created` containers per pod from the retry storm.

**Fix sequence:**

- **Phase 1** — Per-host vSAN OSA disk group rebuild with `esxcli vsan storage tag add -t capacityFlash` on the misclassified disks (eliminates the root cause).
- **Phase 2** — Delete the failed VCFA environment from Fleet (the current workload K8s cluster cannot recover; must redeploy from clean).
- **Phase 3** — Fresh VCFA deploy. Bootstrap will succeed because vSAN is now correctly classified all-flash; etcd writes commit in <100 ms; apiserver PostStartHook completes; cluster comes up healthy.

---

## 2. Confidence and Uncertainty

This runbook follows the user's "no guessing" rule. Each instruction is annotated with confidence:

- **VERIFIED** — directly observed in this lab on 2026-05-08, OR documented verbatim in a Broadcom KB.
- **DOCUMENTED** — taken from official Broadcom or upstream Kubernetes documentation but not yet executed in this lab.
- **INFERRED** — reasoned from related verified facts; flagged where used.

**Things I am explicitly NOT confident about** and where the operator should verify before acting:

- Whether nested-ESXi virtual disks accept the `capacityFlash` tag the same way physical flash does. Broadcom KB 315532 documents the procedure for physical disks. Memory note `vsan_osa_allflash_grandfather` says capacityFlash tag "requires real flash device" — but those notes were about *adding* HDDs to all-flash clusters, not retagging virtual disks already on flash. **Test on `esxi03` first** (1 disk, smallest blast radius) before committing to esxi02 (2 disks) and esxi04 (1 disk).
- Whether the VCFA deploy will reuse the cached registry layers across the new env. Memory says ~4.1 GB was cached in the prior `registry` Pod's PVC; the new appliance VM is fresh and the PVC is destroyed when the env is deleted. Plan for full layer pull (~3.2 GB across the wire from Fleet) on the next deploy.
- Whether Fleet's `vmsp delete env --force` cleanly removes ALL state including the LCM database row. RCA Section 12 documents the manual database cleanup as a backup plan if the UI delete leaves orphans.

---

## 3. Environment Reference

| Component | Value | Source |
|---|---|---|
| Workstation | Dell Precision 7920 Tower (dual-CPU) | memory `user_workstation` |

| Component | Value | Source |
|---|---|---|
| Hosts | 4 nested ESXi VMs: esxi01-04 at 192.168.1.74 / .75 / .76 / .82 | memory `reference_homelab_esxi_nodes` |
| vSAN cluster | `vcenter-cl01`, OSA mode, FTT=1 | verified 2026-05-08 PowerCLI |
| Storage | All physical media is SSD/NVMe (no HDDs in service) | memory `reference_7920_storage_topology` |
| vCenter | `vcenter.lab.local` (192.168.1.69) | memory `user_homelab` |
| Fleet (LCM) | `lcm.lab.local` (192.168.1.95) | memory `user_homelab` |
| VCF Operations | `vcf-ops.lab.local` (192.168.1.77) | memory `user_homelab` |
| NSX | `nsx-manager.lab.local` (192.168.1.71) | memory `user_homelab` |
| Failed envId (cleanup target) | `601ea82d-1f72-4d8e-848a-125265ad1aa7` | verified 2026-05-08 |
| Failed requestId | `7405ad8b-78a2-470a-ad8e-84d121344814` | verified 2026-05-08 |
| Broken workload appliance VM | `vcfa-mgmt-wsv57` (192.168.1.96) on esxi04 | verified 2026-05-08 |
| Failed VIP | `192.168.1.91` (kube-vip cannot bind) | verified 2026-05-08 |

---

## 4. Pre-flight Checks

Run these BEFORE starting Phase 1. If any check fails, stop and resolve before proceeding.

### *4.1 Verify cluster currently has FTT=1 redundancy headroom*

**Run from:** any system with PowerCLI 13+ and network access to vCenter.

```
Connect-VIServer vcenter.lab.local -User 'administrator@vsphere.local' -Password '<password>'

## Check cluster health summary:
Get-Cluster vcenter-cl01 | Get-VMHost | Select Name, ConnectionState, PowerState

## All 4 hosts must show: Connected / PoweredOn
```

**Expected:** All 4 hosts (esxi01-04) `Connected` and `PoweredOn`.

**If any host is in MM, NotResponding, or PoweredOff:** STOP. Resolve before starting Phase 1. The disk-group rebuild requires 4 hosts active for FTT=1 RAID1 (one in MM leaves 3 — the minimum for vSAN to keep mirrors compliant).

### *4.2 Verify vSAN cluster has no current resync activity*

**Run from:** SSH session to any inner ESXi host (e.g., `ssh root@esxi01.lab.local`).

```
esxcli vsan debug object health summary get
```

**Expected:** Every category reports `0` objects.

**If any category > 0:** STOP. Wait for vSAN resync to complete before starting Phase 1. Memory `feedback_vsan_resync_first_check` records this as a hard prerequisite.

### *4.3 Verify free vSAN capacity for full-data-migration evacuation*

**Run from:** SSH to any inner ESXi host.

```
esxcli vsan storage list | grep -E "Capacity|Used"
```

**Expected:** Each host's used capacity should be < (cluster total free) / 4. With FTT=1 mirroring across 4 hosts, evacuating one host's worth of data needs that capacity available across the remaining 3 hosts.

**Threshold rule of thumb:** If any single host's used vSAN capacity exceeds 25% of cluster total, full data migration may fail with "insufficient capacity." If you hit this, use `Ensure Object Accessibility` mode instead — but be aware of memory note `feedback_vsan_mm_full_data_migration` which records that EA caused a vCenter journal abort on 2026-05-01. **Prefer Full Data Migration when capacity allows.**

### *4.4 Confirm DNS, Fleet, NSX are healthy (independent of broken VCFA)*

```
 ## From the workstation:
Test-NetConnection -ComputerName vcenter.lab.local -Port 443 -WarningAction SilentlyContinue
Test-NetConnection -ComputerName lcm.lab.local -Port 443 -WarningAction SilentlyContinue
Test-NetConnection -ComputerName nsx-manager.lab.local -Port 443 -WarningAction SilentlyContinue
Test-NetConnection -ComputerName vcf-ops.lab.local -Port 443 -WarningAction SilentlyContinue
```

**Expected:** Each `TcpTestSucceeded : True`.

---

## 5. Phase 1 — Fix vSAN OSA Disk Misclassification

### *5.1 Target disks by host*

**Verified 2026-05-08 via PowerCLI VsanSystem enumeration:**

| Host | Misclassified capacity disks | Action needed |
|---|---|---|
| esxi01 | None | Skip |
| **esxi02** | 2 disks (the 400 GB and 100 GB capacity virtuals) | Rebuild disk group |
| **esxi03** | 1 disk (the 100 GB capacity virtual) | Rebuild disk group |
| **esxi04** | 1 disk (the 400 GB capacity virtual) | Rebuild disk group |

Cache tier on every host already shows `SSD=True` — leave the cache disks alone.

### *5.2 The procedure (per host)*

**Source:** Broadcom KB 315532 ("How to manually remove and recreate a vSAN disk group") + KB 327008 (capacity disk replacement procedure).

The procedure below is verbatim from KB 315532 with the `capacityFlash` tag step inserted between disk-group remove and disk-group re-create.

#### Step 1 — Identify exact device names (NAA IDs) on the host

**Run from:** SSH to the inner ESXi host (e.g., `ssh root@esxi02.lab.local`). Use the ESXi root password from your credential vault (memory `reference_homelab_esxi_creds`).

```
esxcli vsan storage list
```

**Expected (excerpt):** for each disk, you'll see fields including `Device:`, `In CMMDS:`, `Is SSD:`, `Used by this host:`. **Find every disk under "Used by this host: true" where "Is SSD: false"** — those are your retag targets.

```
esxcli vsan storage list | grep -B 4 "Is SSD: false"
```

**Record:** - The capacity disk's NAA (`naa.xxxxxxxxxxxxxxx`) — you'll need this for the tag step - The cache disk's NAA — you'll need this when re-creating the disk group - **The VSAN Disk Group UUID** of the cache disk — required for the remove step

#### Step 2 — Enter Maintenance Mode with Full Data Migration

**Run from:** vCenter UI is the safest path — avoids CLI typos.

- vCenter → Hosts and Clusters → Cluster `vcenter-cl01` → right-click host (e.g., esxi02) → Maintenance Mode → Enter Maintenance Mode
- In the dialog, choose **"Full data migration"** (NOT "Ensure accessibility", NOT "No data migration")
- Click OK

**Or via SSH (CLI equivalent per Broadcom KB 315532):**

```
esxcli system maintenanceMode set --enable true -m evacuateAllData
```

**Expected:** Host transitions to MM. vSAN evacuates all objects from this host to the other 3 hosts. Wall time: 10-30 min depending on data volume.

**Verify MM with no warnings:**

```
esxcli system maintenanceMode get # returns "Enabled"
esxcli vsan debug object health summary get # all categories 0 -- if non-zero, wait for resync
```

#### Step 3 — Remove the existing disk group

```
## Use the VSAN Disk Group UUID you recorded in step 1:
esxcli vsan storage remove -u <VSAN-Disk-Group-UUID>
```

**Expected:** Command returns silently on success. Verify:

```
esxcli vsan storage list
## Should show NO disk group on this host
```

> **Critical:** double-check the UUID before running `esxcli vsan storage remove`. The KB explicitly warns: "*Always double check the disk group UUID.*" Removing the wrong group could destroy data on a healthy host. Since

we're already in MM with full data migration, the local data is empty — but the command operates on UUIDs, not just "this host", so type carefully.

#### Step 4 — Apply `capacityFlash` tag to each misclassified disk

For each disk you recorded as `Is SSD: false`:

```
esxcli vsan storage tag add -d <naa.xxxxxxxxxxxxxxx> -t capacityFlash
```

#### Step 5 — Verify the tag stuck

```
vdq -q -d <naa.xxxxxxxxxxxxxxx> | grep -E "IsCapacityFlash|IsSSD"
```

**Expected:**

```
"IsCapacityFlash": 1
```

**INFERRED — uncertain:** `IsSSD` may still report `0` because the underlying VMware Workstation virtual disk doesn't propagate `is_ssd=true` from the physical layer. The `capacityFlash` tag is what vSAN OSA actually checks for tier selection. **If `IsCapacityFlash: 1` is set, vSAN should treat the disk as flash regardless of `IsSSD`.** Verify this in step 7 by checking that the recreated disk group reports SSD=True via PowerCLI.

#### Step 6 — Re-create the disk group with the cache disk and tagged capacity disks

```
## Single-line; use the cache and ALL capacity NAAs you recorded:
esxcli vsan storage add -s naa.<cache-naa> -d naa.<capacity1-naa> -d naa.<capacity2-naa> -d naa.
<capacity3-naa>
```

**Expected:** Command returns silently. Verify with:

```
esxcli vsan storage list | grep -E "Device|In CMMDS|Is SSD|Used by"
```

All disks should show: - `In CMMDS: true` - `Used by this host: true`

#### Step 7 — Verify from vCenter that capacity tier reports as Flash

**Run from:** PowerCLI on the workstation.

```
$h = Get-VMHost esxi02.lab.local
$sys = Get-View -Id $h.ExtensionData.ConfigManager.VsanSystem
$sys.Config.StorageInfo.DiskMapping | ForEach-Object {
  Write-Host "Cache:    $($_.Ssd.CanonicalName) SSD=$($_.Ssd.Ssd)"
  foreach ($cap in $_.NonSsd) {
    Write-Host "Capacity: $($cap.CanonicalName) SSD=$($cap.Ssd)"
  }
}
```

**Expected after fix:** every line ends `SSD=True`. If any capacity row still shows `SSD=False`, the tag may need re-applying or the disk group may need a second rebuild — STOP and investigate before exiting MM.

#### Step 8 — Exit Maintenance Mode

**vCenter UI:** right-click host → Maintenance Mode → Exit Maintenance Mode

**Or via SSH:**

```
esxcli system maintenanceMode set --enable false
```

#### Step 9 — Wait for vSAN resync to complete

```
esxcli vsan debug object health summary get
```

**Expected:** All categories return to `0` after resync. Wall time: 10-60 minutes depending on data being mirrored back.

**DO NOT start the next host's procedure until this completes.** The cluster degrades to barely-FTT=1 if a second host enters MM while the first is still resyncing.

### *5.3 Suggested host order and blast radius*

| # | Host | Capacity disks to retag | Reason for order |
|---|---|---|---|
| 1 | **esxi03** | 1 (100 GB) | Smallest blast radius — fewest disks. Use as canary to confirm the procedure works before committing to esxi02 and esxi04. |
| 2 | **esxi02** | 2 (400 GB + 100 GB) | Two disks — second-smallest impact. |
| 3 | **esxi04** | 1 (400 GB) | This host currently runs the broken VCFA appliance VM ( `vcfa-mgmt-wsv57` ). Doing it last means we don't migrate the broken VM during fix; we'll delete it in Phase 2 instead. |

### *5.4 Rollback if something goes wrong*

If a disk group fails to re-create OR vSAN reports degraded after exit-MM:

1. **Do NOT panic.** Maintenance Mode with FDM evacuated all data; the local disk group held no unique data when removed.
2. SSH back to the host, check `esxcli vsan storage list` — if the disk group is incomplete, remove it again with `esxcli vsan storage remove -u <UUID>` .
3. Remove the `capacityFlash` tags if you want to revert: `esxcli vsan storage tag remove -d <naa> -t capacityFlash` (per KB 315532's storage-tag mechanics).
4. Re-create the disk group WITHOUT the tag: `esxcli vsan storage add -s <cache-naa> -d <capacity-naa> ...`
5. The host returns to its prior misclassified state (which was bad but not broken).

**Worst case:** you abort the rebuild, exit MM, and have one host with no disk group. The vSAN cluster keeps running with 3 hosts contributing storage. Re-add the disk group from vCenter UI later.

---

## 6. Phase 2 — Clean the failed VCFA environment

### *6.1 Path A — Fleet UI (preferred)*

**Run from:** browser, logged into VCF Operations as `admin@local` .

1. Navigate: vcf-ops → Fleet Management → Lifecycle → VCF Management → **Tasks**
2. Find the FAILED environment row for envId `601ea82d-1f72-4d8e-848a-125265ad1aa7`
3. Click the kebab menu (⋮) → **Delete environment**
4. Confirm the destructive prompt

**What this does (DOCUMENTED — Broadcom KB 410338 retry-then-clean path):** Fleet executes `vmsp delete env --force` which: - Kills the workload appliance VM ( `vcfa-mgmt-wsv57` ) in vCenter - Removes the `prelude` namespace and associated K8s resources - Deletes the LCM database row for the environment - Removes the cluster bootstrap state

**Wall time:** 10-30 minutes.

**Expected end state:** - vCenter: no `vcfa-mgmt-*` VMs in inventory - Fleet UI: no environment with that envId in the Tasks list - Fleet UI: no `vcfa-bundle` in the Components page

### *6.2 Path B — API + database cleanup (if UI delete fails)*

**Source:** `VCF-Automation-Deployment-Troubleshooting-Report.md` Section 12 (full SQL cleanup script). Use this only if the UI Delete leaves orphaned database records.

Refer to that section for the exact `psql` commands; do not improvise.

---

## 7. Phase 3 — Fresh VCFA deploy

### *7.1 Pre-deploy preconditions*

**VERIFIED — already in place from prior attempts (do not redo):**

- vSAN OSA capacity tier all-flash (Phase 1 just done)
- Fleet nginx `/repo/` location has `proxy_read_timeout 120m` + `proxy_send_timeout 120m` (applied 2026-05-08 18:31; backup at `/etc/nginx/nginx.conf.bak-20260508-183153` )
- Fleet's self-signed CA injected into workload `platform-trust` Trust Manager Bundle — this state is destroyed by Phase 2 along with the appliance, so it WILL need re-injection AFTER the new appliance bootstraps. See §11.7.13.1 of the RCA.
- Fleet log-bundle script at `/data/LogBundleSupport.sh` (per KB 394146) — keep available for diagnostics.

**To verify before deploy (DOCUMENTED — from RCA §11.7.11 preventive checklist):**

1. **DRS in Partially Automated** mode on `vcenter-cl01` (NOT Fully Automated — prevents mid-deploy vMotion).
2. **vCenter VM is NOT on the host VCFA will land on.** Pre-position vCenter on a different host before submit.
3. **DNS resolution verified** from the Fleet appliance for: `automation-node1.lab.local` → `192.168.1.91` , `vcfa-mgmt-{0,1,2}.lab.local` → `.92/.93/.94` .
4. **Cert in locker:** `vcfa-import` entry exists with valid cert chain (SAN must include all VCFA FQDNs).
5. **Password in locker:** `VCFA root` entry.

### *7.2 Submit the deploy*

**Run from:** Fleet appliance shell (192.168.1.95) OR via VCF Operations UI.

Per RCA §11.4 (API deployment path), the recommended submission is:

```
 ## On Fleet, with the vra-deploy.json file at /root/vra-deploy.json:
curl -sk -u 'admin@local:<password>' \
```

```
  -H 'Content-Type: application/json' \
  -X POST \
  -d @/root/vra-deploy.json \
  https://localhost/lcm/lcops/api/v2/environments
```

**Expected:** HTTP 202 Accepted with a JSON body containing `requestId` and `envId`. Record both.

### *7.3 Apply systemd-resolved /etc/hosts workaround on the new appliance*

**Run from:** any system with PowerCLI access to vCenter, AS SOON AS the new VCFA appliance VM appears in vCenter inventory and finishes first-boot (~20-30 min after submit).

**Why:** Per RCA §11.7.12, `systemd-resolved` on the appliance fails to resolve lab FQDNs even when the DNS server is reachable. SSH password auth is disabled — use Guest Operations API.

```
 $cred = New-Object System.Management.Automation.PSCredential('vmware-system-user', `
  (ConvertTo-SecureString '<vcfa-root-password>' -AsPlainText -Force))

$vm = Get-VM 'vcfa-mgmt-*' # the new appliance — match by current name pattern

$bash = @'
sudo bash -c 'cat >> /etc/hosts << EOF
192.168.1.95   lcm.lab.local fleet.lab.local
192.168.1.69   vcenter.lab.local
192.168.1.91   automation-node1.lab.local
192.168.1.71   nsx-manager.lab.local
192.168.1.77   vcf-ops.lab.local
192.168.1.230  sddc-manager.lab.local
EOF'
'@

Invoke-VMScript -VM $vm -ScriptText $bash -GuestCredential $cred -ScriptType Bash
```

**Expected:** Command returns silently. Verify by re-running:

```
curl -sk --max-time 10 -o /dev/null -w "HTTP %{http_code}\n" \
  https://lcm.lab.local/repo/productBinariesRepo/vra/9.0.1.0/install/vra.tar
```

Returns `HTTP 200` — proves the bundle is reachable through `/etc/hosts` resolution.

### *7.4 Inject Fleet self-signed CA into Trust Manager Bundle*

**Run from:** the same Guest Ops / Invoke-VMScript context.

Per RCA §11.7.13.1:

```
 ## Inside Invoke-VMScript on the new appliance:
KCT=/tmp/kubectl
KC=/etc/kubernetes/admin.conf

## 1. Pull Fleet's CA from its own port 443:
openssl s_client -connect lcm.lab.local:443 -servername lcm.lab.local </dev/null 2>/dev/null \
  | sed -n '/-----BEGIN CERTIFICATE-----/,/-----END CERTIFICATE-----/p' > /tmp/fleet-ca.pem

## 2. Verify it's the right cert (subject CN should be lcm.lab.local):
openssl x509 -in /tmp/fleet-ca.pem -noout -subject -issuer

## 3. Create the labeled Secret:
sudo $KCT --kubeconfig $KC create secret generic fleet-locker-ca \
  -n vmsp-platform \
```

```
  --from-file=ca.crt=/tmp/fleet-ca.pem
sudo $KCT --kubeconfig $KC label secret fleet-locker-ca \
  -n vmsp-platform trust.vmsp.vmware.com/bundle=platform-trust

## 4. Verify Trust Manager picked it up:
sudo $KCT --kubeconfig $KC get bundle.trust.cert-manager.io platform-trust -o yaml | grep -A5 "status:"
```

**Expected:** `status.conditions` shows `Synced: True`.

> This step is required because the workload K8s cluster's bundle controller validates the Fleet TLS chain against `platform-trust` ConfigMap. Without injecting Fleet's self-signed CA, the bundle download will fail with TLS verify error code 18 — verified failure mode from the 6th attempt.

### *7.5 Monitor the deploy*

**Run from:** Fleet appliance shell.

```
 ## Get the new requestId from the POST response, then:
RID='<new-request-id>'
watch -n 30 "curl -sk -u 'admin@local:<password>' https://localhost/lcm/request/api/v2/requests/$RID |
python3 -m json.tool | grep -E '\"state\"|\"requestReason\"' | head -5"
```

**Expected stage transitions (DOCUMENTED — from RCA §11.7.11 5th-attempt timing):**

| Stage | Expected duration | Signal |
|---|---|---|
| Validation + bundle prep | seconds | quick |
| **Bootstrap (kubeadm-init + etcd)** | **~20 min on healthy flash vSAN** | the stage that previously hung at HDD-tier latency |
| Antrea CNI + registry deploy | 5-10 min | network/CNI ready |
| OVA deploy → first-boot | 5-10 min | VM appears in vCenter |
| Cluster init + VIP binding + cert install | 5-10 min | VIP `192.168.1.91` becomes reachable |
| **VCFA bundle pull + push ( `prelude` )** | **15-25 min on healthy flash** | Bundle CR `vra` reaches `Ready` |
| Health checks + integration | 5-10 min | `prelude` namespace gets pods (Postgres, vco-app, tenant-manager, blueprinting, catalog, CCS) |

**Total expected wall time: 60-90 minutes on healthy flash vSAN.**

---

## 8. Optional — C: drive swap to NVMe

Separate maintenance event. Do AFTER Phase 1 completes successfully and BEFORE Phase 3 if you want a drive-swap maintenance window before redeploying.

### *8.1 Clone now, swap tomorrow — data drift*

If you cloned C: today and physically swap tomorrow, any C: changes between clone and swap are lost on the new drive. Mitigation: re-run the same Macrium clone job tomorrow morning right before the swap. With Delta enabled (your settings show `Delta: Y`), the second clone only writes changed blocks and completes much faster than the first run.

### *8.2 Swap procedure*

1. Verify Phase 1 vSAN fix is fully complete (all hosts SSD=True, no resync activity).
2. Maintenance window — clean shutdown of nested ESXi VMs per `VCF_Lab_Shutdown_Startup_Guide.md`.
3. Power off workstation.
4. Install new 4 TB NVMe (PCIe adapter or U.2 FlexBay or Drive Duo/Quad — whichever form factor you have). NO RAW wipe needed for NVMe per memory `feedback_wipe_used_ssd_vmd` (NVMe driver path is not affected by the iaStorAVC issue).
5. Set boot order in BIOS (F12 boot menu) to the new VMD-enumerated NVMe.
6. Power on → **Safe Mode w/ Networking first** (Shift+Restart → Troubleshoot → Advanced → Startup Settings → 5) per memory `feedback_safe_mode_storage_verify`.
7. Verify Windows boots in Safe Mode → reboot to normal Windows → verify desktop.
8. Power on nested ESXi VMs in correct order.
9. Verify vSAN comes up healthy: `esxcli vsan debug object health summary get` returns 0 in every category.
10. Proceed to Phase 2 + Phase 3.

---

## 9. Recovery Verification

After Phase 3 completes successfully, verify by running these checks:

### *9.1 Fleet env state*

**Run from:** Fleet appliance.

```
/root/vra-poll.sh # custom script returns: STATE|envId|errorCause
```

**Expected:** `COMPLETED|<new-envId>|` (errorCause field empty).

### *9.2 VCFA UI reachable*

**Run from:** browser.

Navigate to `https://automation-node1.lab.local/`. **Expected:** VCFA login page loads.

### *9.3 Workload K8s cluster healthy*

**Run from:** Guest Ops / Invoke-VMScript on the appliance.

```
 sudo /tmp/kubectl --kubeconfig /etc/kubernetes/admin.conf get nodes
sudo /tmp/kubectl --kubeconfig /etc/kubernetes/admin.conf get pods -A | grep -vE 'Running|Completed'
```

**Expected:** - All nodes `Ready` - No pods stuck `CrashLoopBackOff`, `Pending`, or `ContainerCreating`

### *9.4 etcd performance verified*

```
ETCDID=$(sudo crictl ps | grep ' etcd ' | awk '{print $1}' | head -1)
```

```
sudo crictl logs --tail=200 "$ETCDID" 2>&1 | grep -c "apply request took too long"
```

**Expected:** `0` — no slow-apply warnings since the cluster came up. **This is the key indicator that the Phase 1 vSAN fix worked.**

### *9.5 No 30-minute EOF on bundle pulls*

```
grep -E 'EOF|reset|timeout' /var/log/nginx/error.log | tail -5
```

**Expected:** No `readv() failed (104: Connection reset by peer)` patterns dating to the new deploy.

---

## 10. Broadcom References

| KB / Doc | Title | Used in this runbook |
|---|---|---|
| KB 315532 | How to manually remove and recreate a vSAN disk group | Phase 1 disk-group rebuild procedure |
| KB 327008 | Identifying and replacing a failed cache or capacity disk in vSAN OSA disk group | Reference for disk replacement semantics |
| KB 326857 | vSAN host fails to enter MM with Full data migration | Pre-flight headroom requirement |
| KB 410338 | Deployment of VCFA from Fleetmgmt fails with deadline exceeded (LCMVMSP10002) | Retry-then-clean Fleet path; cert/FQDN preventive checklist |
| KB 406528 | LCMVMSP10002 error when deploying VCF Automation 9.0 | Lowercase FQDN requirement |
| KB 411354 | VCF Automation 9 install through Fleet fails LCMVMSP10002 | Cert SAN requirements |
| KB 394146 | Generating a log bundle from a failed VCF Automation deployment | Diagnostic capture if Phase 3 fails |
| KB 382405 | Stuck vSAN catalog vmnamespace blocking CSI volume provisioning | Issue 12 root cause if it recurs |
| KB 415800 | VCF 9 known issues | Cross-referenced — does NOT cover the 30-min EOF pattern observed in this lab |
| TechDocs | VCF Automation 9.0 known issues page | Cross-referenced — no match for our containerd state corruption either |

**Verified 2026-05-08:** None of the Broadcom KBs above directly cover the specific apiserver `[::1]:6443 connection refused` PostStartHook timeout pattern this lab hit. The fix in this runbook treats the underlying vSAN performance problem (which is documented in KB 315532) and lets the deploy succeed naturally. If the next deploy still hits the apiserver crashloop AFTER the vSAN fix, generate a log bundle (KB 394146) and consider a Broadcom support case.

---

### 11. Document Information

| Field | Value |
|---|---|
| Document Title | VCF Automation 9.0.1 Deployment — Verified Fix Runbook |
| Prepared By | Virtual Control LLC |
| Date | May 8, 2026 |
| Document Version | 1.0 |
| Classification | Internal — Lab Environment |
| VCF Version | 9.0.1.0 |
| Source RCA | `VCF-Automation-Deployment-Troubleshooting-Report.md` v4.6 (cover 2.1) — sections 11.7.13 and 14 contain the verified failure analysis this runbook implements |
| Broadcom KBs Referenced | 315532, 327008, 326857, 410338, 406528, 411354, 394146, 382405, 415800 |
| Root Cause Layer Verified | vSAN OSA capacity disk SSD=False misclassification on nested ESXi virtual disks |
| Confidence Level | High on Phase 2 + 3 (verified-Broadcom paths). **Medium on Phase 1** — virtual-disk capacityFlash tagging is documented for physical disks; nested-ESXi behavior to be verified on the esxi03 canary host. |

---

*This runbook was derived from real troubleshooting of the 6th VCF Automation deploy attempt on 2026-05-08. Every command was either executed in the lab or pulled verbatim from a cited Broadcom KB. Where I am uncertain, I have flagged the step explicitly. Always test on the smallest-blast-radius host first.*



---

## 7. Deploy Monitoring & Health Checks

*Source: VCFA-Deploy-Monitoring-Checks-2026-05-20.md*

**Prepared by:** Virtual Control LLC **Date:** May 20, 2026 **Document Version:** 1.0 **Classification:** Internal — Lab Environment **VCF Version:** 9.0.1.0

> **Scope.** Verified check commands used to monitor a VCFA Simple deploy in a nested ESXi lab. Each section gives the exact query, what its output means, and what "good" looks like. Copy-paste reference for live deploys.
>
> **Companion docs.** - `VCFA-Simple-Deploy-Wizard-Precheck-Gotchas-2026-05-18.md` — precheck gotchas to verify BEFORE submit - `Failed-VCFA-Deployment-Cleanup-Runbook-2026-05-18.md` — cleanup procedure for failed envs

---

**Table of Contents**

1. TL;DR — The Five Checks You Actually Run
2. Env Status (via Postgres)
3. Bootstrap Log (tail + grep)
4. Fleet KIND Control Plane Health
5. Fleet Load Average + Thresholds
6. VCFA Application Endpoint Probes
7. NFS Share Usage + Write Activity
8. vCenter-side Checks (Bootstrap VM + Tasks)
9. What "Good" Looks Like Per Phase
10. The Consolidated One-Shot Status Script
11. Document Information

---

## 1. TL;DR — The Five Checks You Actually Run

For a live deploy, you only need five fast queries to know whether everything is healthy or failing:

| # | Check | Tool | What it tells you |
|---|---|---|---|
| 1 | `environmentstatus` column in `vm_lcops_environments` | psql on Fleet | INPROGRESS / FAILED / COMPLETED |
| 2 | `tail` the bootstrap log + grep for `ERR:` / `LCMVSPHERE` / `etcd failed` | SSH to Fleet | Current phase + any failure beacon |
| 3 | Fleet KIND control plane pods + restart counts | docker exec kubectl | Fleet-side K8s stability |
| 4 | `uptime` on Fleet | SSH to Fleet | Load / saturation level vs nproc |
| 5 | `curl https://<primary-vip>/` and `/provider` | curl from Fleet | VCFA web app binding state |

A complete one-shot script that runs all five together is in §10.

---

## 2. Env Status (via Postgres)

The single source of truth for deploy state is Fleet's `vrlcm` Postgres database, table `vm_lcops_environments`.

### *2.1 The query*

```
 ## On Fleet (SSH as root):
sudo -u postgres /opt/vmware/vpostgres/14/bin/psql -d vrlcm -At -c \
  "SELECT vmid, environmentstatus, to_timestamp(lastupdatedon/1000) \
   FROM vm_lcops_environments \
   WHERE vmid='<env-uuid>';"
```

### *2.2 Output interpretation*

| Output column | Meaning |
|---|---|
| `vmid` | The env UUID from the POST response |
| `environmentstatus` | The Fleet-tracked state. Values seen: empty (just submitted, pre-INIT0001), `INPROGRESS`, `FAILED`, `COMPLETED` |
| `to_timestamp` of `lastupdatedon/1000` | Last time Fleet wrote anything to this row. **If frozen for >15 min during INPROGRESS, the workflow may be stalled** |

### *2.3 What "good" looks like*

- `environmentstatus = INPROGRESS` with `lastupdatedon` advancing every few minutes — deploy is actively working
- `environmentstatus = COMPLETED` — success

### *2.4 What "bad" looks like*

- `environmentstatus = FAILED` — deploy is dead; cross-reference the bootstrap log (§3) and Fleet KIND health (§4) for the failure beacon
- `lastupdatedon` frozen for >30 min during INPROGRESS — check Fleet KIND health (§4) and bootstrap log (§3) for clues; may indicate a stalled retry loop

### *2.5 Listing all envs (sanity check)*

```
sudo -u postgres /opt/vmware/vpostgres/14/bin/psql -d vrlcm -At -c \
  "SELECT vmid, environmentstatus FROM vm_lcops_environments;"
```

Expected during a deploy: one INPROGRESS row for the current env, plus the existing COMPLETED vrops env (e.g., `97dee43f-...`). Any extra FAILED rows are orphans — see [Failed-VCFA-Deployment-Cleanup-Runbook-2026-05-18.md].

---

## 3. Bootstrap Log Tail and Grep

Fleet writes the VMSP bootstrap workflow output to `/var/log/vrlcm/vmsp_bootstrap_<env-uuid>.log`. This is the **phase tracker** — it shows what the workflow is currently doing.

### *3.1 Tail for current activity*

```
 LOG=/var/log/vrlcm/vmsp_bootstrap_<env-uuid>.log
tail -15 $LOG
```

### *3.2 Phase markers to look for*

| Log line | Phase | Typical wall-time to this point |
|---|---|---|
| `>>> INIT0001 - Validating configuration` | Cluster-shared storage check (KB 421541) | ~30 sec after submit |
| `>>> INIT0003 - Deploying infrastructure components` | Image push to Fleet's local registry | ~5 min after submit |
| `>>> INIT0004 - Configure bootstrap machine` | Cluster-api / CAPI install in Fleet KIND | ~10 min after submit |

| Log line | Phase | Typical wall-time to this point |
|---|---|---|
| `>>> INFRA0002 - Waiting for VCF services platform cluster nodes to become ready` | Bootstrap VM created + booting + kubeadm init | ~20-40 min after submit |
| `cluster <name> provisioned` | CAPI declared the cluster object successful (still waiting for K8s API) | ~30-45 min |
| `Retrieving kubeconfig for <cluster>` | Fleet downloading kubeconfig from new cluster | ~30-45 min |
| `All synthetic checks status successful` | Bootstrap VM K8s control plane fully functional and healthy | ~50-75 min |
| `Cluster provisioning completed. Enjoy your new VCF services platform cluster!` | **INFRA0002 done — the highest-risk phase passed** | ~75-90 min |
| (log goes static after this; workflow continues in `vmware_vrlcm.log`) | App deploy onto bootstrap K8s | through final completion |

### *3.3 Grep for failure beacons*

```
grep -iE "ERR:|LCMVSPHERE|LCMVMSP|etcd failed|no route to host|timed out|exception|FAILED" $LOG | tail -10
```

| Pattern in output | Meaning |
|---|---|
| `ERR:INIT0001 - Validating configuration` | Cluster-shared storage check failed (KB 421541) — bootstrap VM target is host-local |
| `ERR:INIT0003 - Deploying infrastructure components` | Fleet KIND cluster creation failed (typically Fleet-side etcd timeout from CPU/storage starvation) |
| `ERR:DEPLOY0005 - Transferring control of infrastructure to VCF services platform cluster` | Bootstrap VM's K8s API became unreachable mid-handoff — storage latency typical |
| `LCMVSPHERECONFIG1000095` | Generic "Failed to create services platform cluster" — cause varies; cross-reference bootstrap log |
| `[-]etcd failed: reason withheld` (in apiserver healthz output) | Bootstrap VM's internal etcd died — nested vSAN write amplification |
| `dial tcp <vip>:6443: connect: no route to host` | Bootstrap VM's primary VIP unreachable from Fleet (often a downstream symptom of etcd death) |
| `error: timed out waiting for the condition on clusters/<name>` | Fleet polling timed out — usually a transient retry, not fatal alone |

### *3.4 Verbatim-success markers (the ones you WANT)*

These exact strings indicate forward progress past the prior failure points:

```
Cluster provisioning completed. Enjoy your new VCF services platform cluster!
All synthetic checks status successful on VCF services platform cluster <name>
```

If you see these two lines, you are past INFRA0002 — the highest-risk phase. Application deploy still needs to run, but the hard part is done.

---

## 4. Fleet KIND Control Plane Health

Fleet's KIND container runs the local CAPI/CAPV controllers that drive the bootstrap VM provisioning. Monitor this throughout the deploy.

### *4.1 The query*

```
docker exec kind-bootstrap-control-plane kubectl get pod -n kube-system \
  -l 'tier=control-plane' \
  -o custom-
columns=NAME:.metadata.name,READY:.status.containerStatuses[0].ready,RESTARTS:.status.containerStatuses[0]
.restartCount
```

### *4.2 Pass criteria*

```
NAME                                                 READY   RESTARTS
etcd-kind-bootstrap-control-plane                    true    0
kube-apiserver-kind-bootstrap-control-plane          true    0
kube-controller-manager-kind-bootstrap-control-plane   true    0-3
kube-scheduler-kind-bootstrap-control-plane          true    0-3
```

- READY = `true` on all four
- RESTARTS = 0-3 acceptable; transient spikes during heavy helm install can cause 1-3 restarts that recover

### *4.3 Fail criteria — escalate*

If READY = `false` on controller-manager or scheduler for >10 min, OR restart counts climbing every 2 min, Fleet's KIND is unstable. Required prerequisites to avoid this:

- Fleet on local NVMe-backed VMFS (not nested vSAN) — see `Fleet-Sizing-Increase-Runbook-2026-05-18.md`
- Fleet sized at 6 vCPU minimum

If both prerequisites are met and KIND is still unstable, capture state and abort the deploy rather than let it limp.

---

## 5. Fleet Load Average and Thresholds

```
uptime
nproc
```

### *5.1 Reading the output*

`uptime` returns:

```
HH:MM:SS up <uptime>, <users> user, load average: <1m>, <5m>, <15m>
```

The three load averages are counts of processes (RUNNING + RUNNABLE + UNINTERRUPTIBLE I/O wait) at each window.

### *5.2 Saturation thresholds for Fleet (assuming 6 vCPU after our resize)*

| 1-min load | Ratio to nproc | State |
|---|---|---|
| 0-4.2 | < 0.7 | Healthy, headroom |
| 4.2-6.0 | 0.7-1.0 | Approaching capacity |
| 6.0-12.0 | 1.0-2.0 | Saturated but functional — deploys live here |
| 12.0-18.0 | 2.0-3.0 | Heavy saturation; **K8s leader-election timeouts likely** |
| >18.0 | >3.0 | Critical — pods will restart; failure imminent |

### *5.3 Expected load profile during a healthy deploy*

| Phase | 1-min load (6 vCPU Fleet) |
|---|---|
| Pre-deploy idle | 0.1-0.5 |
| Image push (low intensity) | 2-3 |
| Heavy helm install batch | 8-12 |
| Brief spike during cluster validation | up to 20 (recovers within 10 min) |
| Post-cluster-provisioned wait phase | 2-3 |

**Brief spikes are tolerable if KIND pods recover.** Sustained load above 3x nproc for >15 min indicates trouble — cross-reference §4 KIND health.

---

## 6. VCFA Application Endpoint Probes

Once the bootstrap VM is up and the K8s ingress is bound to the primary VIP, you can probe the VCFA application HTTP endpoint to see when the app starts answering routes.

### *6.1 The probes*

```
 ## From Fleet (or anywhere with DNS resolution for automation-node1)
for p in / /provider /tenant /login /ui; do
  curl -sk --connect-timeout 5 -o /dev/null \
    -w "$p -> HTTP %{http_code}\n" \
    https://automation-node1.lab.local$p
done

## Also probe the raw VIP
curl -sk --connect-timeout 5 -o /dev/null \
  -w "HTTPS /  -> %{http_code} | content-type: %{content_type}\n" \
  https://<primary-vip>/
```

### *6.2 Interpreting HTTP codes*

| HTTP Code | Meaning |
|---|---|
| `000` / connection refused / timeout | The VIP is not held, OR the web service is not yet listening — either bootstrap K8s isn't up, or ingress controller isn't deployed yet |

| HTTP Code | Meaning |
|---|---|
| `404` on all paths | **Ingress alive, no app routes bound yet.** Normal during early app deploy phase. Web server is up but the actual VCFA application hasn't bound its routes. |
| `502` / `503` | Ingress alive, backend pod unhealthy or not ready. Could be temporary during app deploy. |
| `200` on `/provider` or `/login` | **VCFA application is up.** This is the success signal. |
| `301` / `302` redirect to `/provider` | Final routing in place — deploy is essentially done. |

### *6.3 Expected endpoint progression*

- T+0 (submit): connection refused (VIP not held)
- T+45m (bootstrap VM up, K8s alive): `404` on `/`, `/provider`, `/tenant`, `/login`
- T+90m+ (app deploy in progress): still `404`
- T+150m+ (completion): `200` / 301-redirect on `/provider`

If endpoints stay at `404` for >3 hours after bootstrap VM is up, inspect Fleet's `vmware_vrlcm.log` for ERROR lines tied to the env.

---

## 7. NFS Share Usage and Write Activity

If the deploy target is the Linux NFS datastore (`nfs-vcfa-shared` backed by Ubuntu Server NFS), monitor the share to detect storage-side hangs.

### *7.1 The query (SSH to NFS server)*

```
 ## SSH to NFS server (e.g., 192.168.1.161 as vcfadmin)
df -h /srv/nfs/vcfa
ls -la /srv/nfs/vcfa/vcfa-mgmt-*/ 2>&1 | head -25
```

### *7.2 What "good" looks like*

| Phase | NFS used (df) | Notes |
|---|---|---|
| Pre-clone | < 1 GB | Just the empty share + lost+found |
| Clone in progress | Growing 5-50 MB/s | flat.vmdk file growing steadily |
| Bootstrap VM running, K8s coming up | 20-40 GB used | Active VM writes to virtual disk |
| App deploy in progress | 40-100 GB used | Pod images + persistent volumes |
| Deploy completed | 60-150 GB used | Final stable footprint |

### *7.3 Fail criteria*

- **File size stuck at a round number** (e.g., exactly 100 GB) for >10 min while writes continue elsewhere — the NFS server is silently dropping writes beyond a per-file limit. Use Linux `nfs-kernel-server` only (Windows-based NFS servers have hit this issue in lab testing).
- **NFS server unreachable** from any ESXi host (`vmkping` fails to NFS server IP) — check NFS server health, firewall, network bridge.

### *7.4 Common file pattern (Linux NFS, EagerZeroedThick or thin)*

```
 vcfa-mgmt-<hash>-flat.vmdk # the actual disk data (size = virtual disk size)
vcfa-mgmt-<hash>.vmdk # descriptor
vcfa-mgmt-<hash>-<hash2>.vswp # memory swap file (size = VM RAM allocation)
vcfa-mgmt-<hash>.nvram # NVRAM
vcfa-mgmt-<hash>.vmx # VM config
vmware.log # VM-side ESXi log
.lck-<hash> # vSphere lock files (transient)
```

---

## 8. vCenter-side Checks (Bootstrap VM and Tasks)

```
 ## PowerCLI on workstation (with VIServer connected)

## Find bootstrap VM (the CAPI-created VM lives in Discovered virtual machine folder)
$f = Get-Folder -Name 'Discovered virtual machine'
$boot = Get-VM -Location $f | Where-Object {$_.Name -like 'vcfa-mgmt-*'}
$boot | Select-Object Name,PowerState,NumCpu,MemoryGB,
                      @{N='Host';E={$_.VMHost.Name}},
                      @{N='Datastore';E={(Get-Datastore -RelatedObject $_)[0].Name}}
```

### *8.1 What "good" looks like (during deploy)*

| Stage | Bootstrap VM state |
|---|---|
| Pre-bootstrap VM creation | (not found) |
| CAPI clone running | Name visible, PowerState=PoweredOff |
| Clone done, CAPI reconfiguring | NumCpu transitions 4 → 24, MemoryGB transitions 8 → 96 |
| VM powered on | PoweredOn, NumCpu=24, MemoryGB=96 |

### *8.2 Active clone task*

```
Get-Task | Where-Object {$_.State -eq 'Running' -and $_.Name -match 'Clone'} |
  Select-Object Name,
    @{N='Pct';E={$_.PercentComplete}},
    @{N='Elapsed';E={[int]((Get-Date) - $_.StartTime).TotalMinutes}}
```

**Caveat:** `CloneVM_Task` is **not cancellable** via vCenter API once it has entered the data-copy phase. If you need to abort a hung clone, you must break it from below (stop the NFS server, then the clone errors out).

### *8.3 Cluster-shared storage validation*

```
 $ds = Get-Datastore 'nfs-vcfa-shared'
$hostsConnected = $ds.ExtensionData.Host | ForEach-Object { (Get-View $_.Key).Name }
Write-Output ("Datastore visible on " + $hostsConnected.Count + " hosts: " + ($hostsConnected -join ','))
```

Must be visible on ALL hosts in the target cluster, or KB 421541 `ERR:INIT0001` will fail the deploy.

---

## 9. What "Good" Looks Like Per Phase

A working VCFA Simple deploy progresses through these phases. Use this as the timeline reference.

| Phase | Time after submit | Marker |
|---|---|---|
| Submit accepted | 0 | HTTP 200 from POST, `requestId` returned |
| Env created in DB | +30 sec | `vm_lcops_environments` has the env row |
| INIT0001 validation | +1-2 min | `>>> INIT0001 - Validating configuration` in bootstrap log |
| Image push to local registry | +5-30 min | Multiple `Installing package <name>.tzst` lines |
| INIT0004 bootstrap machine config | +30-40 min | `>>> INIT0004 - Configure bootstrap machine` |
| CAPI cluster object created | +35-45 min | `kubernetescluster.controlplane.vmsp.vmware.com/vcf-mgmt-<id> condition met` |
| Bootstrap VM created in vCenter | +40-50 min | Bootstrap VM visible, PoweredOff, in Discovered virtual machine folder |
| Bootstrap VM PoweredOn at 24/96 | +45-55 min | Bootstrap VM PoweredOn with full VCFA sizing |
| Bootstrap K8s API available | +55-75 min | "Retrieving kubeconfig" succeeds; synthetic check loop starts |
| All synthetic checks pass | +70-90 min | `All synthetic checks status successful` |
| **Cluster provisioning complete** | +75-90 min | `Cluster provisioning completed. Enjoy your new VCF services platform cluster!` ← **HIGHEST-RISK PHASE PASSED** |
| App deploy (helm install on bootstrap K8s) | +90-150 min | Bootstrap log goes static, activity moves to `vmware_vrlcm.log` |
| VCFA app routes bound | +150-180 min | `/provider` returns HTTP 200 instead of 404 |
| Env transitions to COMPLETED | +180-240 min | `environmentstatus = COMPLETED` |

**Total expected wall time:** 3-4 hours for VCFA Simple on a properly-sized lab.

---

## 10. The Consolidated One-Shot Status Script

Copy-paste this onto Fleet (SSH as root) for a full status snapshot. Replace `<env-uuid>` with the env in question:

```
 ENV="<env-uuid>"

echo "=== 1. Env status ==="
sudo -u postgres /opt/vmware/vpostgres/14/bin/psql -d vrlcm -At -c \
  "SELECT vmid, environmentstatus, to_timestamp(lastupdatedon/1000) \
   FROM vm_lcops_environments WHERE vmid='$ENV';"

echo ""
echo "=== 2. Bootstrap log last 15 lines ==="
tail -15 /var/log/vrlcm/vmsp_bootstrap_${ENV}.log 2>/dev/null

echo ""
echo "=== 3. Failure beacons ==="
grep -iE "ERR:|LCMVSPHERE|etcd failed|no route|timed out|exception|FAILED" \
  /var/log/vrlcm/vmsp_bootstrap_${ENV}.log 2>/dev/null | tail -5
```

```
echo ""
echo "=== 4. Fleet KIND control plane health ==="
docker exec kind-bootstrap-control-plane kubectl get pod -n kube-system \
  -l 'tier=control-plane' \
  -o custom-
columns=NAME:.metadata.name,READY:.status.containerStatuses[0].ready,RESTARTS:.status.containerStatuses[0]
.restartCount 2>&1 | head -10

echo ""
echo "=== 5. Fleet load average (compare to 6 nproc) ==="
uptime
nproc

echo ""
echo "=== 6. VCFA endpoint probes ==="
for p in / /provider /tenant /login /ui; do
  curl -sk --connect-timeout 5 -o /dev/null \
    -w "$p -> HTTP %{http_code}\n" https://automation-node1.lab.local$p
done

echo ""
echo "=== 7. docker containers ==="
docker ps --format "table {{.Names}}\t{{.Status}}"
```

For the workstation-side checks (bootstrap VM state, clone task progress, NFS share usage), run them separately as PowerCLI / NFS-SSH calls per §7 and §8.

---

### 11. Document Information

| Field | Value |
|---|---|
| Title | VCFA Deploy Monitoring & Health Check Runbook |
| Document ID | VC-RUNBOOK-VCFA-MONITORING-20260520 |
| Version | 1.0 |
| Issued | May 20, 2026 |
| Author | Virtual Control LLC |
| Classification | Internal — Lab Environment |
| Related Documents | `VCFA-Simple-Deploy-Wizard-Precheck-Gotchas-2026-05-18.md`, `Failed-VCFA-Deployment-Cleanup-Runbook-2026-05-18.md` |
| Distribution | Lab notebook |

---

## 8. Root-Cause Analysis & Troubleshooting Report

*Source: VCF-Automation-Deployment-Troubleshooting-Report.md*

**Prepared by:** Virtual Control LLC **Date:** April 2026 **Document Version:** 2.1 **Classification:** Internal -- Lab Environment **VCF Version:** 9.0.1

**Table of Contents**

1. Executive Summary
2. Environment Reference
   2.1 Physical Infrastructure
   2.2 VCF Component Inventory
   2.3 Key Terms and Acronyms
3. Issue 1 -- Memory Calculation Bug (April 14)
   3.1 What Is This?
   3.2 What Happened
   3.3 Why Did This Happen?
   3.4 The Fix
   3.5 How to Prevent This
4. Issue 2 -- NSX HTTP Service Stopped After Reboot (April 14-15)
   4.1 What Is This?
   4.2 What Happened
   4.3 Why Did This Happen?
   4.4 The Fix
   4.5 How to Prevent This
5. Issue 3 -- LCM OVA Wrong IP (April 15)
   5.1 What Is This?
   5.2 What Happened
   5.3 Why Did This Happen?
   5.4 The Fix
   5.5 How to Prevent This
6. Issue 4 -- LCM Registration Wrong Node (April 15)
   6.1 What Is This?
   6.2 What Happened
   6.3 Why Did This Happen?
   6.4 The Fix
   6.5 How to Prevent This
7. Issue 5 -- Offline Depot Configuration (April 15)
   7.1 What Is This?
   7.2 What Happened
   7.3 Why Did This Happen?
   7.4 The Fix
   7.5 How to Prevent This
8. Issue 6 -- Identity Broker: Embedded vs. Standalone (April 15)
   8.1 What Is This?
   8.2 What Happened
   8.3 Why Did This Happen?
   8.4 The Fix
   8.5 How to Prevent This
9. Issue 7 -- Parallel Deployment Block (April 15-16)
   9.1 What Is This?
   9.2 What Happened
   9.3 Why Did This Happen?
   9.4 The Fix: Complete Database Cleanup
   9.5 How to Prevent This
10. Issue 8 -- Certificate SAN Mismatch (April 16)
   10.1 What Is This?

10.2 What Happened
10.3 Why Did This Happen?
10.4 The Fix
10.5 How to Prevent This
11. Issue 9 -- API Deployment Bypass (April 16)
11.1 What Is This?
11.2 What Happened
11.3 Why Did This Happen?
11.4 The Fix: API Deployment
11.5 The Correct JSON Payload
11.6 How to Prevent This
12. Complete LCM Database Cleanup Reference
12.1 Table Dependency Tree
12.2 Complete Cleanup Script
13. Lessons Learned
14. Current Status
15. Document Information

---

## 1. Executive Summary

> **What is this section?** An executive summary gives you the "big picture" of the entire document in just a few paragraphs. Think of it as a movie trailer for the full report -- it tells you what happened, why it matters, and how it turned out.

Between April 14 and April 17, 2026, **eleven** interconnected issues were discovered while deploying the VMware Aria Suite on a VCF (VMware Cloud Foundation) 9.0.1 lab environment. Issues 1–9 were resolved; issues 10 and 11 are **in active remediation** as of April 17. VCF is a bundle of VMware software that turns physical servers into an automated, software-defined data center. The Aria Suite is a collection of management tools that sit on top of VCF:

1. **Aria Suite LCM (Lifecycle Manager)** -- The "installer for the installers." It deploys, upgrades, and manages all other Aria products.
2. **Identity Broker (VIDB)** -- A single sign-on (SSO) service that lets users log into multiple tools with one set of credentials, similar to "Sign in with Google."
3. **VCF Automation** -- A self-service portal where users can request virtual machines, networks, and other IT resources automatically -- like an online ordering system for infrastructure.

The lab runs on a single Dell Precision 7920 workstation with 384 GB of RAM. Four "nested" ESXi hosts (virtual computers running inside other virtual computers) simulate a production data center.

Over three days, the following blocking issues were identified and resolved:

| # | Issue | Severity | Time to Resolve |
|---|---|---|---|
| 1 | Memory calculation bug in health check app | Medium | 30 minutes |
| 2 | NSX Manager HTTP service stopped after reboot | High | 20 minutes |
| 3 | LCM OVA deployed with wrong IP address | Medium | 15 minutes |

| # | Issue | Severity | Time to Resolve |
|---|---|---|---|
| 4 | LCM registration used wrong node address | Medium | 30 minutes |
| 5 | Offline depot URL and file structure errors | High | 2 hours |
| 6 | Unnecessary standalone Identity Broker deployment | Medium | 30 minutes |
| 7 | Parallel deployment block from orphaned database records | **Critical** | 6+ hours |
| 8 | Certificate SAN mismatch blocking precheck | High | 1 hour |
| 9 | UI submit always fails; API deployment bypass required | **Critical** | 3 hours |
| 10 | VMSP etcd CrashLoopBackOff on HDD-backed nested vSAN | **Critical** | In progress (SSD upgrade) |
| 11 | esxi01 host disk (D:) saturated — host NotResponding | High | In progress (move to F:) |

**The Most Critical Issue (Issue 7):** The VCF Operations UI silently creates database records in the LCM PostgreSQL database every time you open an Aria product page. If a deployment fails or is abandoned, these "ghost" records are never cleaned up. They block ALL future deployment attempts with a confusing "parallel deployment" error. The fix requires directly editing the LCM database -- something VMware never documents in their standard troubleshooting guides. This is a known bug (Broadcom KB 388305). Even after cleaning the database, the UI recreates the entries immediately, which is why Issue 9 (API bypass) was ultimately required.

**New Critical Issue (Issue 10):** On April 17, Fleet Management re-triggered a VCF Automation deployment that completed Stages 1–2 cleanly but failed at Stage 3 ("Deploy appliance cluster") after 1 hour 57 minutes. Error: `LCMVSPHERECONFIG1000095 / ERR:DEPLOY0003`. The root cause is architectural: VMSP's etcd (500 ms heartbeat, 10 s election timeout) cannot sustain fsync latency on HDD-backed nested vSAN during the bootstrap write burst. kubeadm-init alone took 1h 17m (normal is 20–30 min) — the signature of etcd coming up on slow storage and falling over under real load. No VMware-provided config tunable can compensate. Fix requires SSD-backed storage for the VMSP VM. A Crucial T500 2 TB NVMe has been ordered; interim plan is to move esxi04 off HDD and relocate the VMSP VMDKs to an existing SSD.

**Outcome:** Issues 1–9 are resolved. Issues 10 and 11 are being remediated simultaneously on April 17. VCF Automation deployment is paused until esxi01 disk capacity is restored AND VMSP storage is relocated to SSD.

---

## 2. Environment Reference

### *2.1 Physical Infrastructure*

**What is this section?** Before troubleshooting anything, you need to know what hardware and software you are working with. This table describes the physical computer and the key configuration details of the lab.

| Parameter | Value | Explanation |
|---|---|---|
| Physical Host | Dell Precision 7920 | The actual, physical computer sitting on a desk |
| RAM | 384 GB | The computer's working memory (upgraded from 192 GB during this timeline) |
| Deployment Type | Nested ESXi | Virtual machines running inside virtual machines -- like a dream within a dream |
| VCF Version | 9.0.1 | The version of VMware Cloud Foundation being tested |
| Lab Domain | lab.local | The DNS domain name used for all lab components |
| DNS/NTP Server | 192.168.1.230 | The server that translates names to IP addresses (DNS) and keeps time synchronized (NTP) |

### *2.2 VCF Component Inventory*

**What is this section?** VCF is made up of many individual software appliances (pre-built virtual machines), each with its own job. This table lists every component in the lab, its hostname (the name computers use to find it), and its IP address (the numerical address on the network).

**What is an FQDN?** FQDN stands for Fully Qualified Domain Name. It is the complete hostname including the domain -- for example, `vcenter.lab.local` instead of just `vcenter`. Computers need the full FQDN to look up the correct IP address through DNS (Domain Name System).

| Component | FQDN | IP Address | Role |
|---|---|---|---|
| vCenter Server | vcenter.lab.local | 192.168.1.69 | Central management for all virtual machines and hosts |
| SDDC Manager | sddc-manager.lab.local | 192.168.1.241 | Orchestrates the entire VCF stack (Software-Defined Data Center) |
| NSX VIP | nsx-vip.lab.local | 192.168.1.70 | Virtual IP that floats to whichever NSX Manager is active |
| NSX Node 1 | nsx-node1.lab.local | 192.168.1.71 | The actual NSX Manager appliance (handles networking and security) |
| VCF Operations | vcf-ops.lab.local | 192.168.1.77 | Monitoring and alerting for the entire VCF environment |
| Fleet Manager | fleet.lab.local | 192.168.1.78 | Manages connections between VCF Operations and other Aria products |
| VCF Ops for Logs | logs.lab.local | 192.168.1.242 | Centralized log collection and analysis |
| VCF Automation (cluster VIP) | automation-node1.lab.local | 192.168.1.91 | The main entry point for VCF Automation |
| Automation Node 0 | vcfa-mgmt-0.lab.local | 192.168.1.92 | First Kubernetes worker node for Automation |
| Automation Node 1 | vcfa-mgmt-1.lab.local | 192.168.1.93 | Second Kubernetes worker node for Automation |
| Automation Node 2 | vcfa-mgmt-2.lab.local | 192.168.1.94 | Third Kubernetes worker node for Automation |

| Component | FQDN | IP Address | Role |
|---|---|---|---|
| Aria Suite LCM | lcm.lab.local | 192.168.1.95 | Lifecycle Manager -- installs and updates all Aria products |
| ESXi Host 1 | esxi01.lab.local | 192.168.1.74 | Virtual hypervisor (nested) |
| ESXi Host 2 | esxi02.lab.local | 192.168.1.75 | Virtual hypervisor (nested) |
| ESXi Host 3 | esxi03.lab.local | 192.168.1.76 | Virtual hypervisor (nested) |
| ESXi Host 4 | esxi04.lab.local | 192.168.1.82 | Virtual hypervisor (nested) |

### *2.3 Key Terms and Acronyms*

> **What is this section?** Technical documentation is full of abbreviations and jargon. This glossary defines every term used in this report so you never have to guess what something means.

| Term | Full Name | Plain English Explanation |
|---|---|---|
| API | Application Programming Interface | A way for software to talk to other software. Like a waiter taking your order to the kitchen. |
| CLI | Command Line Interface | A text-only way to control software by typing commands (like MS-DOS or Terminal). |
| DNS | Domain Name System | The internet's phone book -- translates names like `vcenter.lab.local` to IP addresses like `192.168.1.69`. |
| ESXi | ESX integrated | VMware's hypervisor -- software that runs virtual machines on bare metal hardware. |
| FQDN | Fully Qualified Domain Name | A complete hostname including domain, like `vcenter.lab.local`. |
| K8s | Kubernetes | An open-source system for running containerized applications. VCF Automation runs on Kubernetes. |
| LCM | Lifecycle Manager | The Aria Suite component that installs, upgrades, and patches all other Aria products. |
| NSX | (brand name -- no expansion) | VMware's network virtualization and security platform. |
| NTP | Network Time Protocol | Keeps all computers' clocks synchronized. Critical because certificates fail if clocks are wrong. |
| OVA | Open Virtual Appliance | A pre-packaged virtual machine file, like a ZIP file containing everything needed to run an application. |
| OVF | Open Virtualization Format | The standard format for OVA files. OVF properties are settings (like IP address) applied during deployment. |
| PostgreSQL | (pronounced "post-gres-Q-L") | An open-source relational database. LCM uses it to track deployment state. |

| Term | Full Name | Plain English Explanation |
|---|---|---|
| RCA | Root Cause Analysis | The process of finding the underlying reason why a problem occurred, not just the symptoms. |
| SAN | Subject Alternative Name | A field in a TLS/SSL certificate that lists all the hostnames and IP addresses the certificate is valid for. |
| SDDC | Software-Defined Data Center | A data center where all infrastructure (compute, storage, networking) is virtualized and automated. |
| SSH | Secure Shell | A protocol for securely connecting to a remote computer's command line over the network. |
| SSO | Single Sign-On | A system that lets you log into multiple applications with one username and password. |
| TLS | Transport Layer Security | Encryption protocol that secures network communication (the "S" in HTTPS). |
| VCF | VMware Cloud Foundation | VMware's integrated platform that bundles vSphere, NSX, vSAN, and management tools together. |
| VIDB | VMware Identity Broker | The SSO component for VCF Aria Suite products. |
| VIP | Virtual IP Address | An IP address shared between multiple servers. Traffic goes to whichever server is currently active. |
| VM | Virtual Machine | A software-based computer running inside a physical computer. |
| VRA | VMware vRealize Automation | The legacy name for VCF Automation (VMware renamed many products in 2023-2024). |
| vSAN | Virtual Storage Area Network | VMware's software-defined storage that pools local disks from multiple hosts into shared storage. |

---

## 3. Issue 1 -- Memory Calculation Bug (April 14)

### *3.1 What Is This?*

> **What is this issue about?** After upgrading the physical server from 192 GB to 384 GB of RAM, the VCF health check application (a custom monitoring tool we built) started showing 99% memory usage on all ESXi hosts. That is obviously wrong -- we just doubled the memory, so usage should have gone down, not up. The problem turned out to be a bug in our code where we were mixing up different measurement units.

### *3.2 What Happened*

After the physical RAM upgrade from 192 GB to 384 GB, the health check application displayed alarming memory usage figures:

```
ESXi Host Memory Usage:
  esxi01.lab.local:  99.2% (CRITICAL)
  esxi02.lab.local:  99.1% (CRITICAL)
  esxi03.lab.local:  98.7% (CRITICAL)
  esxi04.lab.local:  99.4% (CRITICAL)
```

This made no sense. The lab was running the same workloads on twice the memory. The numbers should have shown approximately 40-60% usage.

### *3.3 Why Did This Happen?*

**Root Cause:** The VMware vSphere API (the programming interface that lets tools query vCenter for information) returns two memory-related values that use **different units**:

- `summary.totalMemory` -- returned in **bytes** (the smallest unit of computer memory, like counting in pennies)
- `summary.effectiveMemory` -- returned in **megabytes** (1 megabyte = 1,048,576 bytes, like counting in ten-thousand-dollar bills)

The health check code was subtracting megabytes from bytes without converting either one. This is like subtracting "$50" (dollars) from "500,000" (pennies) and wondering why the result seems wrong.

**Example of the math going wrong:**

```
 Total Memory (bytes):      412,316,860,416  (384 GB in bytes)
Effective Memory (MB):              393,216  (375 GB in megabytes)

WRONG calculation (what the code did):
  Used = totalMemory - effectiveMemory (no conversion)
  Used = 412,316,860,416 - 393,216 = 412,316,467,200
  Percent = Used / Total * 100 = 99.9999%
```

The subtraction barely changed the total because megabytes are a million times smaller than bytes. The "used" value was essentially the same as the total, giving 99.99%.

VMware designed different API properties at different times and never standardized the units. The documentation does specify the units, but it is easy to miss when the property names look similar.

### *3.4 The Fix*

**Fix:** Multiply `effectiveMemory` by 1,048,576 (which is 1024 x 1024, or the number of bytes in a megabyte) to convert it to bytes before doing any math.

**Code change** in `vcf-health-check.sh` , line 2315:

**Before (broken):**

```
em = int(props.get('summary.effectiveMemory', 0) or 0)
```

**After (fixed):**

```
em = int(props.get('summary.effectiveMemory', 0) or 0) * 1048576
```

**Result after fix:**

```
ESXi Host Memory Usage:
  esxi01.lab.local:  41.3% (OK)
  esxi02.lab.local:  47.2% (OK)
  esxi03.lab.local:  53.1% (OK)
  esxi04.lab.local:  59.0% (OK)
```

### *3.5 How to Prevent This*

- **Always check the VMware API documentation** for the units of every property you use. Do not assume two similarly-named properties use the same units.
- **Add unit tests** that verify memory calculations produce reasonable percentages (between 0% and 100%).
- **Log the raw values** during development so you can see when numbers are wildly different scales.

---

## 4. Issue 2 -- NSX HTTP Service Stopped After Reboot (April 14-15)

### *4.1 What Is This?*

**What is this issue about?** NSX Manager is the component that handles all virtual networking and security in a VCF environment. It runs as a virtual machine (called an "appliance") with a web interface and an API. After rebooting the lab, the NSX appliance was running and responding to ping, but its web interface and API were completely unresponsive. The problem was that a critical internal service called `http` did not automatically start after the reboot.

**What is the difference between "ping works" and "API works"?** The `ping` command tests whether a computer is powered on and connected to the network. It is like calling someone's phone and hearing it ring. But that does not mean they will answer. The API requires specific software services to be running inside the VM -- if those services are stopped, the VM is "alive" but not doing its job.

### *4.2 What Happened*

After rebooting the lab, NSX Manager at `nsx-node1.lab.local` (192.168.1.71) was unresponsive:

- **Ping:** Responded (the VM was running)
- **Web UI (https://192.168.1.71):** Connection refused
- **API calls:** Timed out
- **Health check status:** "NSX not accessible"

SSH into the NSX appliance and running `get services` revealed the problem immediately:

```
http                       stopped
```

### *4.3 Why Did This Happen?*

**Root Cause:** After a reboot, the NSX Manager appliance booted at the operating system level (Linux started successfully), but the **`http` service** -- which is the reverse proxy that handles all web traffic on port 443 -- did not auto-start.

**What is a reverse proxy?** A reverse proxy is a service that sits in front of other services and forwards incoming web requests to the right place. Think of it like a receptionist at a front desk -- all visitors must go through the receptionist to reach anyone in the building. If the receptionist is not at the desk (the http service is stopped), nobody can get through, even though all the people in the building (the other NSX services) are there.

**Why did it not start?** In resource-constrained environments (like a nested lab where many VMs share the same physical hardware), NSX Manager's services have strict timeout windows during boot. If the system is slow to start, the http service may fail to initialize within its timeout window and simply stops trying.

#### *4.4 The Fix*

**Fix:** Manually start the http service from the NSX CLI (Command Line Interface):

```
nsx-node1> start service http
```

Wait 30 seconds, then verify:

```
nsx-node1> get services
```

Confirm `http` now shows `running`. Once http is running, the web UI and API become available within a few minutes.

**Verification:** Open a web browser and navigate to `https://192.168.1.71`. The NSX Manager login page should appear.

#### *4.5 How to Prevent This*

- **After every lab reboot,** SSH into NSX Manager and run `get services` to verify the `http` service is running.
- **Do not rely on ping** to confirm NSX Manager is healthy. Ping only proves the VM is on the network, not that the application is working.
- **Wait at least 10 minutes** after powering on the NSX Manager VM before assuming it has failed. In nested environments, services can take a long time to start.

---

### 5. Issue 3 -- LCM OVA Wrong IP (April 15)

#### *5.1 What Is This?*

**What is this issue about?** Aria Suite Lifecycle Manager (LCM) is deployed by importing an OVA file (a pre-built virtual machine package) into vCenter. During deployment, you specify the IP address, hostname, and other network settings as "OVF properties." In this case, the intended IP address was 192.168.1.95, but the appliance came up with IP 192.168.1.92 instead.

#### *5.2 What Happened*

After deploying the LCM OVA through vCenter, the appliance was unreachable at its planned IP address:

```
ping lcm.lab.local # Request timed out
ping 192.168.1.95 # Request timed out
```

The VM was powered on in vCenter and showed a healthy heartbeat, but no network connectivity to the expected IP.

### *5.3 Why Did This Happen?*

**Root Cause:** The OVA was deployed with IP address **192.168.1.92** instead of the intended **192.168.1.95**. The OVF properties (the settings you fill in during the deployment wizard) either were not applied correctly or were entered incorrectly during deployment.

**How was this discovered?** Since the VM was running but unreachable at .95, the VM console in vCenter was opened (right-click VM > Open Console) and `ip addr show` was run, revealing `inet 192.168.1.92/24` -- the wrong IP.

### *5.4 The Fix*

**Fix:** Manually correct the IP address inside the Linux-based LCM appliance.

**Step 1:** Open the VM console in vCenter and log in.

**Step 2:** Edit the network configuration file:

```
vi /etc/systemd/network/10-eth0.network
```

**Step 3:** Find the `Address` line and change it:

```
## Before (wrong):
Address=192.168.1.92/24

## After (correct):
Address=192.168.1.95/24
```

**What does `/24` mean?** The `/24` is the subnet mask in CIDR notation. It means the first 24 bits of the IP address define the network (192.168.1.x), and the remaining 8 bits define the specific host. In plain terms, this computer is on the 192.168.1.0 network and can talk directly to any address in the 192.168.1.1-254 range.

**Step 4:** Restart the networking service:

```
systemctl restart systemd-networkd
```

**Step 5:** Verify from the appliance:

```
ip addr show eth0
## Should now show: inet 192.168.1.95/24
```

**Step 6:** Verify from your workstation:

```
 ping 192.168.1.95
## Should now respond
```

### *5.5 How to Prevent This*

- **Always verify the IP address from the VM console** immediately after deploying any OVA. Do not assume the OVF properties were applied correctly.
- **Check the VM console network info** in vCenter (Summary tab > IP Addresses) to see what IP the guest OS actually has.
- **Pre-create DNS records** and test them before deploying the OVA, so you can immediately detect mismatches.

---

## 6. Issue 4 -- LCM Registration Wrong Node (April 15)

### *6.1 What Is This?*

**What is this issue about?** After deploying LCM, it needs to be "registered" with VCF Operations so the two systems can communicate. Registration is the process of telling one system about another so they can trust each other and share data -- like adding a new contact to your phone. The problem was that the wrong hostname was entered during registration, and DNS caches had stale (outdated) entries.

### *6.2 What Happened*

When attempting to register LCM with VCF Operations through the Fleet Management interface, the following error appeared:

```
Error: Error connection Fleet Management node
```

### *6.3 Why Did This Happen?*

**Root Cause:** Two problems combined to cause this failure:

**Problem 1: Wrong hostname entered.** The administrator entered `fleet.lab.local` (192.168.1.78, the Fleet Manager appliance) instead of `lcm.lab.local` (192.168.1.95, the Aria Suite Lifecycle Manager). These are completely different appliances:

- **Fleet Manager** ( `fleet.lab.local` / .78): A component of VCF Operations that manages connections. It is already part of VCF Operations -- you do not register it.
- **Aria Suite LCM** ( `lcm.lab.local` / .95): The Lifecycle Manager that was just deployed. This is what needs to be registered.

It is like trying to register a new student at a school by giving the school's own address instead of the student's home address.

**Problem 2: DNS cache poisoning.** Even after correcting the hostname, the DNS cache on both the Windows workstation and the VCF Operations appliance still had the old IP (.92) cached for `lcm.lab.local` from before Issue 3 was fixed.

### *6.4 The Fix*

**Fix:** Four steps were required:

**Step 1:** Update the DNS A record for `lcm.lab.local` from the old IP (.92) to the correct IP (.95) on the DNS server at 192.168.1.230.

**Step 2:** Flush the DNS cache on the Windows workstation:

```
ipconfig /flushdns
```

**Step 3:** Flush the DNS cache on the VCF Operations appliance:

```
ssh root@192.168.1.77
systemd-resolve --flush-caches
```

**Step 4:** Re-attempt registration at:

```
https://vcf-ops.lab.local/admin/
> Fleet Management
> Connect
```

Enter `lcm.lab.local` as the node address and provide the admin password.

**Result:** LCM registered successfully. Fleet Management status shows "Connected."

### *6.5 How to Prevent This*

- **Document every component's FQDN and role** in a table (like Section 2.2) and refer to it every time you enter a hostname.
- **After changing any IP address,** update DNS records AND flush DNS caches on all systems that might have the old address cached.
- **Test DNS resolution** before attempting registration: `bash nslookup lcm.lab.local # Verify it returns 192.168.1.95, not .92`

---

## 7. Issue 5 -- Offline Depot Configuration (April 15)

### *7.1 What Is This?*

**What is this issue about?** An "offline depot" is a local server that stores all the software binaries (OVA files, update packages, tar archives) that VCF needs to install products. In production environments, VCF downloads these from Broadcom's internet servers. In a lab without reliable internet, you host them locally. The problem

was a combination of a trailing space in the URL, binaries in the wrong folder, and a missing depot folder structure.

### *7.2 What Happened*

When LCM attempted to download binaries from the offline depot, it failed with multiple errors:

```
Error: "Illegal character in scheme name at index 6"
Error: HTTP 404 Not Found
```

### *7.3 Why Did This Happen?*

**Root Cause: Three separate problems in the depot configuration.**

**Problem 1: Trailing space in the depot URL.** When the URL was entered in the VCF Operations UI, an invisible trailing space was included at the end. A URL must be an exact string with no extra characters.

**Problem 2: VIDB binary in the wrong directory.** LCM expects software binaries to be organized in a specific `PROD/COMP/` folder structure. The binary was in a different location than where LCM was looking.

**Problem 3: Missing depot metadata.** The depot needs `vcfManifest.json` and `productVersionCatalog.json` (plus `.sig` signature files) in metadata folders for LCM to know what binaries are available.

### *7.4 The Fix*

**Fix:** Instead of using a webserver-based depot (which requires careful URL configuration and an HTTPS server), switch to a **Local Path depot** that reads directly from the LCM appliance's disk.

**Step 1:** Create the `PROD/COMP/` directory structure on the LCM disk at `/data`:

```
/data/PROD/COMP/
  VRA/               (VCF Automation tar archive)
  VIDB/              (Identity Broker tar archive)
  VROPS/             (VCF Operations OVA + upgrade PAK)
  VRLI/              (VCF Operations for Logs OVA + PAK)
  VRSLCM/            (Lifecycle Manager OVA)
  VCENTER/           (vCenter Server OVA)
  NSX_T_MANAGER/     (NSX Manager OVA)
  NSX_EDGE/          (NSX Edge OVA)
```

**What are these folder names?** They use **legacy VMware product codes** from before Broadcom renamed everything:

| Folder Code | Current Name (VCF 9.0) |
|---|---|
| VRA | VCF Automation |
| VROPS | VCF Operations |
| VRLI | VCF Operations for Logs |

| Folder Code | Current Name (VCF 9.0) |
|---|---|
| VIDB | Identity Broker |
| VRSLCM | Aria Suite Lifecycle Manager |

**Step 2:** Create symlinks from the downloaded binaries to the expected paths:

```
ln -s /path/to/downloaded/binary.tar /data/PROD/COMP/VRA/
```

**Step 3:** Include the required metadata files: - `vcfManifest.json` in the depot root - `productVersionCatalog.json` and its `.sig` file in the metadata folders

**Step 4:** Configure the depot as "Local Path" pointing to `/data` in the VCF Operations admin UI.

**Reference:** Broadcom KB 432806

### *7.5 How to Prevent This*

- **Use Local Path depot instead of a webserver** for lab environments. It is simpler and eliminates URL-related errors.
- **Set up the complete `PROD/COMP/` folder structure** before deploying LCM.
- **Match binary versions exactly.** Check SDDC Manager for the current BOM (Bill of Materials) to see what versions are required.

---

## 8. Issue 6 -- Identity Broker: Embedded vs. Standalone (April 15)

### *8.1 What Is This?*

**What is this issue about?** Identity Broker (VIDB) is the "login service" for VCF Aria products. It provides SSO (Single Sign-On) so users can log in once and access all tools. There are two ways to deploy it: embedded (built into vCenter, no extra appliance needed) and standalone (a separate 3-node cluster for multi-site environments). We were trying to deploy the standalone version, which was unnecessary and causing the VIDB binary download to block everything.

### *8.2 What Happened*

Every attempt to deploy VCF Automation was blocked because LCM insisted on deploying Identity Broker first. The VIDB binary download kept failing, preventing any progress.

### *8.3 Why Did This Happen?*

**Root Cause:** A standalone Identity Broker appliance is **not required** for a single-instance lab environment. VCF 9.0 supports two modes:

**Mode 1: Embedded (correct for our lab)** - No additional appliance needed - SSO is handled by the vCenter SSO service that is already running - Suitable for single data center environments

**Mode 2: Standalone Appliance (overkill for a lab)** - Deploys a separate 3-node Identity Broker cluster - Required only for multi-site environments - Needs additional resources and the VIDB binary

The deployment wizard may default to requiring Identity Broker, making it easy to assume you need the standalone version without reading the documentation carefully.

### *8.4 The Fix*

**Fix:** Skip the standalone Identity Broker deployment entirely and use the embedded mode instead.

**How to configure embedded Identity Broker:** 1. Navigate to: `https://vcf-ops.lab.local/admin/` 2. Go to: **Fleet Management** > **Identity & Access** 3. Select the embedded identity provider (vCenter SSO) 4. Proceed with Automation deployment without the VIDB dependency

### *8.5 How to Prevent This*

- **Read the VCF 9.0 Planning and Preparation Guide** before deploying any Aria product. The "Identity and Access Management" section explains which mode you need.
- **For lab environments,** always choose the simplest deployment option. Not every component needs a separate appliance.
- **Ask yourself:** "Does this environment span multiple data centers?" If no, use embedded mode.

---

## 9. Issue 7 -- Parallel Deployment Block (April 15-16)

**This is the most complex and critical issue in this entire report.** It took over 6 hours to diagnose and resolve. It involves directly editing a PostgreSQL database, which VMware's standard documentation rarely covers. If you ever encounter the "parallel deployment" error, the database cleanup described here is the only reliable solution.

### *9.1 What Is This?*

**What is this issue about?** After resolving all previous issues, every attempt to submit a VCF Automation deployment through the UI failed immediately with an error about "parallel deployments." No deployment was actually running -- the system was stuck believing previous deployments were still in progress. The error was caused by "ghost" records in the LCM database that were created by the UI and never cleaned up.

**What is a "parallel deployment"?** LCM prevents two deployments from running at the same time because they could conflict. The error means LCM thinks another deployment is already running -- even though it is not.

### *9.2 What Happened*

Every time the "Submit" button was clicked to deploy VCF Automation, this error appeared immediately:

```
Parallel Deployments of automation and identity broker are not allowed.
If an automation or identity broker deployment request is in progress,
wait for it to be completed before submitting this request.
```

This error appeared even though no deployment was visibly running, no progress bar showed activity, and rebooting the LCM appliance did not help.

### *9.3 Why Did This Happen?*

**Root Cause: Multiple layers of "ghost" records in the LCM PostgreSQL database.**

**What is PostgreSQL?** PostgreSQL (often called "Postgres") is a database -- a system that stores structured data in tables, like a sophisticated spreadsheet. LCM uses PostgreSQL to track every deployment, request, and product status.

**What are "orphaned" or "ghost" records?** When a deployment is started, LCM creates database records to track it. Orphaned records are records that were created but never updated to a final state -- they are stuck "in progress" forever, like a library book that was checked out but never returned.

**The key table:** `vm_lcops_environments`

Every time someone opened the Automation or Networks product page in the VCF Operations UI, a new environment record was created with `status='UI_INPROGRESS'`. These records were never cleaned up -- not when you navigated away, not when you closed the browser, and not when you rebooted the appliance.

**Critical discovery:** The UI creates entries under BOTH the `vra` (Automation) AND `vmsp` (VMSP platform) product IDs. It also creates entries with `SUBMITTED` and `INPROGRESS` statuses, not just `UI_INPROGRESS`. Any cleanup must check for ALL product IDs (`vra`, `vmsp`, `vidb`, `vrni`) and ALL non-completed statuses.

The parallel deployment check works like this:

```
IF any row in vm_lcops_environments has status NOT IN ('COMPLETED', 'FAILED')
THEN block all new deployments with "parallel deployment" error
```

**Database table relationships (parent-child):**

```
vm_lcops_environments (parent -- delete LAST)
  +-- vm_lcops_infrastructure_properties (child)
  +-- vm_lcops_product (child)
        +-- vm_lcops_product_properties (grandchild)
        +-- vm_lcops_product_nodes (grandchild)
              +-- vm_lcops_node_properties (great-grandchild)
```

You must delete children before parents due to **foreign key constraints** -- database rules that say "you cannot delete a parent record if children still reference it."

**References:** - Broadcom KB 388305 - GitHub Holodeck Issue #91

### *9.4 The Fix: Complete Database Cleanup*

**Fix:** Directly clean the LCM PostgreSQL database by deleting all orphaned records, working from child tables up to parent tables, with BOTH the `vrlcm-server` service stopped AND the `vmware-vcops-web` service on VCF Operations restarted afterward to clear cached state.

**WARNING:** You are about to directly modify a production database. Before proceeding: 1. Take a VM snapshot of the LCM appliance so you can roll back 2. Double-check every SQL command before pressing Enter 3. Never delete records where `status='COMPLETED'` -- those are legitimate

**Step 1: SSH into the LCM appliance and stop the service.**

```
 ssh root@192.168.1.95
systemctl stop vrlcm-server
```

The LCM application must be stopped first. If it is running while you edit the database, it will recreate records you just deleted and potentially corrupt data.

**Step 2: Examine the current state.**

```
/opt/vmware/vpostgres/14/bin/psql -U vrlcm -d vrlcm -c \
  "SELECT environmentid, environmentstatus, status FROM vm_lcops_environments;"
```

**What this command does:** `/opt/vmware/vpostgres/14/bin/psql` is the PostgreSQL command-line client. `-U vrlcm` means connect as user `vrlcm`. `-d vrlcm` means connect to the database named `vrlcm`. `-c "..."` runs the SQL query inside the quotes.

You will see rows with `COMPLETED` status (legitimate, leave these alone) and rows with `UI_INPROGRESS`, `SUBMITTED`, `IN PROGRESS`, or `FAILED` status (orphans to clean up).

**Step 3: Run the complete cleanup.** See Section 12 for the full cleanup script. The short version:

```
 ## Delete deepest children first, then work up to parents
## Level 4: Node properties
/opt/vmware/vpostgres/14/bin/psql -U vrlcm -d vrlcm -c \
  "DELETE FROM vm_lcops_node_properties WHERE node_vmid IN (
    SELECT vmid FROM vm_lcops_product_nodes WHERE product_vmid IN (
      SELECT vmid FROM vm_lcops_product WHERE environment_vmid IN (
        SELECT vmid FROM vm_lcops_environments WHERE status != 'COMPLETED')));"

## Level 3: Product nodes
/opt/vmware/vpostgres/14/bin/psql -U vrlcm -d vrlcm -c \
  "DELETE FROM vm_lcops_product_nodes WHERE product_vmid IN (
    SELECT vmid FROM vm_lcops_product WHERE environment_vmid IN (
      SELECT vmid FROM vm_lcops_environments WHERE status != 'COMPLETED'));"

## Level 3: Product properties
/opt/vmware/vpostgres/14/bin/psql -U vrlcm -d vrlcm -c \
  "DELETE FROM vm_lcops_product_properties WHERE product_vmid IN (
    SELECT vmid FROM vm_lcops_product WHERE environment_vmid IN (
      SELECT vmid FROM vm_lcops_environments WHERE status != 'COMPLETED'));"

## Level 2: Products (catches vra, vmsp, vidb, vrni -- all product IDs)
/opt/vmware/vpostgres/14/bin/psql -U vrlcm -d vrlcm -c \
```

```
  "DELETE FROM vm_lcops_product WHERE environment_vmid IN (
    SELECT vmid FROM vm_lcops_environments WHERE status != 'COMPLETED');"

## Level 2: Infrastructure properties
/opt/vmware/vpostgres/14/bin/psql -U vrlcm -d vrlcm -c \
  "DELETE FROM vm_lcops_infrastructure_properties WHERE environment_vmid IN (
    SELECT vmid FROM vm_lcops_environments WHERE status != 'COMPLETED');"

## Level 1: Environments (parent -- delete last)
/opt/vmware/vpostgres/14/bin/psql -U vrlcm -d vrlcm -c \
  "DELETE FROM vm_lcops_environments WHERE status != 'COMPLETED';"

## Clean independent tables
/opt/vmware/vpostgres/14/bin/psql -U vrlcm -d vrlcm -c \
  "UPDATE vm_rs_request SET state='FAILED' WHERE state NOT IN ('COMPLETED','FAILED');"
/opt/vmware/vpostgres/14/bin/psql -U vrlcm -d vrlcm -c \
  "UPDATE vm_engine_execution_request SET enginestatus='FAILED'
   WHERE enginestatus NOT IN ('SUCCESS','FAILED','COMPLETED');"
```

**Step 4: Verify the database is clean.**

```
/opt/vmware/vpostgres/14/bin/psql -U vrlcm -d vrlcm -c \
  "SELECT environmentid, environmentstatus, status FROM vm_lcops_environments;"
```

You should see only rows with `COMPLETED` status (for VCF Operations and Logs, which are already deployed).

**Step 5: Restart LCM.**

```
systemctl start vrlcm-server
```

**Step 6: Restart the VCF Operations web service** to clear its cached state:

```
ssh root@192.168.1.77
systemctl restart vmware-vcops-web
```

This is critical -- the VCF Operations appliance caches deployment state in memory. Without restarting `vmware-vcops-web`, the UI will still show the old "parallel deployment" error even though the database is clean.

### *9.5 How to Prevent This*

- **Never abandon a deployment wizard halfway through.** There is no "cancel" button that cleans up the database records.
- **Do not browse Aria product pages in the UI "just to look."** Each page visit can create orphaned records.
- **If you see the "parallel deployment" error,** do not keep retrying from the UI. Each retry creates additional orphaned records.
- **Take a VM snapshot** of the LCM appliance before every deployment attempt.

---

## 10. Issue 8 -- Certificate SAN Mismatch (April 16)

### *10.1 What Is This?*

> **What is this issue about?** After clearing the database block, the Automation deployment wizard ran a "precheck" -- automated tests that verify everything is ready. One precheck validates the TLS certificate, and it

failed because the certificate did not list all the hostnames and IP addresses that VCF Automation requires.

**What is a TLS certificate?** A TLS certificate is a digital document that proves a server's identity and enables encrypted communication. It is like a driver's license for a computer. Certificates contain a list of names and IPs they are valid for -- this list is called the SAN (Subject Alternative Name). If a name is not on the SAN, the connection is rejected.

### *10.2 What Happened*

The precheck failed with:

```
PRECHECK FAILED: The hosts in the certificate doesn't match
with the provided/product hosts
```

### *10.3 Why Did This Happen?*

**Root Cause:** The TLS certificate did not include all the hostnames and IP addresses that VCF Automation requires. VCF Automation runs on Kubernetes with a cluster of three nodes plus one virtual IP. The certificate SAN must include ALL of these:

- **Cluster FQDN:** `automation-node1.lab.local`
- **All node FQDNs:** `vcfa-mgmt-0.lab.local` , `vcfa-mgmt-1.lab.local` , `vcfa-mgmt-2.lab.local`
- **All short hostnames:** `automation-node1` , `vcfa-mgmt-0` , `vcfa-mgmt-1` , `vcfa-mgmt-2`
- **All IP addresses:** `192.168.1.91` , `192.168.1.92` , `192.168.1.93` , `192.168.1.94`

The original certificate only included the cluster FQDN and cluster VIP, missing all node-specific entries.

**Reference:** Broadcom KB 412322

### *10.4 The Fix*

**Fix:** Generate a new TLS certificate with all required SAN entries using OpenSSL.

**Step 1: Create the OpenSSL configuration file** ( `vcfa-openssl.cnf` ):

```
[req]
default_bits = 2048
prompt = no
default_md = sha256
req_extensions = v3_req
distinguished_name = dn

[dn]
C = US
ST = State
L = City
O = Lab
OU = VCF
CN = automation-node1.lab.local

[v3_req]
```

```
basicConstraints = CA:FALSE
keyUsage = digitalSignature, keyEncipherment
extendedKeyUsage = serverAuth, clientAuth
subjectAltName = @alt_names

[alt_names]
DNS.1 = automation-node1.lab.local
DNS.2 = automation-node1
DNS.3 = vcfa-mgmt-0.lab.local
DNS.4 = vcfa-mgmt-0
DNS.5 = vcfa-mgmt-1.lab.local
DNS.6 = vcfa-mgmt-1
DNS.7 = vcfa-mgmt-2.lab.local
DNS.8 = vcfa-mgmt-2
IP.1 = 192.168.1.91
IP.2 = 192.168.1.92
IP.3 = 192.168.1.93
IP.4 = 192.168.1.94
```

**What does each section mean?** - `[req]` -- General settings. `default_bits = 2048` is the encryption key size. - `[dn]` -- Distinguished Name: the identity information in the certificate. `CN` is the primary hostname. - `[v3_req]` -- X.509 v3 extensions (modern certificate features). - `[alt_names]` -- **THE CRITICAL SECTION**: All hostnames (DNS) and IP addresses the certificate covers.

**Step 2: Generate a private key.**

```
openssl genrsa -out vcfa.key 2048
```

**Step 3: Generate the certificate.**

```
openssl req -new -x509 -key vcfa.key -out vcfa.crt -days 730 \
  -config vcfa-openssl.cnf -extensions v3_req
```

**Step 4: Verify the SAN entries.**

```
openssl x509 -in vcfa.crt -text -noout | grep -A 20 "Subject Alternative Name"
```

**Step 5: Create DNS records** for all node FQDNs on your DNS server (192.168.1.230):

| Hostname | Type | IP Address |
|---|---|---|
| vcfa-mgmt-0.lab.local | A | 192.168.1.92 |
| vcfa-mgmt-1.lab.local | A | 192.168.1.93 |
| vcfa-mgmt-2.lab.local | A | 192.168.1.94 |

**Step 6: Upload the new certificate** in the VCF Automation deployment wizard (import into LCM's certificate locker first) and re-run the precheck.

**Result:** Precheck passes.

### *10.5 How to Prevent This*

- **Before starting any VCF Automation deployment,** create the certificate with ALL required SAN entries.

- **The naming convention is always:** `{node-prefix}-0`, `{node-prefix}-1`, `{node-prefix}-2` for the three Kubernetes nodes, plus the cluster FQDN.
- **Include both FQDNs and short hostnames** in the SAN. Kubernetes uses both.
- **Include IP addresses** in the SAN. Some internal health checks connect by IP.
- **Reference:** Broadcom KB 412322

---

## 11. Issue 9 -- API Deployment Bypass (April 16)

### *11.1 What Is This?*

**What is this issue about?** Even after cleaning the LCM database (Issue 7), the VCF Operations UI kept recreating the "parallel deployment" block every time the deployment page was opened. The UI itself was the source of the bug. The solution was to bypass the UI entirely and submit the deployment via the LCM REST API using `curl` commands.

**What is an API deployment?** Instead of clicking buttons in a web interface, you send commands directly to the server using a tool called `curl`. Think of it like texting someone instead of calling them -- you send the same information through a different channel. The API bypasses the buggy web UI entirely.

### *11.2 What Happened*

After cleaning the LCM database, the UI submit button still failed with the "parallel deployment" error. Every visit to the deployment page in VCF Operations created a new `UI_INPROGRESS` record in the LCM database, immediately re-blocking deployment.

### *11.3 Why Did This Happen?*

**Root Cause:** The "Parallel Deployments" check runs in the **VCF Operations browser code**, not in LCM itself. When you open the Automation deployment page in VCF Operations:

1. The UI sends a request to LCM to create a new "environment" record
2. LCM creates the record with status `UI_INPROGRESS`
3. The UI then checks for existing `UI_INPROGRESS` records and blocks submission
4. Navigating away never cleans up the record
5. Cleaning the database and going back to the UI just creates a NEW record

This is an endless loop when using the UI. The only way to break it is to submit the deployment through the LCM REST API directly, which does not trigger the VCF Operations UI code.

### *11.4 The Fix: API Deployment*

**Fix:** Submit the deployment via the LCM REST API ( `POST /lcm/lcops/api/v2/environments` ) which bypasses the UI entirely.

**Step 1: Create an authentication credentials file.**

```
 ## On the LCM appliance (192.168.1.95):
echo 'admin@local:<your-password>' > /tmp/auth.txt
```

We use a file because the password contains `!` characters that bash interprets as history expansion commands. Putting the password in single quotes inside an `echo` command prevents this.

**Step 2: Verify API access.**

```
curl -sk -u "$(cat /tmp/auth.txt)" \
  'https://localhost/lcm/lcops/api/v2/environments' 2>&1 | head -20
```

**What each flag means:**

| Flag | What It Does |
|---|---|
| `-s` | Silent mode -- do not show the progress bar |
| `-k` | Skip SSL certificate verification (LCM uses a self-signed certificate) |
| `-u "$(cat /tmp/auth.txt)"` | Read the username:password from the auth file |

If this returns JSON data, the API is working and credentials are correct.

**Step 3: Create the deployment JSON payload.**

Use the UI's **Export Configuration** feature to capture the correct payload format. In the VCF Operations deployment wizard, there is an "Export Configuration" button that downloads a JSON file with all the correct property names and locker references. This is critical because the property names and structure are not well documented.

Save the payload as `/tmp/vra-deploy.json` on the LCM appliance (or download it from another server).

**Step 4: Submit the deployment.**

```
curl -sk -u "$(cat /tmp/auth.txt)" \
  -X POST 'https://localhost/lcm/lcops/api/v2/environments' \
  -H 'Content-Type: application/json' \
  -d @/tmp/vra-deploy.json 2>&1 | head -10
```

| Flag | What It Does |
|---|---|
| `-X POST` | Send a POST request (used to create new resources) |
| `-H 'Content-Type: application/json'` | Tell the server we are sending JSON data |
| `-d @/tmp/vra-deploy.json` | Use the file contents as the request body. The `@` means "read from file" |

**Successful response:**

```
{"requestId":"aef386fe-9407-42a9-9933-13722d69284f"}
```

**Step 5: Monitor the deployment.**

```
## Check status using the request ID:
curl -sk -u "$(cat /tmp/auth.txt)" \
  'https://localhost/lcm/request/api/v2/requests/REQUEST_ID_HERE' 2>&1 \
  | python -c "import sys,json;d=json.load(sys.stdin);print(d.get('state','?'))"
```

**Possible states:**

| State | Meaning |
|---|---|
| `INPROGRESS` | Deployment is still running -- check back later |
| `COMPLETED` | Deployment finished successfully |
| `FAILED` | Something went wrong -- check the `errorCause` field |

### *11.5 The Correct JSON Payload*

> **CRITICAL:** The `ipPool`, `primaryVip`, and `internalClusterCidr` properties must go inside `products[0].nodes[0].properties`, NOT in `products[0].properties`. Getting this wrong causes a `LCMVMSP10000` error: "Certain Mandatory properties are missing in the component spec properties : [ipPool]". The correct structure was captured using the UI's Export Configuration feature.

Here is the exact correct JSON payload (from `E:\VCF-Depot\certs\vra-deploy.json`):

```
{
  "environmentName": "vcfa-deployment",
  "infrastructure": {
    "properties": {
      "dataCenterVmid": "DEFAULT_DC",
      "regionName": "",
      "zoneName": "",
      "vCenterName": "vcenter.lab.local",
      "vCenterHost": "vcenter.lab.local",
      "vcUsername": "svc-lcm-vc-dcc-1@vsphere.local",
      "vcPassword": "locker:password:<UUID>:vcenter.lab.local",
      "acceptEULA": "true",
      "enableTelemetry": "true",
      "certificate": "locker:certificate:<UUID>:vcfa-import",
      "cluster": "vcenter-dc01#vcenter-cl01",
      "storage": "vcenter-cl01-ds-vsan01",
      "folderName": "group-v12(Discovered virtual machine)",
      "network": "vcenter-cl01-vds01-pg-vm-mgmt",
      "dns": "192.168.1.230",
      "domain": "lab.local",
      "gateway": "192.168.1.1",
      "netmask": "255.255.255.0",
      "searchpath": "lab.local",
      "timeSyncMode": "ntp",
      "ntp": "192.168.1.230",
      "isDhcp": "false",
      "vcfProperties": "{\"vcfEnabled\":true,\"sddcManagerDetails\":[]}",
      "isVcfEnabledEnv": "true"
    }
  },
  "products": [
    {
      "id": "vra",
      "version": "9.0.1.0",
      "properties": {
```

```
        "certificate": "locker:certificate:<UUID>:vcfa-import",
        "productPassword": "locker:password:<UUID>:automation root",
        "productHostName": "automation-node1.lab.local",
        "deployOption": "small"
      },
      "clusterVIP": {
        "clusterVips": [
          {
            "type": "vcfa",
            "properties": {
              "hostName": "automation-node1.lab.local",
              "controllerType": "INTERNAL"
            }
          }
        ]
      },
      "nodes": [
        {
          "type": "vcfa-primary",
          "properties": {
            "vmNamePrefix": "vcfa-mgmt",
            "primaryVip": "192.168.1.91",
            "ipPool": "192.168.1.92-192.168.1.94",
            "internalClusterCidr": "198.18.0.0/15"
          }
        }
      ]
    }
  ]
}
```

**Key things to notice:**

1. `ipPool` **is in** `nodes[0].properties` -- NOT in `products[0].properties`. This is the most common mistake.
2. `primaryVip` **is also in** `nodes[0].properties` -- this is the cluster VIP (192.168.1.91).
3. **Locker references** (like `locker:password:<UUID>:...`) point to credentials stored in LCM's secure vault, not plaintext passwords.
4. `ipPool` **format is a range** (`192.168.1.92-192.168.1.94`), giving the 3 IPs for Kubernetes nodes.
5. `internalClusterCidr` (`198.18.0.0/15`) is the internal network range for Kubernetes pod communication. It does NOT conflict with your 192.168.1.x management network.

#### *11.6 How to Prevent This*

- When the VCF Operations UI has a persistent bug, **bypass it with the LCM REST API**.
- **Always use the UI's "Export Configuration" button** to capture the correct JSON payload format before resorting to API deployment.
- The `ipPool` property is **mandatory** and must be in `nodes[0].properties`.
- Create JSON payloads on a machine where you can verify the content, then download them to LCM.

---

### 11.5 Issue 10 -- VMSP etcd CrashLoopBackOff on HDD-Backed Nested vSAN (April 17)

#### *11.5.1 What Is This?*

**What is this issue about?** After all nine prior issues were resolved and the first API-based deployment attempt (April 16) appeared to start cleanly, a subsequent Fleet Management-driven redeployment on April 17 failed at stage 3 of 14 after running for nearly two hours. The VMSP Kubernetes platform came up but etcd (the distributed key-value store Kubernetes depends on) could not keep up with disk writes, causing the control

plane to fall over repeatedly. The root cause was physical: the underlying datastore is spinning disk (HDD), not SSD, and VMSP's write patterns cannot survive on rotational storage.

### *11.5.2 What Happened*

Fleet Management (VCF Operations → Fleet Management → VCF Management → Tasks) showed the 14-stage deployment pipeline failing at Stage 3 ("Deploy appliance cluster") with:

```
 Error Code: LCMVSPHERECONFIG1000095
ERR:DEPLOY0003 -- Deploy registry on VCF services platform cluster
Failed to create services platform cluster.
Unable to connect to the server: dial tcp 192.168.1.91:6443:
  connect: no route to host
```

Total elapsed before failure: **1 hour 57 minutes**. The appliance VM `vcfa-mgmt-z6mnh` was created successfully, booted, kubeadm-init completed, and the VIP 192.168.1.91 was briefly held. Then etcd entered `CrashLoopBackOff`, the kube-apiserver lost its backing store, the VIP went stale, the registry PackageDeployment hit its 10-minute retry deadline, and Fleet Management rolled back the VM (powering it off and deleting it — which is why `.91` and `.92` stopped pinging after the failure).

From `/var/log/vrlcm/vmsp_bootstrap_3fcac375-3686-4bbe-afa9-34bc1bdc76a2.log`:

```
 message: CrashLoopBackOff
reason: PodFailed
message: 'Failed to connect to the etcd Pod on the vcfa-mgmt-z6mnh Node:
         could not establish a connection to any etcd node:
         unable to create etcd client:
         failed to get etcd status: context deadline exceeded'
reason: MemberInspectionFailed
```

Dependency cascade observed in logs:

```
 registry                   False   dependency 'cluster-api-installer' not ready
cluster-api-installer      False   (depends on antrea)
antrea                     False   dependency unmet -- CNI cannot initialize
vsphere-csi                False   dependency 'antrea' not ready
seaweedfs                  False   dependency 'vmsp-scripts' not ready
kube-prometheus-stack      False   api-server unreachable at 192.168.1.91:6443
```

### *11.5.3 Why Did This Happen?*

**Root Cause:** esxi04's virtual disks (including the nested vSAN capacity VMDK at `D:\VMs\...` initially, later on HDD after the lab reorg) are backed by spinning 4 TB HDDs on the physical Dell Precision 7920 workstation host. Nested vSAN layers additional write amplification on top — every write is checksummed, replicated per FTT policy, and metadata-updated, all serialized onto rotational storage.

VMSP's etcd ships with these defaults (from the cluster kubeadm config in the bootstrap log):

```
etcd:
  local:
    extraArgs:
```

```
      election-timeout: "10000" # 10 seconds
      heartbeat-interval: "500" # 500 milliseconds
```

These are already the maximum values VMware ships. On spinning disks, fsync latency during burst writes routinely exceeds 100-300 ms. Missing a single heartbeat marks the etcd member as unavailable; missing five triggers re-election; re-election needs another fsync that also misses; etcd enters `CrashLoopBackOff`.

**Why host-level metrics hid this:**

vCenter's `disk.maxTotalLatency.latest` metric showed only **4-6 ms average** and **max 6 ms** during the two-hour failure window, with CPU at 3% and memory at 19%. That metric is sampled 20 seconds out of every 5 minutes and reports at the outer-host storage stack level. Nested-vSAN write amplification and inner-guest fsync bursts happen below the host's measurement point. The "signature" that pinned it to disk was **1 hour 17 minutes for kubeadm-init** (should take 15-30 minutes) on an otherwise-idle host -- the classic profile of etcd coming up on storage that barely keeps up while idle and falls apart the moment real writes arrive.

**Why this is architectural, not tunable:** No VCF 9 configuration option can compensate for rotational storage under VMSP's workload pattern. Even ESA (Express Storage Architecture), reduced FTT policies, or disabling vSAN encryption do not lower fsync latency enough on HDD. The only fix is SSD.

### *11.5.4 The Fix: Move VMSP Storage to SSD*

**Fix:** Relocate the VMSP appliance VMDKs from the HDD-backed datastore ( `vcenter-cl01-ds-vsan01` ) to an SSD-backed VMFS datastore on esxi04. Update the LCM environment storage property to target the new datastore, clean the failed LCM environment, and redeploy.

**Hardware in progress (ordered April 17, ETA April 21):**

- Crucial T500 2 TB NVMe SSD ( `CT2000T500SSD8` ) — TLC with DRAM, 1200 TBW endurance
- Sabrent EC-PCIE PCIe-to-M.2 adapter with built-in heatsink (passive PCIe x4 adapter)

**Interim plan (before the new drive arrives):**

If an existing SSD drive letter has available space on the Workstation host, carve a 500 GB partition and assign to esxi04 as a local datastore. Otherwise wait for the T500.

**Installation + retry procedure:**

```
 ## Step 1 (physical): Power down the 7920 workstation. Install Sabrent adapter
## in any free PCIe x4/x8/x16 slot. Insert T500 into the adapter's M.2 socket.
## Boot Windows. Confirm new drive appears in Disk Management.

## Step 2: Initialize and format the new drive
## Use Disk Management UI: Initialize GPT, create single simple volume, format NTFS,
## assign drive letter (assume F: in examples below).

## Step 3: In VMware Workstation, add a virtual disk to the esxi04 VM.
##   Storage type: SCSI (not SATA, not NVMe -- nested ESXi handles it cleanest)
##   Size: 1500 GB, Thick Provisioned, Lazy Zeroed
##   Location: F:\VMs\esxi04-ssd-storage.vmdk

## Step 4: Inside esxi04 via vCenter, rescan storage adapters, create new VMFS 6
## datastore on the new disk: name = esxi04-ssd-ds
```

```
## Step 5: Clean the failed LCM environment per section 12 of this report
## (environment id: 3fcac375-3686-4bbe-afa9-34bc1bdc76a2)

## Step 6: Update the storage target in the VRA deployment JSON
## E:\VCF-Depot\certs\vra-deploy.json:
##   infrastructure.properties.storage = "esxi04-ssd-ds" (was vcenter-cl01-ds-vsan01)

## Step 7: Retry the deployment via Fleet Management or LCM API
## Expected: Stage 3 kubeadm-init completes in ~20 minutes (down from 1h 17m)
```

#### *11.5.5 How to Prevent This*

- **Never place VMSP etcd on rotational storage or on nested vSAN built on rotational storage.** The cluster will appear to deploy then fail under any real load. Use only NVMe SSD or direct SSD VMFS.
- **Host-level disk latency metrics lie for nested workloads.** The outer host's 20-second sample will show low latency while the inner guest experiences crippling fsync bursts. Measure from inside the guest (iostat / systemd-journal for fsync warnings) if diagnosing performance problems in nested environments.
- **Watch kubeadm-init duration.** If it takes over an hour on a VMSP bootstrap, abort and reassess storage before the registry PackageDeployment hits its 10-minute retry cap.
- **Reserve headroom on nested vSAN.** FTT=1 policies need space to rebuild when a host goes unstable. Below 20% free is risky; below 10% is dangerous.

---

### 11.6 Issue 11 -- esxi01 Backing Drive Saturated, Host NotResponding (April 17)

#### *11.6.1 What Is This?*

**What is this issue about?** The physical Windows drive (D: SSD) that holds esxi01's VMDK files filled to 100% capacity. When Windows reports zero bytes free, VMware Workstation can no longer write to the nested ESXi host's disks. esxi01 stopped responding to vCenter management heartbeats even though the VM appeared to still be "running." Two critical lab VMs ( `nsx-manager` , `fleet` ) are on esxi01 and are affected. Nested vSAN cluster free space dropped to 8.8% (176 GB of 2000 GB).

#### *11.6.2 What Happened*

On April 17, during concurrent VCFA redeployment work, esxi01 began showing `NotResponding` in vCenter. PowerCLI probes of esxi01 returned `ConnectionState: NotResponding` with no CPU or memory readings. A File Explorer properties check of the D: drive on the Workstation host showed:

```
Drive D:\ (Storage-954GB)
  Used space:  1,024,208,138,240 bytes   (953 GB)
  Free space:                0 bytes     (0 GB)
  Capacity:    1,024,208,138,240 bytes   (953 GB)
```

Drive layout and free space on the Workstation host at the time:

| Drive | Label | Size | Free | Free % | Contents |
|---|---|---|---|---|---|
| C: | Windows | 952 GB | 116 GB | 12.2% | Windows OS + user profiles |
| D: | Storage-954GB | 954 GB | **0 GB** | **0%** | esxi01 VM files ONLY |

| Drive | Label | Size | Free | Free % | Contents |
|---|---|---|---|---|---|
| E: | Storage-3726GB | 3726 GB | 1806 GB | 48.5% | VCF-Depot (depot, docs, screenshots) |
| F: | Storage-3726GB | 3726 GB | 2407 GB | 64.6% | (mostly empty) |
| G: | Backups | 931 GB | 179 GB | 19.2% | Backup files |

Contents of `D:\VMs\esxi01.lab.local\`:

| File | Size GB | Role |
|---|---|---|
| esxi01-datastore-cl1.vmdk | 354 | Local datastore |
| esxi01-vsan-capacity-cl1.vmdk | 349 | vSAN capacity tier |
| esxi01-vsan-cache-cl1.vmdk | 98 | vSAN cache tier |
| 564dfd4e-7467-...-fbfa3ca9.vmem | **88** | Stale suspend memory file (VM suspended April 14, not cleaned) |
| esxi01.lab.addon.vmdk | 57 | Add-on disk |
| esxi01.lab.local-cl1.vmdk | 5 | ESXi boot disk |
| **Total** | **~951** | **100% of D:** |

### *11.6.3 Why Did This Happen?*

**Root Cause:** Two compounding issues caused the drive to fill:

1. **Thin-provisioned vSAN VMDK growth.** The `esxi01-vsan-capacity-cl1.vmdk` file was thin-provisioned by Workstation to allow on-demand growth as the nested vSAN cluster stored data. As vSAN objects were written across the cluster (every VM's disks are mirrored per FTT=1), the capacity VMDK on each host grew. Over weeks of lab use, this disk reached 349 GB on esxi01 alone.
2. **Orphaned suspend memory file.** VMware Workstation creates `.vmem` files when a VM is suspended. This file equals the guest's allocated RAM (in this case 88 GB, because esxi04 was configured at 88 GB). When a suspended VM is resumed, Workstation does not always delete the .vmem file automatically -- it can linger indefinitely. The 88 GB `.vmem` on D: was dated April 14 (three days stale), while the VMDKs show write activity as recent as today, meaning the VM was resumed long ago and the .vmem was orphaned.

Together, these pushed D: to 953 of 954 GB used. When ESXi inside esxi01 attempted its next disk write, Workstation failed the write. ESXi logged I/O errors, lost its ability to complete vCenter heartbeats, and was marked `NotResponding` by vCenter.

**Why the cluster survived (so far):** vSAN FTT=1 policy means every object has at least one mirror on a different host. esxi01's inability to write means its mirrors went stale, but the other three hosts (esxi02, esxi03, esxi04) still have the primary copies. The cluster can read but cannot currently restore full redundancy because vSAN has insufficient headroom (8.8% free) to rebuild esxi01's portion onto the other three hosts.

### *11.6.4 The Fix: Relocate esxi01 to F: and Clear Stale .vmem*

**Fix:** Power off esxi01 in Workstation, delete the orphaned .vmem file (frees 88 GB instantly), then move the entire `esxi01.lab.local\` folder from D: (0 GB free) to F: (2407 GB free). Re-register the VM in Workstation from the new location. Power on and let vSAN resync.

**Step-by-step:**

```
## Step 1: In VMware Workstation, power off esxi01 hard (vCenter clean shutdown
## not possible -- host is NotResponding). VM menu -> Power -> Power Off.

## Step 2: Delete the orphan .vmem to free immediate space
Remove-Item "D:\VMs\esxi01.lab.local\564dfd4e-7467-4cb4-f5d0-4125fbfa3ca9.vmem" -Force

## Verify drive is no longer at 0 bytes -- should show ~88 GB free now.

## Step 3: Remove the VM from Workstation inventory (does NOT delete files):
##   Right-click esxi01 tab -> Remove -> confirm.

## Step 4: Move entire folder to F: (expect several hours for ~863 GB after .vmem delete)
Move-Item "D:\VMs\esxi01.lab.local" "F:\VMs\esxi01.lab.local"

## Step 5: Open the VM from its new location:
##   VMware Workstation: File -> Open -> F:\VMs\esxi01.lab.local\esxi01.lab.local.vmx

## Step 6: Power on the VM. When asked "I moved it" vs "I copied it", select
## "I MOVED IT" to preserve the UUID -- otherwise vCenter will see it as a new host
## and vSAN will refuse to re-admit the node.

## Step 7: Wait for esxi01 to boot, reconnect to vCenter, and for vSAN to begin
## resync of out-of-date components. Monitor via:
##   vCenter > Cluster > Monitor > vSAN > Resync Objects
## Expect the resync window to take 1-3 hours depending on cluster churn.

## Step 8: Once vSAN resync completes and esxi01 shows green health, D: is free.
## That 954 GB of SSD space can then be used for the VMSP relocation (Issue 10 fix).
```

#### *11.6.5 How to Prevent This*

- **Monitor Workstation host drive free space.** Nested vSAN's thin VMDKs grow silently until they fill. Set a Windows alert or run a weekly scheduled task that warns when any volume drops below 20% free.
- **Clean up .vmem files after resume.** After resuming any suspended Workstation VM, check the VM's folder for leftover `.vmem` files dated before the resume and delete them.
- **Do not co-locate multiple heavy nested hosts on the same physical drive.** Spread esxi01 through esxi04 across different physical disks so a single-drive failure cannot take out the whole nested cluster.
- **Pre-allocate or cap vSAN capacity VMDK size.** For long-running labs, consider changing the vSAN capacity VMDK from thin to thick-lazy-zeroed so growth is bounded at creation time and a drive-fill scenario cannot happen silently.
- **Keep at least 20% vSAN free.** Under FTT=1, vSAN needs rebuild headroom. When free drops below 20%, a host failure may not be recoverable.

---

### 11.7 Issue 12 -- SeaweedFS Pods Cannot Start in VCFA Internal K8s (May 7, 2026)

#### *11.7.1 What Is This?*

After the 2026-05-06 NSX rebuild (see `NSX-Manager-Cold-Start-Troubleshooting-Report.md` Addendum D), a fresh VCFA 9.0.1.0 install attempt failed with the **same surface symptom as Issue 10** ( `vmsp-platform-core-app-not-regist`

`ry` PackageDeployment timing out repeatedly) but with a different underlying root cause. Storage was now SSD-backed vSAN (the Issue 10 fix), and the K8s control-plane bootstrap (kubeadm-init / etcd) actually completed this time. The new failure is one layer up in the stack: **SeaweedFS pods inside the VCFA appliance's internal K8s cluster cannot start** — their log outputs are empty, indicating the pods never reach Running state.

### *11.7.2 What Happened*

**Deploy timeline (UTC):**

| Time | Stage | Activity |
|---|---|---|
| 02:15 | Submit | Fleet workflow initiated, all Precheck items pass |
| 02:16-05:25 | Pre-OVA | Bundle extract, locker access, network validation, cert generation |
| 05:25 | Stage 6/14 boundary | Fleet creates `vcfa-mgmt-p8zpb` (vm-5020) on esxi04, powers it on |
| 05:25 | Bootstrap start | `2026/05/07 05:25:50 create ssh password secret on VCF services platform cluster vcf-mgmt-4aea749506` |
| 05:25 | Core packages | `2026/05/07 05:25:51 Remote deploy core packages without the registry` — busybox, kube-rbac-proxy, kyverno, kyverno-policies, seaweedfs, vmsp-scripts, zalando-pg-operator: all `created` |
| 05:25 | PackageDeployment | `releases.vmsp.vmware.com/v1alpha1, Kind=PackageDeployment/vmsp-platform/vmsp-platform-core-app-not-registry: created` |
| 05:55:55 | First timeout | `retrying after error: deployment timed out: context deadline exceeded` (30 min wait) |
| 06:26, 06:56, 07:26, 07:56, 08:27 | Retries 2-6 | All identical 30-minute timeouts; helmrelease/seaweedfs patched between retries |
| 08:27:23 | Rollback | Fleet tears down `vcfa-mgmt-p8zpb` (vm-5020) |
| 08:27:36 | Cleanup attempt | `Unable to connect to the server: dial tcp 192.168.1.91:6443: connect: no route to host` (post-teardown, expected) |
| Total | | **3 hours 1 minute** of bootstrap time, all retrying the same failed PackageDeployment |

**Critical evidence from `vmsp_bootstrap_161e26fb-a523-4ead-b734-690cc53a0687.log`:**

The K8s cluster INSIDE the VCFA appliance bootstrapped successfully (unlike Issue 10 where kubeadm-init never completed). Cluster-internal controllers reconcile cleanly:

```
{"level":"info","ts":"2026-05-07T08:25:41Z","msg":"reconciling","controller":"lease",
 "controllerGroup":"coordination.k8s.io","controllerKind":"Lease",
 "Lease":{"name":"kyverno-health","namespace":"vmsp-policies"},...}
```

CAPV controller is running and holding leases:

```
 I0507 08:26:45.602823  1 leaderelection.go:456] lock is held by capv-controller-manager-7b5d9f458b-
f4v5n_ed7679b1-f3d8-4d0b-9d9b-99c9608871df and has not yet expired
```

**But seaweedfs pods all FAILED to start — pod log dumps are EMPTY:**

```
 2026/05/07 08:27:19 Failed Pod vmsp-platform:seaweedfs-filer-0 logs on the vcf-mgmt-4aea749506-admin@vcf-
mgmt-4aea749506 cluster
-- LOG START (vmsp-platform:seaweedfs-filer-0) --
-- LOG END --
2026/05/07 08:27:20 Failed Pod vmsp-platform:seaweedfs-volume-0 logs on the vcf-mgmt-4aea749506-admin@vcf-
mgmt-4aea749506 cluster
-- LOG START (vmsp-platform:seaweedfs-volume-0) --
-- LOG END --
2026/05/07 08:27:20 Failed Pod vmsp-platform:seaweedfs-volume-1 logs on the vcf-mgmt-4aea749506-admin@vcf-
mgmt-4aea749506 cluster
-- LOG START (vmsp-platform:seaweedfs-volume-1) --
-- LOG END --
2026/05/07 08:27:20 Failed Pod vmsp-platform:seaweedfs-volume-2 logs on the vcf-mgmt-4aea749506-admin@vcf-
mgmt-4aea749506 cluster
-- LOG START (vmsp-platform:seaweedfs-volume-2) --
-- LOG END --
```

**Empty pod logs is the signature of pods that never reach Running state.** Possible internal K8s reasons:

- Pod stuck in `Pending` because PVCs are not binding (storage class can't provision PVs)
- Pod stuck in `ContainerCreating` because volume mounts fail
- Pod in `CrashLoopBackOff` so quickly that no log lines flush before container restart

The bootstrap log captures these via `kubectl logs` calls that return empty buffers.

### *11.7.3 Why Did This Happen?*

> **HISTORICAL CONTEXT — THIS SECTION'S HYPOTHESES ARE WRONG.** This section was written before the verified root cause was found. **All five hypotheses below were refuted on 2026-05-07 PM.** The actual root cause is documented in §11.7.8 (KB 382405 stuck vSAN catalog namespace), and the per-VM sizing assumptions in hypotheses #1 and #4 are factually incorrect for VCF Automation 9.0.1 — see §11.7.10. This section is preserved for the reasoning audit trail; do not act on its recommendations.

**This is NOT Issue 10.** Issue 10 was an HDD-storage-induced etcd CrashLoopBackOff that prevented kubeadm-init from completing. Issue 12 occurs AFTER kubeadm-init succeeded — the K8s control plane is healthy, kyverno and CAPV are running. The failure is specifically in the `seaweedfs` deployment within `vmsp-platform` namespace.

Without SSH access into the VCFA appliance during the active deploy (Fleet rolls back the appliance at deploy failure, taking the K8s cluster with it), the exact internal K8s state cannot be inspected post-mortem. **Hypotheses, ranked by plausibility:**

| # | Hypothesis | Why plausible | How to verify | Status |
|---|---|---|---|---|
| 1 | **Insufficient internal disk inside "small" appliance for seaweedfs PVCs** | "small" appliance has ~150 GB total disk; seaweedfs (filer + 3 volumes) requires ~50 GB of PVC capacity; subtracting K8s, kyverno, kube-rbac-proxy, zalando-pg-operator, the Photon OS + container images, the appliance may not have enough free space when seaweedfs tries to provision | Try `medium` deployment type; or SSH into the appliance during a retry attempt and check `kubectl describe pvc -n vmsp-platform` for `Pending` / `Insufficient resources` | **REFUTED** — PVCs were Pending because vSAN catalog namespace was stuck (§11.7.8), not because of disk capacity |

| # | Hypothesis | Why plausible | How to verify | Status |
|---|---|---|---|---|
| 2 | **Single-node anti-affinity violation** | seaweedfs deploys 3 volume replicas + 1 filer; spec may have anti-affinity preventing co-tenancy on the single VCFA appliance node | `kubectl describe pod -n vmsp-platform seaweedfs-volume-0` — look for `0/1 nodes are available: 1 node(s) didn't match pod affinity rules` | **REFUTED** — pods were Pending due to PVC binding failure, not affinity |
| 3 | **Local-path / vSphere CSI storage class not provisioning PVs** | If the K8s cluster's StorageClass uses vSphere CSI and the CSI driver can't reach vCenter (e.g., CM service issue), PVs never get created | `kubectl get pv,pvc -n vmsp-platform`; `kubectl describe storageclass` | **PARTIALLY CORRECT** — vSphere CSI was the failing path, but the cause was not CM-service unreachability; it was a stale `catalog` vmnamespace. See §11.7.8. |
| 4 | **Resource requests exceed appliance capacity** | "small" appliance is 4 vCPU / 16 GB RAM; if seaweedfs container resource requests sum to more than available, pods stay Pending | `kubectl describe nodes` — look at allocatable vs requested | **REFUTED on two counts:** (a) sizing assumption is wrong — VCFA "small" is 24 vCPU / 96 GB, not 4/16 (see §11.7.10); (b) pods were Pending due to PVC binding, not resource requests |
| 5 | **VMSP deployment expects multi-node K8s** | The bundle may be designed for the production multi-node deploy and the single-node appliance "small" path may have an unfixed bug for seaweedfs anti-affinity | Switch to medium/large deployment which provisions multi-node K8s | **REFUTED** — "small" runs the same VMSP bundle on a single 24/96 node successfully once the catalog issue is resolved |

The strongest candidate is **#1 + #2 in combination**: a small single-node K8s with limited disk being asked to host a multi-replica distributed-storage workload.

> **Author's note to future readers:** The "strongest candidate" claim above proved false. The empty pod logs, surface symptom of "seaweedfs failure", and the small appliance size led to a hypothesis that misattributed the failure to the deployment-type choice. The actual issue was at a layer below VMSP entirely — a vSAN datastore namespace that no CSI volume on that datastore could provision through. See §11.7.8.

### *11.7.4 The Fix (Pending Verification) — SUPERSEDED*

> **SUPERSEDED by §11.7.8.** The action plan below was based on the wrong hypotheses in §11.7.3 and the wrong sizing claim in step 2 ("medium = 8 vCPU / 32 GB"). VCF Automation does not have t-shirt sized per-VM allocations — both small and medium use 24 vCPU / 96 GB per VM (see §11.7.10). The verified fix (KB 382405 catalog namespace recovery) is in §11.7.8.

This issue is documented but NOT yet resolved. Action plan for the next attempt:

1. **Clean up the failed environment** per Section 12 of this document. The failed env ID is `161e26fb-a523-4ead-b734-690cc53a0687` (matches the `vmsp_bootstrap_*.log` filename).
2. **Try `medium` deployment type instead of `small`**. The medium form factor provides more resources (8 vCPU / 32 GB RAM, ~250 GB disk) and provisions a multi-node internal K8s cluster (typically 3 nodes), which would resolve both candidate root causes #1 and #2. **[Step is incorrect — sizing claim is wrong; switching to medium is harmful on hosts with < 96 GB physical RAM per nested ESXi. See §11.7.10.]**
3. **During the next retry, capture K8s pod state from inside the VCFA appliance**: SSH into `vcfa-mgmt-*` while the bootstrap is running (before Fleet rolls back), then run `kubectl --kubeconfig=/etc/rancher/k3s/k3s.yaml -n vmsp-platform get pods,pvc,events --sort-by='.lastTimestamp'` to capture the actual failure state.
4. **If `medium` also fails**, raise a Broadcom support ticket with the bootstrap log and the captured K8s pod state. This is at the edge of what nested-lab VCFA deployment can do.

### *11.7.5 How to Prevent This (Once Verified) — SUPERSEDED*

> **SUPERSEDED by §11.7.8 and §11.7.10.** The recommendations below predate verification of the actual root cause. The "minimum-working appliance form factor" framing is wrong — size is fixed per Broadcom and is not a tunable. See §11.7.10 for the correct preventive checklist.

After the next attempt confirms a working deploy size:

- Update `vra-deploy.json` to set `selectedDeploymentType` to the verified-working size (likely `medium`).
- Document the minimum-working appliance form factor in the `feedback_vcf_deploy.md` memory.
- Pre-flight check before submit: confirm host has enough free RAM for the chosen form factor (e.g., `medium` = 32 GB; need at least 40 GB free on target host for headroom). **[Numbers wrong — medium is 96 GB per VM × 3 = 288 GB. See §11.7.10.]**

### *11.7.6 Why "deployment timed out: context deadline exceeded" Was Misleading*

Fleet's error chain surfaces "context deadline exceeded" because the PackageDeployment never completes within the 30-minute kapp timeout. Each retry resets the timer and starts over with the same configuration, so 6 retries took 3 hours of real time. **The error message is the same as Issue 10's etcd-cliff failure** which makes diagnosis ambiguous — both can produce identical Fleet UI outputs. The differentiating evidence is in the bootstrap log content:

| Evidence | Issue 10 | Issue 12 |
|---|---|---|
| etcd cluster forms | No (`infinite retry` on kubeadm-init) | Yes |
| capv-controller-manager pods Running | No | **Yes** |
| kyverno-health controller reconciling | No | **Yes** |
| seaweedfs pod logs | Empty (cluster never came up) | **Empty (cluster came up but seaweedfs specifically can't start)** |
| Storage backing | HDD | **SSD-backed vSAN** |

The fix for Issue 10 (move to SSD) does NOT resolve Issue 12. They are separate failure modes that look identical at the Fleet UI level.

### *11.7.7 Cleanup Status (May 7, 2026)*

After Fleet's automatic rollback:

| Cleanup item | Status |
|---|---|
| `vcfa-mgmt-p8zpb` VM (vm-5020) | ✅ Removed by Fleet at 08:27:23 |
| Other VCFA-related VMs in vCenter | ✅ None (verified via `/api/vcenter/vm` list) |
| Failed env in Fleet DB ( `161e26fb-...` ) | ✅ Cleaned via Section 12 cascade DELETE on 2026-05-07 morning |
| LCM bootstrap log ( `/var/log/vrlcm/vmsp_bootstrap_161e26fb-*.log` ) | Preserved for diagnosis |

### *11.7.8 VERIFIED ROOT CAUSE (May 7, 2026 PM) — KB 382405 Stuck vSAN Catalog Namespace*

> **The hypotheses in §11.7.3 were ALL WRONG.** Issue 12 is NOT a sizing problem, anti-affinity violation, storage-class issue, or resource constraint. **The actual root cause is a stuck vSAN FCD/CNS catalog namespace** — a documented Broadcom failure mode (KB 382405) where the `catalog` `vmnamespace` object on the vSAN datastore enters a half-deleted lifecycle state and blocks every subsequent CSI volume provision, regardless of deployment size, host, or storage class.

**How this was discovered (May 7, 2026 afternoon, third deploy attempt):**

After two more retries of the medium-type deploy failed at the same point, deeper inspection of the workload K8s cluster revealed:

```
 kubectl get pvc -n vmsp-platform
data-filer-seaweedfs-filer-0   Pending  (37m)
data1-seaweedfs-volume-0       Pending  (37m)
data1-seaweedfs-volume-1       Pending  (37m)
data1-seaweedfs-volume-2       Pending  (37m)

kubectl get pods -n kube-system -l app=vsphere-csi-node
vsphere-csi-node-mkqdh   1/3   CrashLoopBackOff   14 restarts in 16 min
```

The vCenter Tasks pane simultaneously showed:

```
 Task: Create a virtual disk object
Source: VSPHERE.LOCAL\vsansvc-6ae12778-...
Host: esxi01.lab.local
Status: A general system error occurred: Failed to create namespace:
        nameSpacePath = /vmfs/volumes/vsan:52b68472c149d706-5c7eef39e7136ffa/catalog
        Failed to init fcd-catalog for datastore
```

**This is the verbatim signature of KB 382405** — "CNS PODs fail to start with error: Failed to create directory `/vmfs/volumes/.../catalog`". Not a Fleet bug, not a VCFA bug. A vSAN datastore-level orphan from prior failed CSI volume operations.

**Diagnostic confirmation:**

From any ESXi host:

```
 ## Per KB 382405's own check:
mkdir /vmfs/volumes/vcenter-cl01-ds-vsan01/catalog
mkdir: can't create directory '/vmfs/volumes/vcenter-cl01-ds-vsan01/catalog':
       No such device or address
```

KB 382405's verbatim resolution: "*Try to create a directory for catalog. In case of failure, please engage the Storage Vendor.*" — i.e., the documented self-service fix is exactly the `mkdir` that just failed; everything beyond requires Broadcom support involvement.

**Identifying the orphan vSAN namespace (read-only forensics):**

The vSAN datastore directory listing showed an orphan UUID alongside the legitimate VM directories:

```
 ls /vmfs/volumes/vcenter-cl01-ds-vsan01/
## Among normal entries (vcenter, fleet, nsx-manager, vcfa-mgmt-*, fcd, ...):
2385e169-9480-4df6-ec1f-000c29b4cef4   <-- ls of THIS errors "No such device or address"
```

CMMDS attribute lookup confirms it is the catalog:

```
 cmmds-tool find -u 2385e169-9480-4df6-ec1f-000c29b4cef4

## Output excerpt (DOM_NAME entry):
type=DOM_NAME ... [content = ("catalog" UUID_NULL i6)]
                                        # ^^^ state code i6 -- interpreted from context as a
                                        #     DELETING-class lifecycle marker. Broadcom does NOT
                                        #     publicly enumerate DOM_NAME state codes, so this
                                        #     interpretation is engineering inference from the
                                        #     observed behavior (entry exists in CMMDS, OSFS
                                        #     refuses to mount, mkdir at the same name fails).
type=CONFIG_STATUS ... [content = (... ("state" i7) ...)]
type=POLICY ... [content = (... ("hostFailuresToTolerate" i1) ...)]
Health: Healthy # vSAN sees the object as fine
```

`objtool getAttr` confirms the user-friendly identity:

```
 /usr/lib/vmware/osfs/bin/objtool getAttr -u 2385e169-9480-4df6-ec1f-000c29b4cef4

Object class:        vmnamespace
Object path:         /vmfs/volumes/vsan:52b68472c149d706-5c7eef39e7136ffa/catalog
User friendly name:  catalog
Object size:         273804165120  (~255 GB allocated capacity)
Allocated size:      754974720     (~720 MB actually used = orphan FCD metadata)
Group uuid:          0000...       (NULL &amp;mdash; abnormal)
Container uuid:      0000...       (NULL &amp;mdash; abnormal)
```

**The verified self-service fix** (run from the host that **owns** the orphan namespace object):

To find the owner host, do NOT use `Sub-Cluster Master UUID` (that is the cluster CMMDS coordinator; it is not necessarily the owner of any specific DOM object). Instead, read the `owner=` field directly from the CMMDS lookup:

```
 ## Find the owner host UUID for the orphan object:
cmmds-tool find -u <orphan-UUID> | grep -E '^owner='
## example output line:  owner=69f5436c-76ba-2b9a-ffcf-000c29649d0a(Health: Healthy) ...

## Map that UUID back to a hostname using the cluster member table:
esxcli vsan cluster get
## Cross-reference the UUID against "Sub-Cluster Member UUIDs" / "Sub-Cluster Member HostNames".
```

```
## (In this lab, the orphan owner UUID 69f5436c-... mapped to esxi02 -- which happened to also
##  be the cluster master, but that is coincidental, not guaranteed.)
```

Then SSH to the owner host and run:

```
 ## 1) Remove the stuck namespace object (`objtool delete -u <UUID> -f` syntax is documented by
##    Broadcom in [KB 428195](https://knowledge.broadcom.com/external/article/428195/resolving-
inaccessible-vsan-objects-and.html)
##    for orphaned snapshot objects; using the same syntax against a vmnamespace is not in any KB):
/usr/lib/vmware/osfs/bin/objtool delete -u 2385e169-9480-4df6-ec1f-000c29b4cef4
## Expected: rc=0

## 2) Recreate the catalog namespace fresh:
mkdir /vmfs/volumes/vcenter-cl01-ds-vsan01/catalog
## Expected: rc=0

## 3) Verify a new vmnamespace symlink was created:
ls -la /vmfs/volumes/vcenter-cl01-ds-vsan01/catalog
## Expected: catalog -> <new-UUID>  (e.g. 81cbfc69-7e58-c68d-73d0-000c29649d0a)
```

**Safety caveat:** KB 382405 stops at the failed `mkdir` and says "engage the Storage Vendor." The `objtool delete` step IS NOT in any KB — it was inferred from cross-referencing `cmmds-tool` and `objtool` evidence. **It is safe ONLY when** the catalog namespace contains no real CNS volume metadata; in this lab that is true because all VMs use traditional VMDK/named-directory storage and the only catalog content was 720 MB of orphan metadata from prior failed deploys. **In production, do not run `objtool delete` against the catalog without a Broadcom case approval** — you would orphan every CNS-managed volume on that datastore.

**Verified outcome (2026-05-07 13:25 EDT):**

Within 60 seconds of the `mkdir`:

```
 kubectl get pvc -n vmsp-platform
data-filer-seaweedfs-filer-0   Bound    pvc-827a650f-...   10Gi    (65s)
data1-seaweedfs-volume-0       Bound    pvc-e9fd4555-...   150Gi   (65s)
data1-seaweedfs-volume-1       Bound    pvc-bda51b17-...   150Gi   (63s)
data1-seaweedfs-volume-2       Bound    pvc-ae71f26a-...   150Gi   (62s)

kubectl get pods -n kube-system -l app=vsphere-csi-node
vsphere-csi-node-c479w   3/3   Running   (no longer crashlooping)
vsphere-csi-node-mkqdh   3/3   Running   (no longer crashlooping)
vsphere-csi-node-x2wfs   3/3   Running   (no longer crashlooping)

## Bootstrap log: SuccessfulAttachVolume on all three seaweedfs-volume pods within 30s of bind
```

**Why prior attempts both yesterday and earlier today all failed identically:** The orphan `2385e169-...` namespace was created on 2026-05-06 during the first failed VCFA deploy ( `161e26fb-...` ) when CSI tried to provision PVs for that deploy's seaweedfs StatefulSet. When that deploy was rolled back, the catalog was dereferenced but not properly finalized — leaving the DOM_NAME entry stuck in state 6 (DELETING). Every subsequent deploy attempt — small, medium, with different hosts, with different cleanup procedures — tried to create a `catalog` namespace, found one already in CMMDS at the target name, and failed identically with `No such device or address` . The retries during 2026-05-06's 5h13m run, the 3 retries on 2026-05-07 morning, and the 3.5h run before this fix were ALL hitting this same blocker.

**Why the surface symptom looked like seaweedfs:** seaweedfs is the FIRST workload package after kubeadm-init that requires CSI-provisioned PVs. Its 4 PVCs (filer + 3 volumes) hit the broken catalog first, which crashlooped

vsphere-csi-node, which made workload nodes flap NotReady, which made Fleet's bootstrap-watcher time out. The surface error message "*deployment timed out: context deadline exceeded*" hides the storage-layer cause completely.

**Updated diagnostic evidence table** (supersedes §11.7.6):

| Evidence | Issue 10 | Issue 12 (corrected) |
|---|---|---|
| etcd cluster forms | No ( `infinite retry` on kubeadm-init) | Yes |
| capv-controller-manager pods Running | No | Yes |
| kyverno-health controller reconciling | No | Yes |
| seaweedfs pod logs | Empty (cluster never came up) | Empty (pods never schedule because PVCs Pending) |
| **vSphere CSI node DaemonSet status** | N/A | **CrashLoopBackOff** ← key signal |
| **Workload PVC status** | N/A | **All Pending forever** ← key signal |
| **vCenter Tasks: "Failed to init fcd-catalog"** | No | **Yes ← smoking gun** |
| Storage backing | HDD | SSD-backed vSAN, but with orphan catalog namespace |

**To diagnose Issue 12 quickly on a future occurrence:**

1. Check vCenter → Tasks pane during deploy. Look for `Failed to init fcd-catalog` entries from `vsansvc-...` user.
2. From any ESXi host: `mkdir /vmfs/volumes/<vsan-datastore>/catalog`. If `No such device or address`, you have this issue.
3. Identify the orphan UUID: `ls /vmfs/volumes/<vsan-datastore>/` — the entry that errors when you `ls` it is the broken catalog.
4. Apply the `objtool delete` + `mkdir` fix from the **owner host** (sub-cluster master per `esxcli vsan cluster get`).
5. Within 60 seconds, vSphere CSI reconciliation begins binding Pending PVCs.

**Cross-references:**

- Broadcom KB 382405 — "*CNS PODs fail to start with error: Failed to create directory /vmfs/volumes/.../catalog*" (the canonical symptom + `mkdir` self-service test, "engage Storage Vendor" on failure)
- Broadcom KB 401698 — "*Error when trying to create container volume on vSAN datastore (CnsFault: VSLM task failed)*" (related symptom set)
- Broadcom KB 330164 — "*How to Retrieve and reconfigure FCD catalog's profile on vSAN datastore*" (post-recreate policy reapply, if needed)
- Broadcom KB 428195 — "*Resolving Inaccessible vSAN Objects and Orphaned Snapshots*" (documents `/usr/lib/vmware/osfs/bin/objtool delete -u <UUID> -f` syntax for orphan **snapshot** objects; same syntax was used against the vmnamespace catalog object in this lab but that use is NOT covered by the KB)
- Internal memory: `reference_vsan_fcd_catalog_recovery.md`

### *11.7.9 What Issue 12 Means for Future Deploys*

The lesson supersedes the prior recommendation to switch from `small` to `medium`:

- **Sizing is a separate concern from the catalog issue.** `small` failed yesterday because of the catalog. `medium` failed earlier today for the same reason. Either size will work once the catalog is healthy.
- **Always check for vSAN catalog state before retrying after a failed VCFA deploy.** The 5-second `mkdir` test will tell you immediately whether the prior failure left a stuck catalog. If it did, fix it before resubmitting.
- **The "small fits but seaweedfs fails" / "medium has nodes for seaweedfs but doesn't fit" tradeoff documented in earlier hypothesis was a misdiagnosis.** Both sizes can work once the catalog is healthy — but per §11.7.10, `medium` is *physically incompatible* with this lab's nested ESXi host size (each nested host is 92 GB; each VCFA VM needs 96 GB). On this hardware, **only `small` is viable.**

### *11.7.10 VCFA Sizing — Verified Facts and Correction of Earlier Sections*

> **Earlier sections of this document (§11.7.3 hypothesis #4, §11.7.4 step 2, §11.7.5 third bullet) cite VCFA "small" as 4 vCPU / 16 GB and VCFA "medium" as 8 vCPU / 32 GB. Those numbers are wrong. They are the t-shirt sizes for VCF Operations, not VCF Automation.** VCFA 9.0.1 has fixed per-VM sizing — the deployment type changes the *number* of VMs, not the per-VM resources.

**Verified per Broadcom (sources cited below):**

| Deployment Model | Type | VMs | vCPU per VM (default) | RAM per VM (default) | Total RAM (default) |
|---|---|---|---|---|---|
| **Simple** | small | 1 | 24 | 96 GB | 96 GB |
| **HA** | mediu m | 3 | 24 | 96 GB | 288 GB |
| **HA** | large | 3 | 24 (more allocated post-vertical-scale) | 96 GB (more allocated post-vertical-scale) | ~288 GB+ |

**Notes on each line:**

- **Small = "Simple" deployment model = 1 VM.** Medium and large both use the **HA deployment model = 3 VMs.** Per Broadcom TechDocs VCF Automation Deployment Models the difference between medium and large is **per-VM allocation, not VM count**. Earlier versions of this section incorrectly stated large = 5 nodes; that was wrong.
- VCFA supports BOTH **vertical** scaling (CPU/RAM increase per VM — KB 414551 procedure) AND **horizontal** scaling (Simple → HA — deploy a fresh HA cluster). The default at deploy is 24 vCPU / 96 GB per VM regardless of size choice; only the VM count changes between Simple and HA.
- William Lam: "*no workarounds or hacks for the memory requirements as these resources will be used by VCFA.*" The 96 GB is genuinely consumed by Postgres, Harbor registry, Istio, Kyverno, kube-prometheus-stack, seaweedfs, and the VCFA application pods themselves — not idle reservation.
- **vCPU CAN be reduced at clone time.** Lam (June 2025 update) and Tom Fojta both document manually editing a powered-off VCFA VM down to 16 (or even 12) vCPU before initial power-on. This is the only documented homelab-friendly tuning. Memory reduction is undocumented and reportedly fails.

**Sources (verified 2026-05-07):**

- Broadcom TechDocs — VCF Automation Deployment Models (Simple vs HA, VM counts)
- Broadcom TechDocs — Vertical Scale-Up for Automation
- Broadcom KB 414551 — How to Scale Up VCF Automation 9.0
- Broadcom Developer Portal — VcfAutomationSpec ( `ipPool` minItems=2 / maxItems=4)
- William Lam — Minimal resources for deploying VCF 9.0 in a Lab (June 2025)

- Tom Fojta — Deploying VCF 9 in a Minilab (June 2025)

**Practical implications for this homelab (Dell 7920, 192 GB physical, 4× 92 GB nested ESXi):**

1. `medium` **(HA model) is physically incompatible.** A single 96 GB VCFA VM cannot fit on a single 92 GB nested ESXi without memory overcommit, and HA needs 3 such VMs. The 2026-05-07 medium attempts only got VMs to "power on" because vSphere aggressively ballooned/swapped — which manifested as the kube-vip flap that killed `vmsp-platform-extra` package deployment 5 hours in.
2. `small` **(Simple model) fits with ~4 GB headroom.** One 96 GB VCFA VM on a 92 GB nested ESXi works only if that nested ESXi has minimal other workload. Place it on the nested ESXi with the most free RAM at deploy time (DRS Partially Automated will do this automatically post-2026-05-07).
3. **vCPU reduction is the supported homelab tuning knob.** If 24 vCPU is starving the host, edit the VCFA VM to 16 vCPU after Fleet creates it but before first power-on. Watch the deploy log for the clone task and intercept.
4. **Memory reduction is unsupported.** Do not edit the VCFA VM's RAM down from 96 GB. Pods will start, then OOMKill on Postgres or Harbor when their working set grows.
5. **Until physical host RAM exceeds ~512 GB, the HA model (** `medium` **/** `large` **) should not be attempted on this lab.** This rules out HA tests at the VCFA layer until hardware upgrade.

**Concrete preventive checklist for VCFA deploys on this lab:**

- [ ] vSAN catalog mkdir test passes ( `mkdir /vmfs/volumes/<datastore>/test-probe` succeeds and probe is cleaned up via `objtool delete` )
- [ ] Failed env from prior attempt (if any) cleaned via Section 12 cascade DELETE
- [ ] `selectedDeploymentType: small` and `deployOption: small` in `/tmp/vra-deploy.json`
- [ ] `ipPool` size matches deployment model. **API spec (VcfAutomationSpec) requires** `minItems: 2, maxItems: 4` **.** For Simple/small, 2 IPs is the minimum (this lab uses `192.168.1.96-192.168.1.98` for headroom). For HA/medium and HA/large, 4 IPs is the maximum.
- [ ] DRS automation level is **Partially Automated** (load-aware initial placement) or higher; Manual mode caused the 2026-05-07 medium failure by piling all 3 VMs on esxi04
- [ ] At least one nested ESXi host has > 96 GB free physical memory (typically esxi02 or esxi03 with vCenter, sddc-manager, etc. evicted to dedicated hosts)
- [ ] **No co-tenant VMs on the host the VCFA appliance lands on.** vCenter sharing esxi04 with the VCFA appliance caused the 2026-05-07 4th attempt to fail at `vmsp-platform-extra` — combined memory pressure during heavy package waves caused the workload K8s API VIP to flap. After vMotioning vCenter off esxi04 (post-attempt-4), the 5th attempt progressed past every prior failure point. See §11.7.11.
- [ ] **Do NOT vMotion any VM during an active VCFA deploy.** vMotion of a 24 GB working-set VM (e.g. vCenter) on the same overcommitted source host as the active VCFA appliance creates sustained memory-transfer pressure that destabilizes the workload cluster's kube-vip. The 4th attempt's failure at `vmsp-platform-extra` happened DURING a vCenter vMotion off esxi04 — the source host's memory copy starved the VCFA VM's kubelet enough to drop the VIP. **Pre-stage host placement BEFORE submitting the deploy.**
- [ ] `vrlcm-server` recently restarted (clears any lingering UI state)
- [ ] `vmware-certificatemanagement` service running on vCenter (NSX/cert-manager dependency)

### *11.7.11 The 5th Attempt — What Worked (May 7, 2026 ~7-9 PM)*

After 4 prior attempts (1 medium + 3 small/medium variants) all failed for different reasons, the 5th deploy attempt confirmed the full prerequisite stack:

**Pre-flight (in this exact order):** 1. Catalog fix from §11.7.8 (objtool delete + mkdir) — **verified to remain stable across 4 subsequent deploy attempts; the new catalog UUID** `81cbfc69-7e58-c68d-73d0-000c29649d0a` **did not get re-stuck** 2. DRS automation level set to **Partially Automated** so vSphere does load-aware initial placement of new VMs 3. **vMotion vCenter off esxi04** (the host where DRS will likely place the VCFA appliance, since esxi04 had the most free RAM and no other VCF tenants). vMotion took ~18 minutes for 24 GB working set on overcommitted

source. **Done BEFORE deploy POST — not during.** 4. Section 12 DB cleanup of any prior failed env 5. `vra-deploy.json` reconciled with current locker UUIDs (vcUsername, vcPassword, productPassword, certificate — rotated suffixes change after every reconnect cycle, see `feedback_validate_credentials_before_deploy.md` ) 6. JSON values: `deployOption=small` , `network=pg-vm-mgmt` (bare, NOT `vcenter-cl01-vds01-pg-vm-mgmt` ), `ipPool=192.168.1.96-192.168.1.98`

**Stage timing on 5th attempt (envId `601ea82d-1f72-4d8e-848a-125265ad1aa7` ):**

| Stage | Elapsed (from POST) | Duration | Note |
|---|---|---|---|
| POST submitted | 0 | — | requestId `7405ad8b-78a2-470a-ad8e-84d121344814` |
| Pre-OVA (auth, locker, network validation, cert generation) | 0 → ~25 min | ~25 min | |
| OVA deploy + first boot of VCFA appliance | 25 min → ~50 min | ~25 min | |
| Bootstrap-machine init + image+chart push to local OCI registry | 50 min → ~5h22m | hours | Heavy disk I/O writing all bundle artifacts |
| Helm install loop on bootstrap (cert-manager, trust-manager, CAPI, ...) | 5h22m → ~5h45m | ~25 min | |
| Workload K8s cluster `vcf-mgmt-90ad4c7701` provisioned + kubeconfig | 5h22m → 5h22m | seconds | Cluster object reaches Ready immediately on single-node |
| `vmsp-platform-bootstrap` PackageDeployment | 5h38m → 5h46m | **8 min** | Was 16 min on medium with kind cluster |
| `vmsp-platform-init` PackageDeployment | 5h46m → ~5h57m | **~10 min** | Was ~30 min on medium |
| `vmsp-platform-core-app-not-registry` (seaweedfs) | ~5h57m → ~6h0m | **~3-5 min** | Was 46 min with 1 retry on medium; KB 382405 fix held |
| `vmsp-platform-extra` (istio + kube-prometheus-stack + descheduler) | 6h0m → **in progress** | tbd | Medium FAILED here at 30+30+30 min retries |

**Comparison vs failed runs:**

| | Yesterday's medium (failed at 5h13m) | Today's small 5th attempt |
|---|---|---|
| Memory Rate (swap I/O) during heavy package wave | high (paging hot data) | **near zero** (vSphere uses TPS+compression instead) |
| Active memory pattern | sustained at high | **spike + release** with each install burst |
| kube-vip VIP availability | flapping; eventually lost | **single node, no kube-vip needed** |
| Workload K8s API reachability | dropping during installs | continuous |
| Per-stage time | 16-46 min, often retries | 3-10 min, no retries |

**Verified 2026-05-07 21:24 EDT:** `vmsp-platform-extra` PackageDeployment created at 01:03 UTC; deploy continues past every prior failure mode without retry signals.

### *11.7.12 The 5th Attempt Failed Late — `VmspPkgPushTask` / `LCMVMSP10002` (May 8, 2026 ~00:43 EDT)*

The 5th deploy attempt — which was carrying every prior fix — got past every previously-known failure mode (catalog, kube-vip, vmsp-platform-extra) but **failed at 00:43 EDT** at the very last step before VCFA app install:

```
 Error Code:    LCMVMSP10002 / LCM_VMSP_PKG_PUSH_ERROR_0001
Stack:         VmspPkgPushTask.execute(VmspPkgPushTask.java:78)
Cause:         DeploymentFailedException: No failed pods found.
Wall time:     6h13m after POST
Total runtime: longest of any attempt to date
```

**Why this is a DIFFERENT failure from prior attempts:**

| Prior attempts | This attempt |
|---|---|
| Failed during VMSP **bootstrap** (catalog, vmsp-platform-extra) | Failed AFTER VMSP bootstrap completed cleanly |
| Bootstrap log showed clear FAIL signal | Bootstrap log shows `Cluster provisioning completed.` (success) |
| VCFA appliance VM was **rolled back** by Fleet | Appliance stayed alive (Fleet did NOT auto-rollback on this failure mode) |
| All HelmReleases ON the cluster were unfinished | All ~25 HelmReleases SUCCEEDED before failure |
| Cluster VIP unreachable | Cluster VIP reachable; cert correctly bound to `automation-node1.lab.local` |

**What `VmspPkgPushTask` is doing when it fails (per Broadcom KB 434702 + Fleet log evidence):**

After VMSP cluster is up, Fleet runs:

```
/usr/local/bin/vmsp pkg push registry.vmsp-platform.svc.cluster.local:5000 \
  https://lcm.lab.local/repo/productBinariesRepo/vra/9.0.1.0/install/vra.tar \
  --deploy -n prelude --set vcfa.fqdn=automation-node1.lab.local \
  --wait --timeout=120m --remote --set deployment.size=small
```

This **pushes the VCFA application bundle** into the `prelude` namespace. The bundle contains the actual VCFA application components (Postgres, tenant-manager, vco-app, blueprinting, catalog, CCS) that go ON TOP of the VMSP platform.

If the push fails or the bundle never reaches Ready in 120 minutes, Fleet times out, runs an introspection looking for "failed pods" (gets "No failed pods found" because no pods are failing — the bundle never created pods), and declares the env FAILED.

**Verified root cause for this lab (2026-05-08):**

The VCFA appliance's `systemd-resolved` (loopback stub at `127.0.0.53`) was broken — could not forward DNS queries to the configured nameserver (`192.168.1.230`) even though:

- The nameserver was reachable from the appliance via TCP
- Direct `nslookup lcm.lab.local 192.168.1.230` (bypassing systemd-resolved) returned the correct A record
- TCP to Fleet on port 443 was open from the appliance
- `resolvectl status` showed correct config

But `resolvectl query lcm.lab.local` returned `resolve call failed: All attempts to contact name servers or networks failed`. Restarting `systemd-resolved` did NOT fix it. The `vmsp pkg push` command, which uses the system resolver, returned `HTTP 000` on the bundle URL (no DNS resolution).

**Suspected interaction:** Antrea CNI or the K8s networking on the appliance interfering with the loopback resolver stub. The break manifests AFTER the cluster comes up (during the VCFA bundle pull window). Permanent root cause not yet determined.

**Verified workaround — add `/etc/hosts` entries on the VCFA appliance:**

```
 ## SSH password auth is DISABLED on the VCFA appliance.
## Access via vCenter Guest Operations API (PowerCLI Invoke-VMScript) instead:
##   $cred = New-Object System.Management.Automation.PSCredential('vmware-system-user', $secpw)
##   Invoke-VMScript -VM $vm -ScriptText '<bash>' -GuestCredential $cred -ScriptType Bash

sudo bash -c 'cat >> /etc/hosts << EOF
192.168.1.95   lcm.lab.local fleet.lab.local
192.168.1.69   vcenter.lab.local
192.168.1.91   automation-node1.lab.local
192.168.1.71   nsx-manager.lab.local
192.168.1.77   vcf-ops.lab.local
192.168.1.230  sddc-manager.lab.local
EOF'
```

After this, `curl -sk -I https://lcm.lab.local/repo/productBinariesRepo/vra/9.0.1.0/install/vra.tar` returns **HTTP 200**, confirming the bundle is reachable through `/etc/hosts` resolution.

**Recovery: click Retry in Fleet UI.** Per all 5 cited Broadcom KBs (406528, 411354, 410338, 417145, 434702), the prescribed recovery for `LCMVMSP10002` is fix-input-then-retry; Fleet's retry button reuses the existing VMSP cluster (no rebuild needed). Path: vcf-ops → Fleet Management → Lifecycle → VCF Management → Tasks → failed env's Retry.

**Diagnostic procedure for next time** (in priority order):

1. Read `/var/log/vrlcm/vmware_vrlcm.log` on Fleet for the actual `vmsp pkg push` command + exit code. Look for the `lcm.lab.local` URL.
2. From the appliance, run: `curl -sk --max-time 10 -o /dev/null -w "HTTP %{http_code}\n" https://lcm.lab.local/repo/productBinariesRepo/`
3. If HTTP 000 (connection failed): test DNS via both `nslookup lcm.lab.local 192.168.1.230` (direct) AND `resolvectl query lcm.lab.local` (systemd-resolved).
4. If nslookup succeeds but resolvectl fails: confirmed systemd-resolved break. Apply `/etc/hosts` workaround.
5. Cross-reference Broadcom KBs above for the secondary failure modes (uppercase FQDN, cert SAN, HTTPRoute) if DNS is fine.

**Cross-references:**

- Broadcom KB 434702 — vcfa-bundle stuck Progressing, `prelude` namespace
- Broadcom KB 417145 — LCMVMSP10037 / LCM_VMSP_PKG_PUSH_ERROR_0001 (related)
- Broadcom KB 410338 — Cert FQDN/shortname mismatch causing `LCMVMSP10002`
- Broadcom KB 411354 — Cert SAN mismatch
- Broadcom KB 406528 — Uppercase FQDN HTTPRoute regex rejection
- Tom Fojta — VCFA `prelude` namespace contents
- Internal memory: `reference_vcfa_systemd_resolved_break.md`

---

### *11.7.13 6th Attempt -- Trust Manager Fix, vSAN Misclassification Discovery, and `unexpected EOF` Root Cause (May 8, 2026 PM)*

The 5th attempt's `LCMVMSP10002` was resolved by the `/etc/hosts` workaround on the appliance (Section 11.7.12). The **6th attempt** ran into a sequence of distinct failures, each of which uncovered a separate latent issue in the lab environment. Each finding below is documented with the **exact diagnostic command, the system it must be executed from, and the output to expect**.

#### 11.7.13.1 First failure on 6th attempt: TLS verify on Fleet's self-signed cert

**Symptom:** `vmsp pkg push` failed with `LCMVMSP10002`. Bundle CR `vra` in `prelude` namespace flipped to `Failed` within ~30 seconds. Bundle controller log showed `failed to download: unexpected EOF` (TLS handshake error code 18).

**Diagnostic 1 -- read Bundle CR phase and event from the workload K8s cluster.**

- **Run from:** `vcfa-mgmt-XXXXX` appliance, root shell.
- **Why this appliance:** It is the workload K8s control plane. SSH password auth is **disabled**. Use vCenter Guest Operations API (PowerCLI `Invoke-VMScript`) from any host that can reach vCenter, with `vmware-system-user` as the guest credential.
- **Path to kubectl on the appliance:** `/tmp/kubectl --kubeconfig /etc/kubernetes/admin.conf`.

```
 ## inside Invoke-VMScript -ScriptType Bash on the vcfa-mgmt VM:
KCT=/tmp/kubectl
KC=/etc/kubernetes/admin.conf
sudo $KCT --kubeconfig $KC get bundle -A
```

**Expected (failure mode 1 -- TLS verify):**

```
 NAMESPACE   NAME   AGE   PHASE    STATUS
prelude     vra    35s   Failed   failed to download: unexpected EOF
```

**Diagnostic 2 -- confirm TLS error class on the bundle controller.**

```
 sudo $KCT --kubeconfig $KC logs -n vmsp-platform deployment/vmsp-operator --tail=200 \
  | grep -iE "x509|certificate|verify|tls"
```

**Expected for TLS-verify failure:** lines containing `tls: failed to verify certificate: x509: certificate signed by unknown authority`. Absence of such lines rules out TLS as the cause; move to Section 11.7.13.2.

**Fix -- inject Fleet's self-signed CA into the workload cluster's `platform-trust` Trust Manager Bundle.**

- **Run from:** the same appliance shell.
- **Why this works:** vmsp-operator validates upstream TLS using the `platform-trust` ConfigMap that Trust Manager auto-renders from `Bundle/vmsp-platform/platform-trust`. Adding a Secret in `vmsp-platform` with label `trust.vmsp.vmware.com/bundle: platform-trust` and `data.ca.crt = <CA PEM>` causes Trust Manager to merge it into every namespace's trust ConfigMap.

```
 ## 1. Pull Fleet's CA from its own port 443 (Fleet IP = 192.168.1.95):
openssl s_client -connect lcm.lab.local:443 -servername lcm.lab.local </dev/null 2>/dev/null \
  | sed -n '/-----BEGIN CERTIFICATE-----/,/-----END CERTIFICATE-----/p' > /tmp/fleet-ca.pem

## 2. Verify it's the right cert (subject CN should be lcm.lab.local):
openssl x509 -in /tmp/fleet-ca.pem -noout -subject -issuer

## 3. Create the labeled Secret in vmsp-platform:
sudo $KCT --kubeconfig $KC create secret generic fleet-locker-ca \
  -n vmsp-platform \
  --from-file=ca.crt=/tmp/fleet-ca.pem
sudo $KCT --kubeconfig $KC label secret fleet-locker-ca \
  -n vmsp-platform trust.vmsp.vmware.com/bundle=platform-trust

## 4. Verify Trust Manager picked it up (Bundle status should show Synced):
```

```
sudo $KCT --kubeconfig $KC get bundle.trust.cert-manager.io platform-trust -o yaml \
  | grep -A5 "status:"
```

**Expected after step 4:**

```
status:
  conditions:
  - reason: Synced
    status: "True"
    type: Synced
  defaultCAVersion: "..."
  syncedCAs: <N+1>
```

(The `syncedCAs` count increases by 1 after the new Secret is added.)

**Then restart vmsp-operator and delete the failed Bundle CR for a clean retry:**

```
sudo $KCT --kubeconfig $KC rollout restart deployment vmsp-operator -n vmsp-platform
sudo $KCT --kubeconfig $KC delete bundle vra -n prelude
```

Click **Retry** in Fleet UI. The new Bundle CR's controller will pick up the merged trust on its first reconcile.

#### 11.7.13.2 Second failure on 6th attempt: long-duration `unexpected EOF` mid-download

**Symptom (post-trust-fix):** Bundle CR `vra` reaches `Progressing` cleanly but flips to `Failed` after **~26-30 minutes** of download with `failed to download: unexpected EOF`. Bundle download is reading from `https://lcm.lab.local/repo/productBinariesRepo/vra/9.0.1.0/install/vra.tar` (~3.2 GB).

**Diagnostic 1 -- check Fleet's nginx access log for completed bundle GETs.**

- **Run from:** Fleet appliance, root shell (192.168.1.95).
- **Connect via:** `ssh root@192.168.1.95` from any workstation.

```
grep '/repo/productBinariesRepo' /var/log/nginx/access.log | tail -10
```

**Expected during the failure pattern:**

```
192.168.1.96 - - [08/May/2026:17:51:04 +0000] "GET /repo/productBinariesRepo/vra/9.0.1.0/install/vra.tar
HTTP/1.1" 200 3227499424 "-" "Go-http-client/1.1"
192.168.1.96 - - [08/May/2026:18:17:07 +0000] "GET /repo/productBinariesRepo/vra/9.0.1.0/install/vra.tar
HTTP/1.1" 200 3227518415 "-" "Go-http-client/1.1"
```

**Note:** status `200` and body bytes matching the file size. Each request runs ~26-30 minutes. nginx considers this a successful response from its perspective.

**Diagnostic 2 -- check Fleet's nginx error log to find the actual failure on upstream side.**

```
grep -E 'EOF|timeout|upstream|reset' /var/log/nginx/error.log | tail -10
```

**Expected during the failure pattern:**

```
2026/05/08 18:17:07 [error] 1171#0: *7970 readv() failed (104: Connection reset by peer)
  while reading upstream, client: 192.168.1.96, server: lcm.lab.local,
  request: "GET /repo/productBinariesRepo/vra/9.0.1.0/install/vra.tar HTTP/1.1",
```

```
  upstream: "http://127.0.0.1:8080/...",
  host: "lcm.lab.local"
```

**Interpretation:** The `upstream: http://127.0.0.1:8080` (the local vrlcm Java backend) is sending TCP RST to nginx mid-read. The bundle controller (downstream) sees `unexpected EOF` because the chunked HTTP body terminates abruptly.

**Diagnostic 3 -- read Fleet's actual nginx config for the `/repo/` location block.**

```
 grep -nE "proxy_read_timeout|proxy_send_timeout|keepalive_timeout|client_max_body_size"
/etc/nginx/nginx.conf
```

**Expected (default config that triggers EOF):**

```
 26:  client_max_body_size 10240M;
35:  keepalive_timeout  65;
72:      proxy_read_timeout 15m; # /lcm/lcops/api/v2/settings/binaries/
73:      proxy_send_timeout 120m; # /lcm/lcops/api/v2/settings/binaries/
[...]
197:      proxy_read_timeout 120m; # /lcm/crepo/api/content/upload/
198:      proxy_send_timeout 120m; # /lcm/crepo/api/content/upload/
```

**The `/repo/` location block has NO `proxy_read_timeout` or `proxy_send_timeout` overrides**, so nginx falls back to its built-in defaults (60 seconds each). A multi-GB download under any vSAN write back-pressure on the receiving cluster crosses 60 seconds of upstream-read silence and nginx kills the proxy connection.

**Fix -- bump `/repo/` proxy timeouts in `/etc/nginx/nginx.conf`.**

```
 ## Always back up first:
cp /etc/nginx/nginx.conf /etc/nginx/nginx.conf.bak-$(date +%Y%m%d-%H%M%S)

## Apply the edit (atomic Python rewrite of the location block):
python3 - << 'PYEOF'
p = "/etc/nginx/nginx.conf"
s = open(p).read()
old = "    location /repo/ {\n      proxy_pass http://vrlcm-server;\n      proxy_max_temp_file_size
3072m;\n      proxy_temp_path /data/temp/;\n    }"
new = "    location /repo/ {\n      proxy_pass http://vrlcm-server;\n      proxy_read_timeout 120m;\n
proxy_send_timeout 120m;\n      proxy_max_temp_file_size 3072m;\n      proxy_temp_path /data/temp/;\n
}"
if old not in s:
    raise SystemExit("ANCHOR NOT FOUND -- manual edit required")
open(p, "w").write(s.replace(old, new, 1))
print("EDIT_OK")
PYEOF

## Validate config syntax:
nginx -t

## Graceful reload (existing connections keep old config; new connections get bumped timeouts):
nginx -s reload

## Verify nginx workers are still running:
ps -ef | grep nginx | grep -v grep
```

**Expected after `nginx -t`:**

```
 nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
```

**Expected after `nginx -s reload`:** silent return; the master nginx process PID is preserved. New `/repo/` requests now have a 120-minute read/send timeout.

**Important caveat:** The nginx-side fix prevents nginx from giving up at 60 s of upstream silence, but it does NOT change behavior of the **vrlcm Java backend** itself. If the Java app has its own connection lifetime cap, the upstream RST will still occur after that duration. Section 11.7.13.4 below documents the deeper Java-side observation.

#### 11.7.13.3 Third finding on 6th attempt: vSAN capacity disks misclassified as HDD on this homelab

**Symptom in vCenter:** During the 6th attempt's slow downloads, host-perspective vSAN performance graphs (vCenter > Cluster > Monitor > vSAN > Performance > Hosts > VM Consumption) showed:

- Read Latency ~26 ms
- **Write Latency 70 ms** (with peaks to 840 ms)
- **Local Client Cache Hit Rate 0%**
- IOPS 30-100 sustained, throughput 4-6 MB/s

These are HDD-tier numbers on a host where the user knows the underlying physical media is **all NVMe / SSD**. The bundle download bottleneck is real but media is not the cause -- the cause is that vSAN's media-type flag is **wrong** on several disks.

**Diagnostic -- enumerate per-host disk groups and check the `Ssd` flag on each capacity disk.**

- **Run from:** any host with PowerCLI 13+ installed and network access to vCenter (the lab's Windows workstation, in this environment).
- **Why PowerCLI:** the vSAN view object exposes the per-disk `Ssd` boolean directly. Equivalent ESXi-side commands exist ( `esxcli vsan storage list` ) but require SSH-to-host plus parsing.

```
Connect-VIServer vcenter.lab.local -User 'administrator@vsphere.local' -Password '...'

foreach ($h in (Get-Cluster vcenter-cl01 | Get-VMHost)) {
  Write-Host "--- $($h.Name) ---"
  $sys = Get-View -Id $h.ExtensionData.ConfigManager.VsanSystem
  $sys.Config.StorageInfo.DiskMapping | ForEach-Object {
    Write-Host ("  Cache:    {0} - {1} GB - SSD={2}" -f `
      $_.Ssd.CanonicalName,
      [math]::Round($_.Ssd.Capacity.Block * $_.Ssd.Capacity.BlockSize / 1GB, 1),
      $_.Ssd.Ssd)
    foreach ($cap in $_.NonSsd) {
      Write-Host ("  Capacity: {0} - {1} GB - SSD={2}" -f `
        $cap.CanonicalName,
        [math]::Round($cap.Capacity.Block * $cap.Capacity.BlockSize / 1GB, 1),
        $cap.Ssd)
    }
  }
}
```

**Expected on a healthy all-flash OSA cluster:** every line ends `SSD=True`.

**Observed on this lab (2026-05-08):**

```
 --- esxi01.lab.local ---
  Cache:    ...000000000001 - 100 GB - SSD=True
  Capacity: ...000000000001 - 500 GB - SSD=True
  Capacity: ...000000000001 - 400 GB - SSD=True
  Capacity: ...000000000001 - 100 GB - SSD=True
--- esxi02.lab.local ---
  Cache:    ...000000000001 - 100 GB - SSD=True
  Capacity: ...000000000001 - 500 GB - SSD=True
```

```
  Capacity: ...000000000001 - 400 GB - SSD=False
  Capacity: ...000000000001 - 100 GB - SSD=False
--- esxi03.lab.local ---
  Cache:    ...000000000001 - 100 GB - SSD=True
  Capacity: ...000000000001 - 500 GB - SSD=True
  Capacity: ...000000000001 - 100 GB - SSD=False
--- esxi04.lab.local ---
  Cache:    ...000000000001 - 100 GB - SSD=True
  Capacity: ...000000000001 - 500 GB - SSD=True
  Capacity: ...000000000001 - 400 GB - SSD=False
  Capacity: ...000000000001 - 100 GB - SSD=True
```

Four capacity disks across three hosts report `SSD=False` despite physical NVMe backing. **vSAN OSA puts those disk groups into hybrid mode for those disks**, which forces all writes to flush to "HDD" capacity tier first -- exactly explaining the 70 ms write latency and 0% Local Client Cache Hit Rate.

**Why this happens on nested ESXi:** VMware Workstation virtual disks do not propagate `is_ssd=true` to the guest. When the inner ESXi host claims those disks for vSAN, vSAN auto-detects them as HDD. This is **independent of the underlying physical media** on the workstation.

**Fix -- mark each affected disk as flash, then rebuild the disk group.**

- **Run from:** SSH to each inner ESXi host (or use `esxcli` via vSphere CLI). For this lab: `ssh root@esxi02.lab.local`, etc. Passwords per `reference_homelab_esxi_creds.md`.
- **Do NOT run during an in-flight VCFA deploy.** Disk-group rebuild evacuates objects through the cluster network and disrupts every VM on the cluster.

```
 ## 1. List disks and current capacityFlash status on the host:
esxcli vsan storage list | grep -E "Device|Is SSD|Is Capacity|UUID"

## 2. Tag each misclassified disk:
esxcli vsan storage tag add -d <naa.xxxxxxxxxxxxxxx> -t capacityFlash

## 3. Confirm the tag is recorded:
vdq -q -d <naa.xxxxxxxxxxxxxxx> | grep -E "IsCapacityFlash|IsSSD"
## Expected:  "IsCapacityFlash": 1, "IsSSD": 0  (the SSD flag is set by vmkernel autodiscovery,
##            but capacityFlash is what vSAN OSA uses for tier selection)

## 4. Disk-group rebuild path (per cluster):
##    a) Place the host into Maintenance Mode with **Full Data Migration** (NEVER Ensure Accessibility).
##    b) From vCenter > Cluster > Configure > Disk Management, remove the disk group on that host.
##    c) Re-create the disk group; the previously-tagged disks now claim into the capacity tier as flash.
##    d) Exit Maintenance Mode; vSAN resyncs.
## Repeat per host until every Capacity row shows SSD=True.
```

**Why we left this alone in this incident:** Repair-during-deploy would have killed the in-flight bundle copy. The 6th attempt was permitted to continue at HDD-equivalent speeds; the fix is queued for a maintenance window.

#### 11.7.13.4 Bundle controller retry pattern and Fleet `vmsp pkg push` interaction

**Bundle controller behavior (verified 2026-05-08):**

After Bundle phase flips to `Failed`, `bundle_controller` (a logger inside the `vmsp-operator` deployment, not a separate Pod) immediately enqueues another reconcile and starts pulling again. controller-runtime workqueue uses exponential rate-limiting capped at ~16-17 minutes between retries.

**Diagnostic -- watch retry timing on the workload cluster.**

```
 ## (vcfa-mgmt appliance shell, via Invoke-VMScript)
sudo $KCT --kubeconfig $KC logs -n vmsp-platform deployment/vmsp-operator --tail=500 \
  | grep -iE "bundle_controller|reconciling.*bundle|pulling packages|EOF"
```

**Expected pattern (each pull attempt ~26-30 min on this lab):**

```
 17:20:55  bundle_controller  reconciling bundle  bundle: vra
17:20:55  bundle_controller  pulling packages    bundle: vra
17:51:08  bundle_controller  failed to process packages  error: unexpected EOF
17:51:08  bundle_controller  reconciling bundle              ← immediate retry
17:51:08  bundle_controller  pulling packages
18:17:13  bundle_controller  failed to process packages  error: unexpected EOF
18:17:13  bundle_controller  reconciling bundle              ← retry again
18:17:13  bundle_controller  pulling packages
```

**Fleet's `vmsp pkg push` behavior:** the pkg push command runs with `--wait --timeout=120m`. It does NOT exit on Bundle phase= `Failed` -- the controller-runtime in-cluster will keep retrying, so pkg push continues waiting up to its 120-minute deadline.

**Diagnostic -- find the running pkg push process on Fleet to confirm what `--timeout` was passed.**

```
 ## Fleet appliance shell (192.168.1.95):
ps -ef | grep -E 'vmsp.*pkg' | grep -v grep
```

**Expected example:**

```
root  1137610  PID  17:20  /usr/local/bin/vmsp pkg push registry.vmsp-platform.svc.cluster.local:5000 \
  https://lcm.lab.local/repo/productBinariesRepo/vra/9.0.1.0/install/vra.tar \
  --deploy -n prelude --set vcfa.fqdn=automation-node1.lab.local \
  --wait --timeout=120m --remote --set deployment.size=small --kubeconfig=/dev/stdin
```

**Diagnostic -- Fleet env state polling, useful as a single-line summary.**

- **Run from:** Fleet appliance.
- **Why this script exists:** it formats the Fleet `/lcm/request/api/v2/requests/<id>` response into a parseable line for monitor scripts.

```
 ## Pre-existing on Fleet at /root/vra-poll.sh:
RID='7405ad8b-78a2-470a-ad8e-84d121344814'
RESP=$(curl -sk -u "$(cat /tmp/auth.txt)" "https://localhost/lcm/request/api/v2/requests/$RID")
echo "$RESP" | python3 -c '
import sys,json,re
raw=sys.stdin.read()
d=json.loads(raw)
state=d.get("state","?")
rr=d.get("requestReason","")
m=re.search(r"\(([0-9a-f-]{36})\)", rr)
envid=m.group(1) if m else ""
err=str(d.get("errorCause","") or "")[:400]
print(f"{state}|{envid}|{err}")
'
```

**Expected output formats:**

```
 INPROGRESS|601ea82d-1f72-4d8e-848a-125265ad1aa7|<errorCause if any prior fail>
COMPLETED|601ea82d-1f72-4d8e-848a-125265ad1aa7|
```

```
FAILED|601ea82d-1f72-4d8e-848a-125265ad1aa7|[{"messageId":"LCMVMSP10002",...}]
```

#### 11.7.13.5 Internal registry cache survives between attempts

**Useful fact for retry planning:** the in-cluster `registry` Pod (the docker registry sitting on top of seaweedfs in `vmsp-platform`) **retains pushed layers** across `vmsp delete ns prelude --force` cycles. Subsequent retries' imgpkg-copy phase will skip layers already present, dramatically shortening successful pushes after the first attempt that pushes anything.

**Diagnostic -- registry size growth between attempts.**

```
 ## (vcfa-mgmt appliance shell, via Invoke-VMScript)
sudo $KCT --kubeconfig $KC exec -n vmsp-platform deployment/registry -- du -sh /var/lib/registry
```

**Expected growth pattern across attempts:**

```
 attempt 1 first push:    1.6 GB
attempt 1 final push:    4.1 GB
attempt 2 (warm cache):  4.1 GB at start, grows further if pushing additional layers
```

If size is `0` on a cold cluster, no layers have been pushed yet. If size is non-zero on a fresh attempt, the cache is warm and the push phase will be fast.

#### 11.7.13.6 Cross-references for Section 11.7.13

- Broadcom KB 410338 -- LCMVMSP10002 deadline exceeded (cert/FQDN angle, not nginx)
- Broadcom KB 406528 -- LCMVMSP10002 uppercase FQDN HTTPRoute regex
- Broadcom KB 411354 -- Cert SAN mismatch
- Broadcom KB 415800 -- VCF 9 known issues (does NOT cover this nginx pattern as of 2026-05-08)
- Broadcom KB 317004 -- "Unexpected EOF" pulling from VIC registry (different product, similar symptom)
- Internal memory: `reference_homelab_vsan_misclassified_capacity.md`
- Internal memory: `reference_vcfa_systemd_resolved_break.md`
- Internal memory: `reference_vsan_osa_allflash_grandfather.md`

---

## 12. Complete LCM Database Cleanup Reference

### *12.1 Table Dependency Tree*

When LCM gets stuck with orphaned deployment records, you need to clean the database tables in the correct order. Foreign key constraints mean you must delete children before parents -- like emptying the files from a folder before you can throw the folder away.

```
 vm_lcops_environments (parent -- delete LAST)
  +-- vm_lcops_infrastructure_properties (child)
  +-- vm_lcops_product (child)
        +-- vm_lcops_product_properties (grandchild)
        +-- vm_lcops_product_nodes (grandchild)
              +-- vm_lcops_node_properties (great-grandchild)

vm_rs_request (independent -- update stuck entries to FAILED)
vm_engine_execution_request (independent -- update stuck entries to FAILED)
```

**Important:** Check for ALL product IDs when investigating. The UI creates entries under multiple product IDs: - `vra` - - VCF Automation - `vmsp` -- VMSP platform (a separate product ID from `vra`, but related to Automation) - `vidb` -- Identity Broker - `vrni` -- Operations for Networks

**Important:** Check for ALL statuses, not just `UI_INPROGRESS`: - `UI_INPROGRESS` -- Created by UI page visits - `SUBMITTED` -- Created when submit was attempted - `INPROGRESS` -- Deployment started but never finished - `FAILED` -- Can sometimes still block new deployments if related environment records are not cleaned

### *12.2 Complete Cleanup Script*

> **Run this with `vrlcm-server` STOPPED.** If the service is running, it will recreate records you just deleted.

```
 #!/bin/bash
## Complete LCM Database Cleanup Script
## Run on the LCM appliance (lcm.lab.local / 192.168.1.95) as root

PSQL="/opt/vmware/vpostgres/14/bin/psql -U vrlcm -d vrlcm"

echo "=== Step 1: Stop LCM service ==="
systemctl stop vrlcm-server
sleep 5

echo "=== Step 2: Show current state ==="
$PSQL -c "SELECT environmentid, environmentstatus, status FROM vm_lcops_environments;"
$PSQL -c "SELECT vmid, productid, syncedenvironmentstatus FROM vm_lcops_product;"

echo "=== Step 3: Delete deepest children first (node properties) ==="
$PSQL -c "DELETE FROM vm_lcops_node_properties WHERE node_vmid IN (
  SELECT vmid FROM vm_lcops_product_nodes WHERE product_vmid IN (
    SELECT vmid FROM vm_lcops_product WHERE environment_vmid IN (
      SELECT vmid FROM vm_lcops_environments WHERE status != 'COMPLETED')));"

echo "=== Step 4: Delete product nodes ==="
$PSQL -c "DELETE FROM vm_lcops_product_nodes WHERE product_vmid IN (
  SELECT vmid FROM vm_lcops_product WHERE environment_vmid IN (
    SELECT vmid FROM vm_lcops_environments WHERE status != 'COMPLETED'));"

echo "=== Step 5: Delete product properties ==="
$PSQL -c "DELETE FROM vm_lcops_product_properties WHERE product_vmid IN (
  SELECT vmid FROM vm_lcops_product WHERE environment_vmid IN (
    SELECT vmid FROM vm_lcops_environments WHERE status != 'COMPLETED'));"

echo "=== Step 6: Delete products (catches vra, vmsp, vidb, vrni) ==="
$PSQL -c "DELETE FROM vm_lcops_product WHERE environment_vmid IN (
  SELECT vmid FROM vm_lcops_environments WHERE status != 'COMPLETED');"

echo "=== Step 7: Delete infrastructure properties ==="
$PSQL -c "DELETE FROM vm_lcops_infrastructure_properties WHERE environment_vmid IN (
  SELECT vmid FROM vm_lcops_environments WHERE status != 'COMPLETED');"

echo "=== Step 8: Delete orphaned environments (THE KEY STEP) ==="
$PSQL -c "DELETE FROM vm_lcops_environments WHERE status != 'COMPLETED';"

echo "=== Step 9: Clean independent tables ==="
$PSQL -c "UPDATE vm_rs_request SET state='FAILED'
  WHERE state NOT IN ('COMPLETED','FAILED');"
$PSQL -c "UPDATE vm_engine_execution_request SET enginestatus='FAILED'
  WHERE enginestatus NOT IN ('SUCCESS','FAILED','COMPLETED');"

echo "=== Step 10: Verify clean state ==="
$PSQL -c "SELECT environmentid, environmentstatus, status FROM vm_lcops_environments;"
```

```
$PSQL -c "SELECT vmid, productid, syncedenvironmentstatus FROM vm_lcops_product;"

echo "=== Step 11: Restart LCM ==="
systemctl start vrlcm-server

echo "=== Step 12: IMPORTANT -- Also restart VCF Operations web service ==="
echo "SSH into vcf-ops.lab.local (192.168.1.77) and run:"
echo "  systemctl restart vmware-vcops-web"
echo ""
echo "This clears cached deployment state in VCF Operations."
echo "Without this step, the UI will still show the parallel deployment error."
```

---

## 13. Lessons Learned

> **What is this section?** A consolidated summary of the most important lessons from all nine issues. Print this page and pin it to your wall before your next VCF deployment.

| # | Lesson | Details |
|---|---|---|
| 1 | Use Local Path depot, not webserver | A webserver depot introduces URL parsing errors, folder structure mismatches, and HTTPS certificate issues. Local Path reads directly from disk. (KB 432806) |
| 2 | Use embedded Identity Broker for single-instance labs | Standalone VIDB is only needed for multi-site environments. Embedded mode uses existing vCenter SSO. |
| 3 | Use API deployment when the UI is buggy | The LCM REST API ( `POST /lcm/lcops/api/v2/environments` ) bypasses all VCF Operations UI bugs including the "parallel deployment" block. |
| 4 | `ipPool` goes in `nodes[0].properties` | The most common API deployment error. `ipPool` , `primaryVip` , and `internalClusterCidr` must be in the node properties, not the product properties. |
| 5 | Always use UI Export Configuration | Before crafting an API payload manually, use the UI's "Export Configuration" button to capture the correct JSON structure with all locker references. |
| 6 | Do not delete -- set status to COMPLETED or FAILED | When cleaning up stuck records, prefer updating status to FAILED rather than deleting, to preserve binary download records and audit trails. |
| 7 | The `vmsp` product ID is separate from `vra` | The UI creates database entries under both `vra` AND `vmsp` product IDs. Database cleanup must check for both. |
| 8 | Always restart vmware-vcops-web after database cleanup | VCF Operations caches deployment state. Without restarting the web service, the UI continues to show stale errors. |
| 9 | VMware API units are inconsistent | `totalMemory` is in bytes, `effectiveMemory` is in megabytes. Always check the documentation for units. |
| 10 | Certificate SANs must include all nodes | VCF Automation needs cluster FQDN + all 3 node FQDNs + short names + all IPs in the certificate SAN. |

---

## 14. Current Status

**What is this section?** A snapshot of where things stand as of the most recent edit (May 8, 2026 PM).

| Item | Status | Details |
|---|---|---|
| VCF Automation 9.0.1.0 | **6th attempt in flight** | Bundle download phase looping on `unexpected EOF` at ~30-minute mark; root cause investigation under §11.7.13. Prior 5 attempts documented in §11.7.8 / §11.7.11 / §11.7.12 / §11.7.13. |
| Storage migration | **COMPLETE** | All four nested ESXi VMs run on SSD/NVMe. Issue 10 (HDD-backed etcd cliff) is fully resolved at the hardware level. |
| Issue 12 (vSAN catalog) | **RESOLVED** | KB 382405 `objtool delete -u <UUID>` of stuck `catalog` vmnamespace, then `mkdir` from owner host. See §11.7.8. Catalog verified healthy on each subsequent attempt. |
| Issue: VCFA appliance DNS | **WORKAROUND IN PLACE** | `systemd-resolved` on the appliance fails to resolve lab FQDNs even though DNS server is reachable. `/etc/hosts` entries on the appliance keep bundle pulls reachable. See §11.7.12. |
| Issue: Fleet self-signed cert | **RESOLVED** | Fleet's CA (CN=lcm.lab.local) injected into `platform-trust` Trust Manager Bundle via Secret in `vmsp-platform`. See §11.7.13.1. |
| Issue: nginx `/repo/` timeouts | **PATCHED** | `/etc/nginx/nginx.conf` `/repo/` location now has `proxy_read_timeout 120m` + `proxy_send_timeout 120m`. Backup at `/etc/nginx/nginx.conf.bak-20260508-183153`. See §11.7.13.2. |
| Issue: vSAN capacity SSD flag | **KNOWN, UNFIXED** | esxi02/03/04 each have ≥1 capacity disk reporting `SSD=False` on flash media; OSA runs those disk groups in hybrid mode (70 ms write latency, 0% Local Client Cache hit rate). Fix is `esxcli vsan storage tag add -t capacityFlash` + disk-group rebuild — **not safe during in-flight deploy.** See §11.7.13.3. |
| Issue: 30-min `unexpected EOF` cap | **OPEN -- root cause UNVERIFIED** | After the nginx patch the EOF still recurs at a consistent ~26-30 min mark across three consecutive controller retries (17:51, 18:17, 18:46). nginx error log shows `readv() failed (104: Connection reset by peer) while reading upstream, upstream: http://127.0.0.1:8080`, so the RST is initiated by the local vrlcm backend, not by nginx. **No published Broadcom KB covers this pattern (verified 2026-05-08 against KB 415800 + KB 410338 + KB 406528 + KB 411354 + KB 317004 + the VCF Automation 9.0 known-issues page).** Next-step options have not been validated — do NOT modify backend Java settings without a Broadcom support case number authorizing the change. |
| LCM Environment id | `601ea82d-1f72-4d8e-848a-125265ad1aa7` | The current 6th attempt. Fleet retries reuse the same envId; do NOT clean it from the database mid-loop. |

| Item | Status | Details |
|---|---|---|
| LCM requestId (current) | `7405ad8b-78a2-470a-ad8e-84d121344814` | Used by `/root/vra-poll.sh` to inspect state. |
| Workload K8s appliance | `vcfa-mgmt-wsv57` (192.168.1.96) | First VCFA appliance VM that survived bootstrap; on esxi04. |
| Workload K8s control plane VIP | `192.168.1.91` (the VCFA VIP) | Bound and reachable; cert SAN matches. |
| Internal registry cache | **4.1 GB cached** | Across all attempts; survives `vmsp delete ns prelude --force` cycles. Subsequent retries that reach the push phase will skip already-pushed layers. |
| esxi01 / esxi02 / esxi03 / esxi04 | Running, healthy | Cluster in normal operating state. |
| NSX Manager | Operational | At `nsx-manager.lab.local`. |
| Fleet Manager | Operational | LCM=Fleet appliance at 192.168.1.95 (lcm.lab.local). |
| VCF Operations | Operational | At vcf-ops.lab.local (192.168.1.77). |

**Next decision points (in priority order):**

1. **Watch the next bundle controller retry that started post-nginx-reload (18:46 onward).** If it also EOFs at the same ~30 min mark, the nginx-side fix was insufficient and the cause is on the vrlcm backend side. If it succeeds, the nginx fix landed.
2. **Do NOT modify vrlcm Java backend settings without a Broadcom support case authorizing the change.** No published KB documents a sanctioned timeout adjustment for `/repo/` downloads on the Fleet appliance.
3. **Defer:** vSAN OSA capacity-disk SSD-flag rebuild (§11.7.13.3) — perform during a maintenance window after the deploy completes (success or terminal failure). This removes the underlying write back-pressure that lengthens each bundle pull beyond the EOF window.

**Monitoring during the in-flight 6th attempt:**

```
 ## On the LCM appliance (192.168.1.95), state poll script:
/root/vra-poll.sh
## Output format:  STATE|envId|errorCause(first 400 chars)

## On the workload appliance (vcfa-mgmt-wsv57) via vCenter Guest Operations API:
sudo /tmp/kubectl --kubeconfig /etc/kubernetes/admin.conf get bundle -A
```

```
sudo /tmp/kubectl --kubeconfig /etc/kubernetes/admin.conf logs \
  -n vmsp-platform deployment/vmsp-operator --since=30m \
  | grep -iE "bundle_controller|reconciling.*bundle|pulling|EOF|Pushing"

## On Fleet (192.168.1.95):
ps -ef | grep 'vmsp.*pkg' | grep -v grep # is the pkg push command alive?
grep '/repo/productBinariesRepo' /var/log/nginx/access.log | tail -5 # completed pulls
grep -E 'EOF|reset|timeout' /var/log/nginx/error.log | tail -5 # the upstream RST events
```

---

## 15. Document Information

| Field | Value |
|---|---|
| Document Title | VCF 9.0.1 Aria Suite Deployment -- Root Cause Analysis & Troubleshooting Report |
| Prepared By | Virtual Control LLC |
| Date | April 2026 |
| Document Version | 4.6 (cover page bumped to 2.1) |
| Classification | Internal -- Lab Environment |
| VCF Version | 9.0.1 |
| Broadcom KBs Referenced | KB 388305 (Parallel Deployment Bug), KB 412322 (Certificate SAN), KB 432806 (Offline Depot), KB 382405 (CNS PODs / FCD catalog), KB 401698 (CnsFault VSLM task failed), KB 330164 (FCD catalog SPBM policy reconfigure), KB 414551 (Scale Up VCFA 9.0), KB 428195 (objtool delete syntax for orphaned vSAN objects) |
| Total Issues Documented | 12 |
| Most Critical Issues | Issue 7 (Parallel Deployment Block), Issue 9 (API Bypass), Issue 10 (HDD etcd cliff), Issue 11 (esxi01 drive saturation), Issue 12 (stuck vSAN FCD catalog namespace blocking all CSI provisioning — KB 382405) |
| Revision | v2.0 (April 16) -- Issues 1–9 resolved, deploy in progress. v3.0 (April 17) -- Added Issues 10 (VMSP etcd failure on HDD) and 11 (esxi01 drive saturation); marked deploy as failed + in recovery. v4.0 (May 7 AM) -- Added Issue 12 |

| Field | Value |
|---|---|
| History | (then-hypothesized as SeaweedFS pod startup failure on single-node K8s); added 2026-05-06 Deploy Run addendum with Fleet workflow stage map. v4.1 (May 7 PM) -- Issue 12 ROOT CAUSE VERIFIED: stuck vSAN FCD catalog `vmnamespace` (KB 382405) blocking ALL CSI volume provisioning. All prior hypotheses (sizing, anti-affinity, storage class) refuted. Added §11.7.8 with verified diagnostic + objtool delete fix and §11.7.9 future-deploy guidance. v4.2 (May 7 PM, second pass) -- Added §11.7.10 correcting the VCFA sizing claim. Per Broadcom KB 414551 + TechDocs + William Lam + Tom Fojta, VCFA has fixed sizing of 24 vCPU / 96 GB *per VM* in BOTH small and medium; the per-VM numbers in §§11.7.3-5 (4/16, 8/32) are VCF Operations sizes mistakenly applied to VCFA. Added warn-boxes flagging superseded sections. Documented why `medium` is physically incompatible with this homelab (each nested ESXi is 92 GB; one VCFA VM is 96 GB). v4.6 (May 8 PM, after 6th attempt diagnostics) -- Added §11.7.13 documenting the 6th attempt's three distinct failure layers with full per-command run-from / expected-output detail: (1) Trust Manager Bundle Secret injection for Fleet's self-signed cert (TLS verify EOF); (2) `/etc/nginx/nginx.conf` `/repo/` location was missing `proxy_read_timeout` / `proxy_send_timeout` overrides — bumped to 120m each (Broadcom does NOT publish this fix; verify before applying in other environments); (3) verified vSAN OSA capacity-disk misclassification ( `SSD=False` on flash media on esxi02/03/04) explaining the 70 ms write latency / 0% Local Client Cache hit rate observed in vCenter performance graphs; fix is `esxcli vsan storage tag add -t capacityFlash` + DG rebuild during a maintenance window (NOT during in-flight deploy). Rewrote §14 Current Status to reflect the all-flash storage reality and the 6th attempt's in-flight state. Removed the obsolete 2026-05-06 Deploy Run Addendum (superseded by 5 attempts since). Verified against Broadcom: KB 415800, KB 410338, KB 406528, KB 411354, KB 317004, KB 393159, and the VCFA 9.0 known-issues page do NOT cover the 30-min upstream-RST EOF pattern observed on this lab; the Java-backend timeout hypothesis is therefore explicitly flagged UNVERIFIED in §14, and the doc warns against modifying backend Java settings without a Broadcom support case. v4.5 (May 8 AM, after 5th attempt's late failure) -- Added §11.7.12 documenting the post-bootstrap `VmspPkgPushTask` / `LCMVMSP10002` failure that hit the 5th attempt at 00:43 EDT after the bootstrap had completed cleanly. Verified root cause: `systemd-resolved` on the VCFA appliance fails to resolve lab FQDNs (returns `All attempts to contact name servers or networks failed` ) even though the configured DNS server is reachable directly via `nslookup` . The vmsp pkg push command uses the system resolver, gets HTTP 000, the bundle download fails, the Bundle CR in `prelude` never reaches Ready, Fleet times out at the 120-minute mark and declares `LCMVMSP10002` . Documented: (a) the verbatim Fleet log evidence (vmsp pkg push command + exit code from `vmware_vrlcm.log` ); (b) the `/etc/hosts` workaround (verified to make HTTP 200 reachable on bundle URL); (c) the diagnostic procedure for next-time triage; (d) recovery path = click Retry in Fleet UI (per 5 Broadcom KBs). Added cross-references to KB 434702 / 417145 / 410338 / 411354 / 406528. Note: SSH password auth is DISABLED on the VCFA appliance; access requires vCenter Guest Operations API via PowerCLI `Invoke-VMScript` ( `vmware-system-user` /productPassword). v4.4 (May 7 PM, after 5 deploy attempts) -- Added §11.7.11 "The 5th Attempt -- What Worked." Documents: (a) the verified prerequisite stack (catalog fix + DRS Partially Automated + vCenter pre-vMotioned off esxi04 + Section 12 cleanup + reconciled locker UUIDs + bare network name + 3-IP pool); (b) the **vMotion-during-deploy lesson** (the 4th attempt failed at vmsp-platform-extra because we vMotioned vCenter off esxi04 mid-deploy, sustained memory-transfer pressure on the source host destabilized kube-vip); (c) stage-by-stage timing comparison showing small-on-right-sized-host runs each wave 3-5x faster than medium-on-overcommitted-host with no retries. Added "no co-tenant VMs on host" and "no vMotion during deploy" to preventive checklist. v4.3 (May 7 PM, third-pass accuracy verification against Broadcom) -- Six corrections to v4.2: (1) `large` **corrected from "5 VMs" to "3 VMs"** — per Broadcom TechDocs *VCF Automation Deployment Models*, large uses the same HA topology as medium (3 nodes); medium vs large differs in *per-VM allocation* via vertical scale-up, not VM count. (2) Removed the "horizontal scaling is the only knob" framing — both vertical and horizontal scaling exist (KB 414551). (3) Added explicit Simple-vs-HA model labeling to §11.7.10 sizing table. (4) `ipPool` checklist clarified per `VcfAutomationSpec` API: `minItems=2` , `maxItems=4` ; small needs 2 minimum, medium/large cap at 4. (5) §11.7.8 "find owner host" guidance corrected: read `owner=` from `cmmds-tool find -u <UUID>` , not the `Sub-Cluster Master UUID` (those are different concepts). (6) Softened the "state code i6 = DELETING" interpretation — no public Broadcom doc enumerates DOM_NAME state codes; flagged as engineering inference. (7) Added KB 428195 reference for the documented `objtool delete -u <UUID> -f` syntax (sanctioned for orphan snapshots; not for vmnamespaces). |

---

*This document was created based on real troubleshooting events in a VCF 9.0.1 nested lab environment. All IP addresses, hostnames, and error messages are from actual lab sessions. The procedures documented here have been verified to work but should always be tested in a non-production environment first.*

---

## 9. Failed-Deployment Cleanup Runbook

*Source: Failed-VCFA-Deployment-Cleanup-Runbook-2026-05-18.md*

**Prepared by:** Virtual Control LLC **Date:** May 18, 2026 **Document Version:** 1.0 **Classification:** Internal — Lab Environment **VCF Version:** 9.0.1.0 **Fleet:** vmlcm-service-9.0.1.0-SNAPSHOT **Cluster:** vcenter-cl01

**Scope.** This document is the cleanup runbook for VCFA environments that failed to deploy and left orphan state in Fleet's `vrlcm` database. Covers four cleanup methods in increasing order of aggressiveness, the **resource pool deletion gotcha** discovered 2026-05-18, and direct Postgres fallback when the API DELETE returns HTTP 200 but silently transitions to FAILED without telling you why.

**Companion docs.** - `Fleet-vCenter-Trust-Cascade-Repair-2026-05-18.md` — the trust state recovery - `VCFA-Simple-Deploy-Wizard-Precheck-Gotchas-2026-05-18.md` — precheck gotchas - `Nested-ESXi-RAM-Rebalance-for-VCFA-Deploy-2026-05-18.md` — host sizing - `Fleet-Operations-Integration-Recovery-RCA.md` — Appendix C FK tree this doc references

---

**Table of Contents**

1. When to Use This Runbook
2. Pre-Cleanup Checklist
3. Method 1 — API DELETE with deleteFromVcenter=false (cleanest)
4. Method 2 — API DELETE with deleteFromVcenter=true (and the resource-pool gotcha)
5. Method 3 — Manual UI cleanup + API DB-only DELETE
6. Method 4 — Direct Postgres FK-ordered DELETE
7. Method 5 — UI Retry with Skip Task (KB 416691)
8. When the API DELETE Silent-Fails
9. Verification Matrix
10. Rollback
11. Incident Log — 2026-05-18
12. Pre-Redeploy Gotchas (Do These Before Retrying)
13. Lessons Learned
14. Document Information

---

### 1. When to Use This Runbook

Use one of the methods below when any of these apply:

| Symptom | Cause |
|---|---|
| **Fleet UI Components page shows a VCFA env as `Deployment Failed` or `Draft`** that you want to remove | Failed deploy left orphan env record |

| Symptom | Cause |
|---|---|
| **Wizard refuses to start a new VCFA deploy** with "Parallel Deployments of automation and identity broker are not allowed" | Fleet enforces single in-flight VCFA env; orphan blocks new deploys |
| `GET /lcm/lcops/api/v2/environments` returns an extra VCFA env that should not exist | Fleet DB has stale env |
| Wizard's resource pool / cluster dropdown is missing entries | Vcenter-side resources may have been deleted; see §4 gotcha |
| You need to start a fresh deploy after a partial failure | Use Method 1 or 3 |

> **Skip this runbook entirely** if you only need to delete the **VM** (not the env record). For VMs alone, use vSphere Client right-click → Delete from Disk. The Fleet DB record will remain but won't block deploys until you also clean it (Method 3 or 4).

---

## 2. Pre-Cleanup Checklist

| # | Check | Command / Place |
|---|---|---|
| 1 | List current envs to find the orphan vmid | `curl -sk -u admin@local:'<pwd>' https://localhost/lcm/lcops/api/v2/environments` on Fleet, or Components page in Fleet UI |
| 2 | Capture the orphan env's `environmentId` | e.g., `db763d15-5310-4179-b9de-e3534990205a` |
| 3 | Take a snapshot of the Fleet VM | PowerCLI `New-Snapshot -VM fleet -Name 'pre-cleanup-<envId>-<date>' -Memory:$false -Quiesce:$false` |
| 4 | Note any VMs in vCenter created by the failed deploy | `Get-VM \| Where-Object { $_.Name -match 'vcfa-mgmt' }` |
| 5 | Note the resource pool name the deploy was targeting | `Get-VM <name>.ResourcePool.Name` or check Fleet's deployment-target API |

> **Take the snapshot.** Especially before Methods 2 or 4 (the more aggressive options). Fleet snapshots take <5 seconds non-quiesced and give you a hard rollback if anything goes sideways.

---

## 3. Method 1 — API DELETE with deleteFromVcenter=false (cleanest)

**Use when:** the VCFA VM either (a) was never created (deploy failed before VM creation), or (b) was already manually deleted from vCenter. You want to clean only Fleet's DB record.

```
 ENV="<env-vmid>"

cat > /tmp/del.json <<'EOF'
{
  "controllerType": "string",
  "deleteFromInventory": true,
  "deleteFromVcenter": false,
  "deleteLbFromSddc": true,
```

```
  "deleteWindowsVMs": true
}
EOF

curl -sk -u admin@local:'<pwd>' \
  -X DELETE \
  -H "Content-Type: application/json" \
  -d @/tmp/del.json \
  -w "\nHTTP %{http_code}\n" \
  "https://localhost/lcm/lcops/api/environments/$ENV/delete"
## Expected: {"requestId":"<uuid>"} + HTTP 200
```

`deleteFromVcenter: false` **is the safe choice** — Fleet's workflow does NOT touch vCenter (no VM destruction, no resource pool removal, no folder deletion). It only marks the env for DB cleanup.

***Verify completion***

```
 sleep 15
## Check env list — orphan should be gone
curl -sk -u admin@local:'<pwd>' https://localhost/lcm/lcops/api/v2/environments

## Log marker (in /var/log/vrlcm/vmware_vrlcm.log):
## "Updating the Environment request status to COMPLETED for request ID : <uuid>
##  with request type : DELETE_ENVIRONMENT."
```

If the env is gone and the log shows COMPLETED, you're done. If the API returned 200 but the env is still listed, the workflow silent-failed (see §7).

---

## 4. Method 2 — API DELETE with deleteFromVcenter=true (and the resource-pool gotcha)

**Use when:** the VCFA VM is still in vCenter and you want Fleet's workflow to destroy it along with the DB record.

```
 ## Same as Method 1 but with deleteFromVcenter=true
cat > /tmp/del.json <<'EOF'
{
  "controllerType": "string",
  "deleteFromInventory": true,
  "deleteFromVcenter": true,
  "deleteLbFromSddc": true,
  "deleteWindowsVMs": true
}
EOF

curl -sk -u admin@local:'<pwd>' \
  -X DELETE \
  -H "Content-Type: application/json" \
  -d @/tmp/del.json \
  -w "\nHTTP %{http_code}\n" \
  "https://localhost/lcm/lcops/api/environments/$ENV/delete"
```

***⚠️ Critical gotcha — Resource pool deletion***

> **Verified 2026-05-18:** `deleteFromVcenter: true` does NOT just delete the VCFA VM. It can also delete the **resource pool** the VM was placed in, even though the resource pool may have been pre-existing and used by other deploys/scripts.

In the 2026-05-18 incident, env `db763d15` was deleted with `deleteFromVcenter=true`. After the workflow completed, the resource pool `vcenter-cl01-mgmt-001` (moRef `resgroup-4014`) was **gone from vCenter** even though it was pre-existing, not auto-created by the deploy. Only the root `Resources` pool remained.

This breaks the next VCFA deploy attempt because the wizard's "Select Resource Pool" dropdown is empty.

#### *Resource pool recovery if it happened to you*

```
 ## Recreate the resource pool with the same name
New-ResourcePool -Name 'vcenter-cl01-mgmt-001' `
                 -Location (Get-Cluster vcenter-cl01) `
                 -CpuExpandableReservation:$true -CpuReservationMhz 0 -CpuSharesLevel Normal `
                 -MemExpandableReservation:$true -MemReservationGB 0 -MemSharesLevel Normal `
                 -Confirm:$false
```

The new RP will have a different MoRef but the wizard refreshes inventory each time, so it'll pick up the new one. **Always check resource pool inventory before AND after `deleteFromVcenter=true`** to catch this.

#### *When to use Method 2 anyway*

Despite the RP gotcha, Method 2 is still appropriate when: - The deploy is fully self-contained (no shared resource pools) - You don't need to preserve any auto-created vCenter inventory (folders, RPs) - You want a one-call cleanup

For anything else, use Method 1 or 3 to keep destructive scope tight.

---

### 5. Method 3 — Manual UI cleanup + API DB-only DELETE

**Use when:** you want fine-grained control over what gets deleted in vCenter, plus a clean Fleet DB.

#### *Steps*

1. **In vSphere Client:** - Right-click the VCFA VM → **Power** → **Power Off** → confirm - Wait for PowerState = PoweredOff - Right-click again → **Delete from Disk** → confirm
2. **Verify VM is gone** from vCenter inventory
3. **Run Method 1's API DELETE** (`deleteFromVcenter: false`) to clean Fleet's DB record

#### *Why this is sometimes the right answer*

| Criterion | Method 2 (auto) | Method 3 (manual) |
|---|---|---|
| Resource pool destruction risk | **Yes** | No |
| Speed | Single API call (~10 sec workflow) | 3 UI clicks + API call (~30 sec) |
| Audit trail | Single Fleet log entry | Per-step vCenter task entries |
| Reversibility | Limited (RP gone) | Higher (RP intact) |

For homelabs with carefully-curated resource pools (per your `feedback_vcfa_deploy.md` and related memory entries), Method 3 is the conservative choice.

---

## 6. Method 4 — Direct Postgres FK-ordered DELETE

**Use when:** Methods 1-3 all fail. Fleet's DELETE workflow returns HTTP 200 + requestId but the env stays in the list, no exception is logged, the request just silently transitions to FAILED (see §7 for symptom).

Per [[reference_vcf_docs_directory]] memory entry, the FK schema for vCenter LCM environments lives in your own `Fleet-Operations-Integration-Recovery-RCA.md` Appendix C. The procedure below is grounded in that schema.

### *FK tree (verbatim from Appendix C)*

```
vm_lcops_environments (vmid PK)
  ├── vm_lcops_infrastructure_properties (environment_vmid FK)
  └── vm_lcops_product (environment_vmid FK, vmid PK)
        ├── vm_lcops_product_nodes (product_vmid FK, vmid PK)
        │     └── vm_lcops_node_properties (node_vmid FK)
        └── vm_lcops_product_properties (product_vmid FK)

Non-FK references (DELETE separately):
  vm_engine_scheduledrequest (vmid LIKE, targetid LIKE)
  vm_locker_reference (envid =)
  vm_locker_certificate (alias LIKE — optional, see notes)
```

### *Postgres connection*

| | Value |
|---|---|
| Host | `127.0.0.1` (TCP) |
| Port | 5432 |
| Superuser | `postgres` / `vmware` |
| App user/DB | `vrlcm` / `vrlcm` |
| Binary | `/opt/vmware/vpostgres/current/bin/psql` |

### *Cleanup transaction (FK-ordered, ON_ERROR_STOP)*

```
ENV="<env-vmid>"

## Pre-cleanup row counts (sanity check)
su - postgres -c "/opt/vmware/vpostgres/current/bin/psql -U postgres vrlcm -At -c \"
SELECT 'env'           AS table_name, COUNT(*) FROM vm_lcops_environments              WHERE vmid='$ENV'
UNION ALL SELECT 'infra_props',     COUNT(*) FROM vm_lcops_infrastructure_properties WHERE
environment_vmid='$ENV'
UNION ALL SELECT 'product',         COUNT(*) FROM vm_lcops_product                   WHERE
environment_vmid='$ENV'
UNION ALL SELECT 'product_props',   COUNT(*) FROM vm_lcops_product_properties         WHERE product_vmid
IN (SELECT vmid FROM vm_lcops_product WHERE environment_vmid='$ENV')
UNION ALL SELECT 'product_nodes',   COUNT(*) FROM vm_lcops_product_nodes              WHERE product_vmid
IN (SELECT vmid FROM vm_lcops_product WHERE environment_vmid='$ENV')
UNION ALL SELECT 'node_props',      COUNT(*) FROM vm_lcops_node_properties            WHERE node_vmid IN
(SELECT vmid FROM vm_lcops_product_nodes WHERE product_vmid IN (SELECT vmid FROM vm_lcops_product WHERE
environment_vmid='$ENV'))
UNION ALL SELECT 'sched_request',   COUNT(*) FROM vm_engine_scheduledrequest          WHERE vmid LIKE
'%$ENV%' OR targetid LIKE '%$ENV%'
UNION ALL SELECT 'locker_ref',      COUNT(*) FROM vm_locker_reference                 WHERE envid='$ENV'
;\""

## Cleanup transaction
su - postgres -c "/opt/vmware/vpostgres/current/bin/psql -U postgres vrlcm -v ON_ERROR_STOP=1 -c \"
```

```
BEGIN;

-- 1. Leaf: node properties
DELETE FROM vm_lcops_node_properties
 WHERE node_vmid IN (
   SELECT vmid FROM vm_lcops_product_nodes
   WHERE product_vmid IN (SELECT vmid FROM vm_lcops_product WHERE environment_vmid='$ENV')
 );

-- 2. Product nodes
DELETE FROM vm_lcops_product_nodes
 WHERE product_vmid IN (SELECT vmid FROM vm_lcops_product WHERE environment_vmid='$ENV');

-- 3. Product properties
DELETE FROM vm_lcops_product_properties
 WHERE product_vmid IN (SELECT vmid FROM vm_lcops_product WHERE environment_vmid='$ENV');

-- 4. Products
DELETE FROM vm_lcops_product WHERE environment_vmid='$ENV';

-- 5. Infrastructure properties
DELETE FROM vm_lcops_infrastructure_properties WHERE environment_vmid='$ENV';

-- 6. Environment itself
DELETE FROM vm_lcops_environments WHERE vmid='$ENV';

-- 7. Non-FK references
DELETE FROM vm_locker_reference WHERE envid='$ENV';
DELETE FROM vm_engine_scheduledrequest WHERE vmid LIKE '%$ENV%' OR targetid LIKE '%$ENV%';

COMMIT;
\""

## Post-cleanup verification
su - postgres -c "/opt/vmware/vpostgres/current/bin/psql -U postgres vrlcm -At -c \"SELECT COUNT(*) FROM
vm_lcops_environments WHERE vmid='$ENV';\""
## Expected: 0
```

**Use `ON_ERROR_STOP=1`.** If any DELETE step encounters an FK violation (data drift, schema change), the transaction aborts and rolls back cleanly. Without `ON_ERROR_STOP`, you can get partial deletes that are harder to recover from.

#### *Locker cleanup (optional)*

If you also want to clean stale locker entries created by the failed deploy:

```
 ## View locker certs/passwords tied to the env's UUID prefix
su - postgres -c "/opt/vmware/vpostgres/current/bin/psql -U postgres vrlcm -At -c \"
SELECT alias FROM vm_locker_certificate WHERE alias LIKE '%$ENV%';
SELECT alias FROM vm_locker_password    WHERE alias LIKE '%$ENV%';
\""
```

Be conservative here — locker entries are sometimes shared across envs. Only delete if you're sure they're orphan.

---

### 7. Method 5 — UI Retry with Skip Task (KB 416691)

**Use when:** the UI DELETE fails with `LCMVMSP10027 – Unable to trigger request due to missing property kubeconfig`. This occurs when the deploy never got far enough to create a Kubernetes cluster, so Fleet's delete workflow can't

connect to a kubeconfig that doesn't exist.

**Broadcom reference:** KB 416691 — LCMVMSP10027 and LCMVCFA00010 are received when trying to delete a failed VCF Automation deployment

### *Steps*

1. **VCF Operations → Fleet Management → Lifecycle → VCF Management → Tasks**
2. Locate the failed Automation **removal** task (the delete that just failed with LCMVMSP10027)
3. Click the **FAILED** status link to open the task details
4. Click **Retry**
5. For each failed sub-task, set **Skip Task = true**
6. Click **Submit**

Fleet will skip the kubeconfig-dependent steps (Kubernetes cleanup, integration removal) and proceed directly to removing the DB record. Since the VM is already deleted from disk and the Kubernetes cluster was never created, there is nothing for those steps to do anyway.

### *When to use Method 5 vs Method 4*

| Criterion | Method 5 (Skip Task) | Method 4 (Postgres) |
|---|---|---|
| Broadcom-supported | **Yes** (KB 416691) | No (unsupported direct DB edit) |
| Works when VCFA VM already deleted from disk | Yes | Yes |
| Works when kubeconfig never existed | Yes | Yes |
| Cleans all DB references | Yes (via Fleet's own workflow) | Manual FK-ordered DELETE |
| Requires SSH to Fleet | No | Yes |
| Available since | Fleet 9.0.1+ | Any version |

**Recommendation:** Try Method 5 first. Fall back to Method 4 only if Skip Task itself fails.

### *Verified 2026-05-26*

Used to clean up a failed 9.0.1 VCFA deployment record ( `automation-node1.lab.local` , Deployment Failed status) on Fleet 9.0.2.0100. The VCFA bootstrap VM ( `vcfa-mgmt-c5z8t` , 96 GB) had already been powered off and deleted from disk before the UI delete was attempted — triggering the `LCMVMSP10027` kubeconfig error. Skip Task resolved the error and removed the component registration.

---

## 8. When the API DELETE Silent-Fails

Symptom (verified 2026-05-18):

```
 1. Issue DELETE /lcm/lcops/api/environments/<env>/delete
2. Receive: {"requestId":"<uuid>"} + HTTP 200 ← looks fine
3. Wait 10 seconds
4. GET /lcm/lcops/api/v2/environments → env IS STILL THERE
5. Check /var/log/vrlcm/vmware_vrlcm.log:
   - 23:42:37 INFO  Processing request <uuid> with request type DELETE_ENVIRONMENT
                    with request state INPROGRESS
   - 23:42:46 INFO  Updating request status to FAILED for request ID <uuid>
```

```
                    with request type DELETE_ENVIRONMENT
6. No ERROR/Exception lines between INPROGRESS and FAILED
```

The workflow completed its journey, marked itself FAILED, but emitted no useful diagnostic.

### *Causes observed*

| Cause | Symptom |
|---|---|
| Env is in `FAILED` state (not `COMPLETED`) when DELETE is called | DELETE workflow expects to "decommission" a healthy env; orphan-failed envs may not have the right state machine transitions |
| Underlying vCenter call fails (auth, cert, missing resource pool) | Workflow can't execute its vCenter-side steps and silently aborts |
| Stale workflow lock on the env | Previous DELETE attempt didn't fully release the env, blocks subsequent attempts |

### *What to do*

- **Skip directly to Method 4** (direct Postgres). Don't waste cycles iterating on Methods 1-3.
- If Method 4 itself fails (rare — would indicate data corruption), open a Broadcom support case with the env UUID, the FK row counts before/after, and the snapshot timestamp.

---

## 8. Verification Matrix

After any cleanup method completes:

| Check | Expected | Command |
|---|---|---|
| Fleet API env list | Orphan env absent | `curl /lcm/lcops/api/v2/environments` |
| Individual env GET | HTTP 404 | `curl /lcm/lcops/api/v2/environments/<vmid>` |
| Fleet DB env count | 0 | `psql -c "SELECT COUNT(*) FROM vm_lcops_environments WHERE vmid='<vmid>'"` |
| Fleet DB FK children | All 0 | (see §6's pre-cleanup query for the seven counts) |
| vCenter VM (if Method 2/3) | Removed | `Get-VM \| Where-Object { $_.Name -match '<pattern>' }` |
| vCenter resource pool (if Method 2 was used) | **Verify still present** — see §4 gotcha | `Get-ResourcePool -Name <name>` |
| Wizard "Add Component → VCFA" | Doesn't error with "parallel deployment not allowed" | Manual test in Fleet UI |

---

## 9. Rollback

For Methods 1-3 (API + UI), rollback is limited because the changes write to multiple stores (Fleet DB, vCenter, locker). The Fleet snapshot from §2 is the best recovery — revert via PowerCLI:

```
Set-VM -VM fleet `
       -Snapshot (Get-Snapshot -VM fleet -Name 'pre-cleanup-<envId>-*') `
       -Confirm:$false
```

For Method 4 (direct Postgres), the `ON_ERROR_STOP` transaction means the cleanup is atomic — either fully applied or fully aborted. If you decide post-COMMIT that the cleanup was wrong, only the snapshot can revert. There is no Postgres-level undo.

> **The 2026-05-18 incident** used Method 4 successfully. Snapshot `pre-db-cleanup-db763d15-2026-05-18` was created before the transaction, never needed for revert. Keep it around for at least a few hours after a successful cleanup as insurance against discovering downstream issues.

---

## 10. Incident Log — 2026-05-18

Today's failed VCFA deploy cleanup used Method 4 because Methods 1 and 2 both silent-failed.

| Step | Outcome |
|---|---|
| Initial deploy of env `db763d15-5310-4179-b9de-e3534990205a` | FAILED at LCMVSPHERECONFIG1000095 (internal K8s bootstrap timeout) |
| Method 2 attempt (DELETE with deleteFromVcenter=true) | HTTP 200 + requestId returned, but workflow silent-failed. **Resource pool `vcenter-c101-mgmt-001` (resgroup-4014) was deleted as side effect** — see §4. |
| Method 1 attempt (DELETE with deleteFromVcenter=false) | Same silent-failure pattern, env stays in list |
| Manual VM cleanup (vSphere Client Power Off + Delete from Disk) | Worked — VM gone |
| Method 1 retry (deleteFromVcenter=false after VM gone) | Same silent-failure, env still listed |
| **Method 4 (direct Postgres FK-ordered DELETE)** | **Succeeded** — see row counts below |
| Resource pool recreation via PowerCLI | New MoRef `resgroup-2007` |
| Fleet snapshot before Method 4 | `pre-db-cleanup-db763d15-2026-05-18` |

### *Row counts (verified by `psql`)*

| Table | Before | After |
|---|---|---|
| vm_lcops_environments | 1 | 0 |
| vm_lcops_infrastructure_properties | 42 | 0 |
| vm_lcops_product | 2 | 0 |
| vm_lcops_product_properties | 24 | 0 |
| vm_lcops_product_nodes | 2 | 0 |
| vm_lcops_node_properties | 34 | 0 |
| vm_engine_scheduledrequest | 0 | 0 |

| Table | Before | After |
|---|---|---|
| vm_locker_reference | 0 | 0 |

Transaction completed cleanly. Fleet API list returned only `97dee43f` (operations env) afterward.

---

## 12. Pre-Redeploy Gotchas (Do These Before Retrying)

After cleaning up a failed VCFA deployment, the following items must be verified before submitting a new deploy. Each was discovered through failed retry attempts and is not covered in a single Broadcom document.

### *12.1 CNS Volume Reconciliation (KB 320790)*

**When:** After any failed VCFA deploy that got far enough to create persistent volumes (reached `vmsp-platform-bootstrap` or later).

**Why:** The failed deploy creates CNS volumes (vSAN-backed VMDKs) for Kubernetes persistent volume claims. When Fleet tears down the VM, it deletes the VMDKs but the CNS registration in vCenter's database may not be cleaned up. The next deploy's CSI driver creates new VMDKs but vCenter's CNS component fails to attach them with:

```
failed to attach cns volume: "..." Volume with ID ... is not registered as CNS Volume.
```

This causes Prometheus, seaweedfs, and other stateful pods to fail, eventually crashing the K8s API and killing the deploy.

**Fix — run before retrying:**

```
 Connect-VIServer -Server vcenter.lab.local -User administrator@vsphere.local -Password '<password>' -
Force
$ds = Get-Datastore 'vcenter-cl01-ds-vsan01'
$si = Get-View ServiceInstance
$vsom = Get-View $si.Content.VStorageObjectManager
$task = $vsom.ReconcileDatastoreInventory_Task($ds.ExtensionData.MoRef)
## Wait for task to complete — typically 2-5 minutes
```

**Broadcom reference:** KB 320790

**Verified:** 2026-05-27. VCFA 9.0.2 deploy failed at `vmsp-platform-extra` with Prometheus PVC attach failure. Reconciliation resolved it.

---

### *12.2 Containerd Restart on Fleet (KB 425929)*

**When:** After patching Fleet Management from 9.0.x to 9.0.2.

**Why:** The Fleet patch does not restart `containerd`. The stale state causes the VCFA bootstrap to fail immediately at `INIT0002` with `failed to create task for container: failed to start shim`.

**Fix:** `systemctl restart containerd` on Fleet.

**Broadcom reference:** KB 425929

**Verified:** 2026-05-27.

---

### *12.3 Fleet Java Truststore After Upgrade*

**When:** After upgrading Fleet Management to a new version.

**Why:** The upgrade may reset the Java truststore, removing trust for SDDC Manager and vCenter CA certificates. Fleet loops on pre-flight tasks with `VCFAPITokenRetrivalException` — no error surfaced to the UI, the deploy just never advances.

**Fix:**

```
 ## SSH to Fleet
echo | openssl s_client -connect sddc-manager.lab.local:443 2>/dev/null | openssl x509 -out /tmp/sddc-
mgr.crt
curl -sk "https://vcenter.lab.local/afd/vecs/ca" -o /tmp/vcenter-ca.crt
keytool -importcert -keystore /usr/java/jre-vmware/lib/security/cacerts -storepass changeit -alias sddc-
manager -file /tmp/sddc-mgr.crt -noprompt
keytool -importcert -keystore /usr/java/jre-vmware/lib/security/cacerts -storepass changeit -alias
vcenter-ca -file /tmp/vcenter-ca.crt -noprompt
systemctl restart vrlcm-server
```

**Verified:** 2026-05-27. Fleet looped 9+ hours on pre-flight. Trust import resolved it.

---

### *12.4 SDDC Manager Must Be Running*

**When:** Always, during any VCFA deploy via Fleet.

**Why:** Fleet requires a VCF API token from SDDC Manager. If SDDC Manager is powered off, the workflow loops indefinitely with no useful error.

**Fix:** Power on SDDC Manager before submitting the deploy.

**Verified:** 2026-05-27.

---

### *12.5 API Deploy JSON — Infrastructure vCenter Properties Required (Fleet 9.0.2+)*

**When:** Deploying VCFA via the Fleet REST API on Fleet 9.0.2+.

**Why:** Fleet 9.0.2 added a null check requiring `vCenterHost`, `vCenterName`, `vcUsername`, `vcPassword` in infrastructure properties. Missing these causes a silent `NullPointerException` in the `CreateEnvironmentPlanner` — the deploy stays INPROGRESS but the workflow never starts.

**Fix:** Include vCenter details in `infrastructure.properties`. `vcUsername` must be a plain string, not a locker reference.

**Verified:** 2026-05-27.

---

### *12.6 Pre-Redeploy Checklist*

| # | Check | Expected |
|---|---|---|
| 1 | Fleet environments clean (only COMPLETED) | `psql: SELECT vmid, status FROM vm_lcops_environments;` |
| 2 | No INPROGRESS requests | `psql: SELECT vmid FROM vm_rs_request WHERE state='INPROGRES S';` → empty |

| # | Check | Expected |
|---|---|---|
| 3 | Containerd running (if Fleet was patched) | `systemctl is-active containerd` → active |
| 4 | SDDC Manager running | PoweredOn |
| 5 | CNS reconciled (if prior deploy reached K8s stage) | `ReconcileDatastoreInventory_Task` → 100% |
| 6 | Fleet trust certs valid | No recent `VCFAPITokenRetrival` errors in log |
| 7 | Depot reachable from Fleet | `curl -sk https://192.168.1.160:8443/` → HTTP 200 |
| 8 | vSAN healthy, 0 resync | `esxcli vsan debug object health summary get` |
| 9 | Target host has 96+ GB free | esxi04 free memory |
| 10 | William Lam tunings correct (5/5) | 0,0,1,1,1 on all hosts |

---

## 13. Lessons Learned

**1.** `deleteFromVcenter: true` **can destroy more than just the VM.** Verified 2026-05-18: it also removed the resource pool `vcenter-cl01-mgmt-001` . Always default to `deleteFromVcenter: false` and do manual VM cleanup separately.

**2. Method 1 (DELETE deleteFromVcenter=false) is the right starting point, BUT it can silent-fail.** If the env state is `FAILED` (not `COMPLETED` ) when you call DELETE, Fleet's workflow doesn't complain — it just marks the request FAILED and leaves the env in place. No useful log diagnostic.

**3. Direct Postgres (Method 4) is the reliable fallback.** Your own `Fleet-Operations-Integration-Recovery-RCA.md` Appendix C documents the FK tree. Following it with `ON_ERROR_STOP=1` is safe and atomic.

**4. Always snapshot Fleet before Method 4.** Postgres has no undo for committed DELETEs.

**5. The** `controllerType: "string"` **placeholder in the DELETE payload is verbatim from Broadcom KB 402063.** Don't substitute — Fleet ignores the field but rejects unexpected structures.

**6. After Method 2 or 3, verify the resource pool still exists.** This is now a checklist item, not a separate doc — built into §8 verification matrix.

**7. The DELETE workflow does NOT remove locker references unless you delete them yourself.** Locker entries for the env's certs/passwords stick around in the locker after a Method 4 cleanup. Optional sweep in §6 sub-section.

---

## 14. Document Information

| Field | Value |
|---|---|
| Title | Failed VCFA Deployment Cleanup Runbook |

| Field | Value |
|---|---|
| Document ID | VC-RUNBOOK-VCFA-CLEANUP-20260518 |
| Version | 1.0 |
| Issued | May 18, 2026 |
| Author | Virtual Control LLC |
| Classification | Internal — Lab Environment |
| Related Documents | `Fleet-vCenter-Trust-Cascade-Repair-2026-05-18.md`, `VCFA-Simple-Deploy-Wizard-Precheck-Gotchas-2026-05-18.md`, `Nested-ESXi-RAM-Rebalance-for-VCFA-Deploy-2026-05-18.md`, `Fleet-Operations-Integration-Recovery-RCA.md` (the Appendix C source for Method 4) |
| Distribution | Lab notebook |

---

## Document Information

| Field | Value |
|---|---|
| Document ID | VC-VCFA-DEPLOY-COMPENDIUM |
| Version | 1.0 |
| Issued | June 16, 2026 |
| Author | Virtual Control LLC |
| Classification | Internal |

**Consolidates (verbatim):** VCF_Automation_Deployment_Guide · VCFA-Deploy-Airtight-Plan-2026-05-12 · Nested-ESXi-RAM-Rebalance-for-VCFA-Deploy-2026-05-18 · VCFA-PreDeploy-Cleanup-CertReplace-2026-05-16 · VCFA-Simple-Deploy-Wizard-Precheck-Gotchas-2026-05-18 · VCFA-Deployment-Fix-Runbook-2026-05-08 · VCFA-Deploy-Monitoring-Checks-2026-05-20 · VCF-Automation-Deployment-Troubleshooting-Report · Failed-VCFA-Deployment-Cleanup-Runbook-2026-05-18

### Revision History

| Version | Date | Notes |
|---|---|---|
| 1.0 | 2026-06-16 | Consolidated nine VCFA 9.0.1 deployment & troubleshooting documents into one compendium — verbatim chapters under a unified cover + master Table of Contents; no technical content altered. Source documents retained pending review. |

**Document End**